

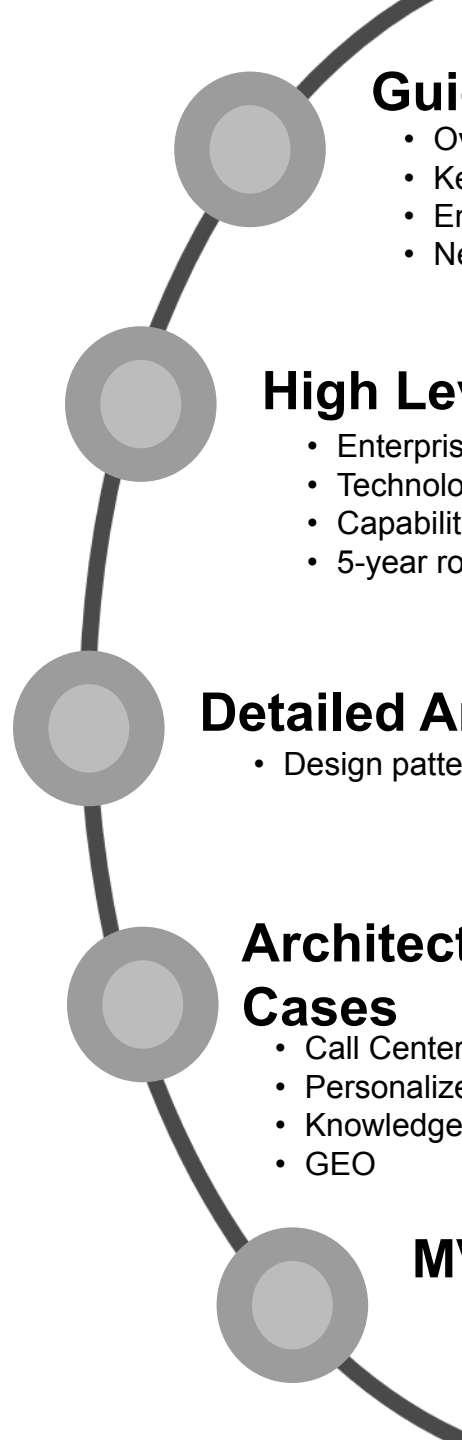


Agentic Architecture

Enterprise Agentic AI Platform Architecture Blueprint

February 2026





Guiding Principles

- Overarching Costco principles
- Key imperatives driving Agentic AI in the enterprise
- Enterprise Agentic AI principles
- Nexus Architecture

High Level Architecture

- Enterprise Agentic AI platform capabilities
- Technology considerations at each capability layer
- Capability design principles
- 5-year roadmap

Detailed Architecture

- Design patterns catalog

Architecture Recommendations for Priority Use Cases

- Call Center
- Personalized Search
- Knowledge Assist
- GEO

MVP Scoping & Implementation Plan

- MVP capabilities
- 6 month MVP implementation plan



Guiding Principles

Overarching Costco Guiding Principles

Informs agentic AI operating and architecture principles

Source	Guiding Principle
Enterprise Architecture	Business and IT Alignment with Measurable Value
	Customer-Centric Design
	Security, Compliance, and Privacy by Design
	Simplicity and Scalability
	Modular and API-Driven Architecture
	Reuse Over Build or Buy
	Global Availability and Resilience
	Data-Driven Decision Making
	Adaptive Governance
	Innovation and Continuous Improvement & Automation
AI Architecture	Lead from the Top
	Responsible development & deployment
	Composable AI Architecture with GCP First
	Interoperability
	Empower the Workforce
	Partner for Acceleration



Key imperatives driving Enterprise AI

1

Take a platform-based approach to maximize reusability & consistency of agentic architecture



2

Prioritize ecosystem interoperability to avoid siloed developments



3

Leverage pre-built agents to accelerate time-to-value



4

Drive persona-based adoption with tailored experiences across user types



5

Embed governance and evaluation frameworks to ensure trustworthy agents



Agentic AI Architecture Design Principles



Agentic AI Architecture Guiding Principles

1

Knowledge-first approach for context engineering

- Semantic data modeling
- Context isolation
- High quality data preparation and normalization
- Continuous knowledge governance and lifecycle Mgmt..

2

Federated deployment, centralized governance

- Domain autonomy, Platform consistency
- Shared reference architecture with local extensions
- Common Guardrails, enforced through a central policy
- Unified Agent registry and identity
- Automated deployment tooling

3

Standards driven and governance by design

- Standardized agent lifecycle management and certification
- Governance embedded into workflows
- Standardized interfaces and protocols
- Global safety and risk framework
- Unified observability, telemetry and auditability

4

Composable design to adapt to rapid innovation

- Service-Oriented design approach
- Loose coupling via abstractions
- Declarative orchestration

5

Design for elasticity and scaling for high-volume processing

- Elastic, on-demand orchestration
- Resilient and fault tolerant execution
- Inferencing through request batching
- High throughput, low latency

6

High-performance, safety-first agent operations

- Standardized red teaming and AI judge framework
- Tunable agent reasoning levels based on task complexity
- Defensive UI for agentic experience
- Network isolation

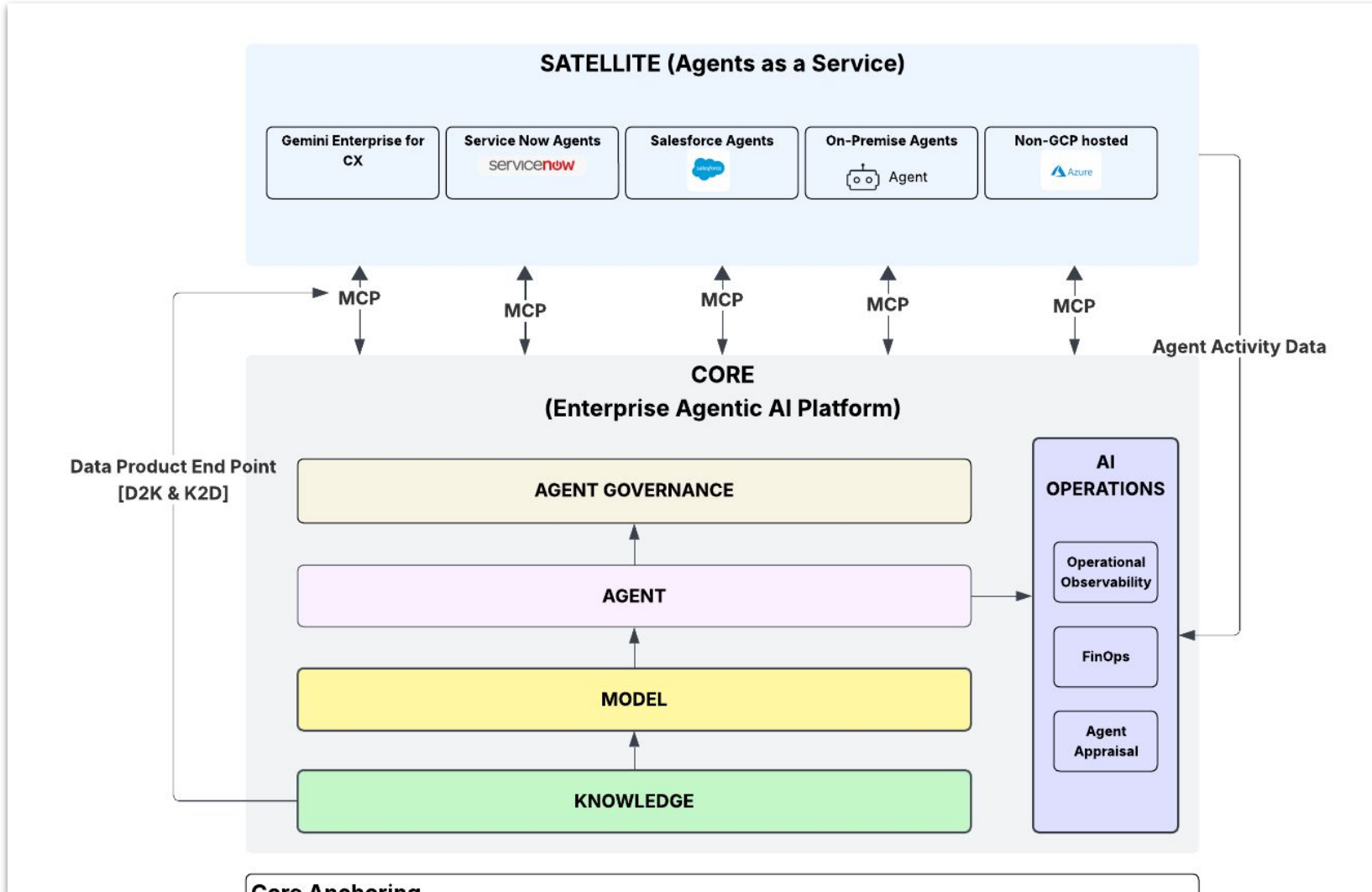
7

Cost efficient by design

- Right sized models and adaptive routing
- FinOps by design: Cost visibility and guardrails
- Operational simplification through platform consolidation
- Semantic caching



Agents will be federated in an organization



Core Anchoring

The core represents the solutions developed as foundational and differentiated capabilities of the organization, which will primarily include the knowledge layer, utility agents, Centralized governance and AI operations

Satellite Autonomy

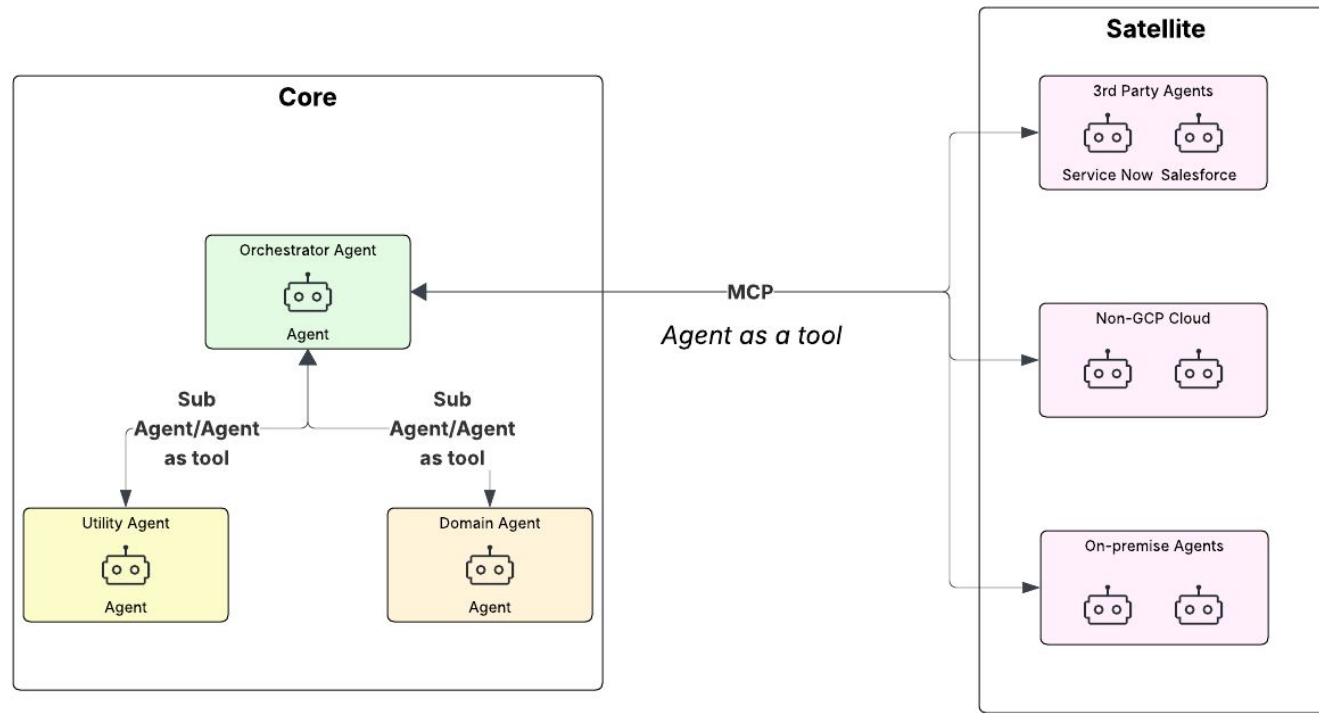
Satellites represent the non-differentiated or commodity agentic capabilities that are developed by ecosystem products (*Salesforce, SAP, ServiceNow*) and solutions for faster time to market. Satellites enables “agents as service” where agents stay close to the data, process and experience affinity.

The core enables **centralized governance** for both differentiated custom as well as commodity agents

[Go to Lucidchart](#)



Agents will leverage communication protocols



Within the platform, we will leverage Sub Agent or Agent as tool pattern for inter agent communication. For 3rd party agents, we will leverage MCP and Agent as tool pattern for agent communication. A2A and other emerging patterns will be explored based on applicability and maturity of the protocols

Agent Interaction: Agent2Agent Protocol (A2A)

- Remote Agents publish an Agent Card describing the agent's capabilities/skills and authentication mechanism
- Communication between a Client and a Remote Agent is oriented towards Task completion
- Tasks are used to transmit Artifacts (results) and Messages (thoughts, instructions, anything else)
- Agents accomplish tasks for end-users without sharing memory, thoughts, or tools

Agent as Tool : Model Context Protocol (MCP)

- 3rd party agents which may not be A2A compliant can be exposed as tools
- A2A compliant agents connect to those agents through MCP protocol
- MCP protocol also connects Agents to Tools, Resources, and Prompts
- Standard Client-Server architecture built upon structured messages with well-defined schemas





Enterprise Agentic AI Platform Capabilities

AI Capabilities View

1

AI STRATEGY & TECH BUSINESS MANAGEMENT

Establishes AI technology strategy and standards and governs investments and organizational behavior to ensure alignment with organizational priorities and responsible behaviors.

2

SOLUTION DELIVERY & MANAGEMENT

Designs, delivers, and manages AI use cases end-to-end, ensuring solutions are built, deployed, and continuously improved to deliver measurable business value.

3

AI PLATFORM

Creates shared enterprise platform that supplies the core orchestration, tooling, integration, technical governance, and operational services needed to enable AI agents.

4

DATA & ANALYTICS

Delivers and governs high-quality, trusted data to power AI agents, while providing data and analytics capabilities that inform and continuously shape the enterprise AI strategy.

5

INFRASTRUCTURE, OPERATIONS, & SECURITY

Provides resilient, secure, and optimized cloud and infrastructure services to continuously run AI solutions responsibly across the enterprise



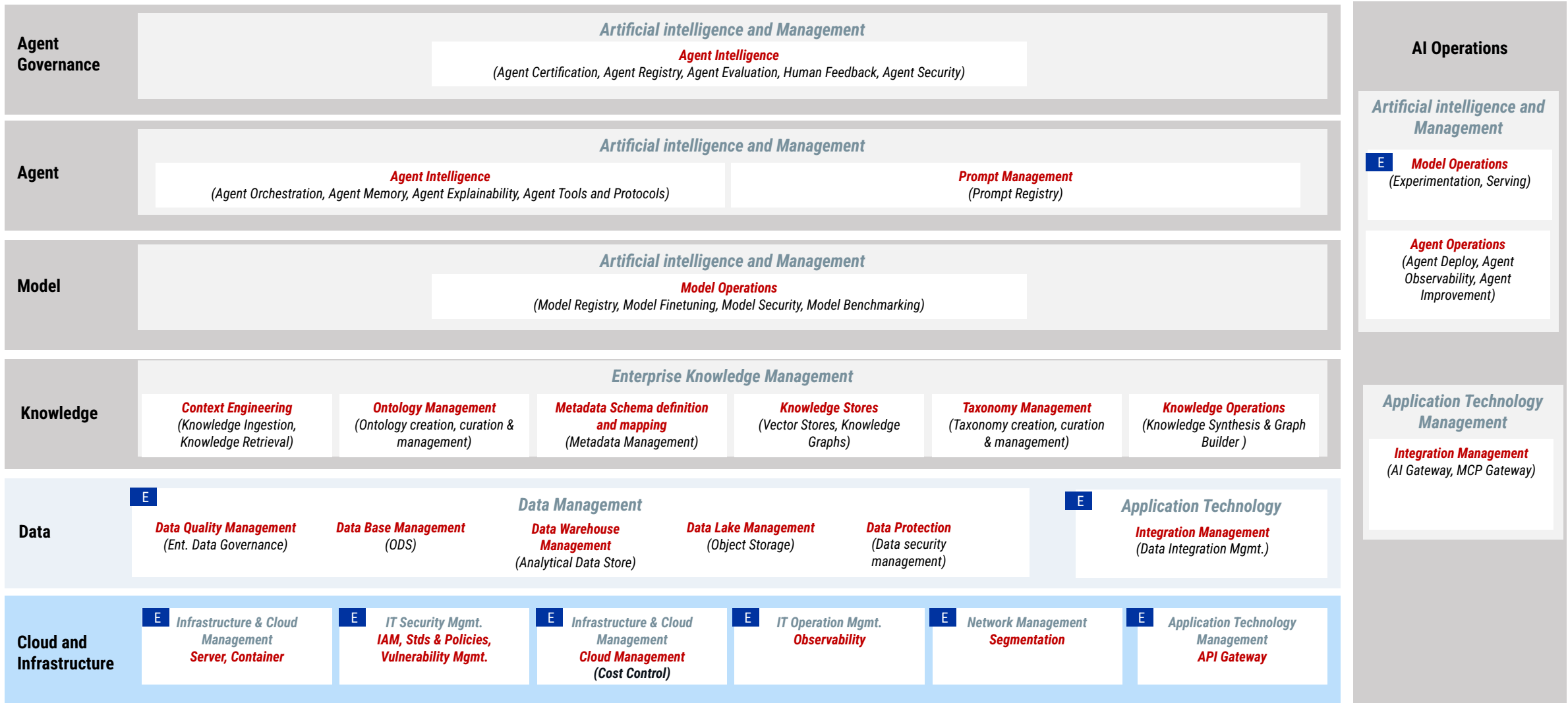
CLOUD & INFRA

PLATFORM

DATA

Agentic AI Platform Capabilities - Level 1/2

*Existing capabilities will need enhancements during use case enablement



Level 1 Capability

Level 2 Capability

EXISTING CAPABILITY

CLOUD & INFRA

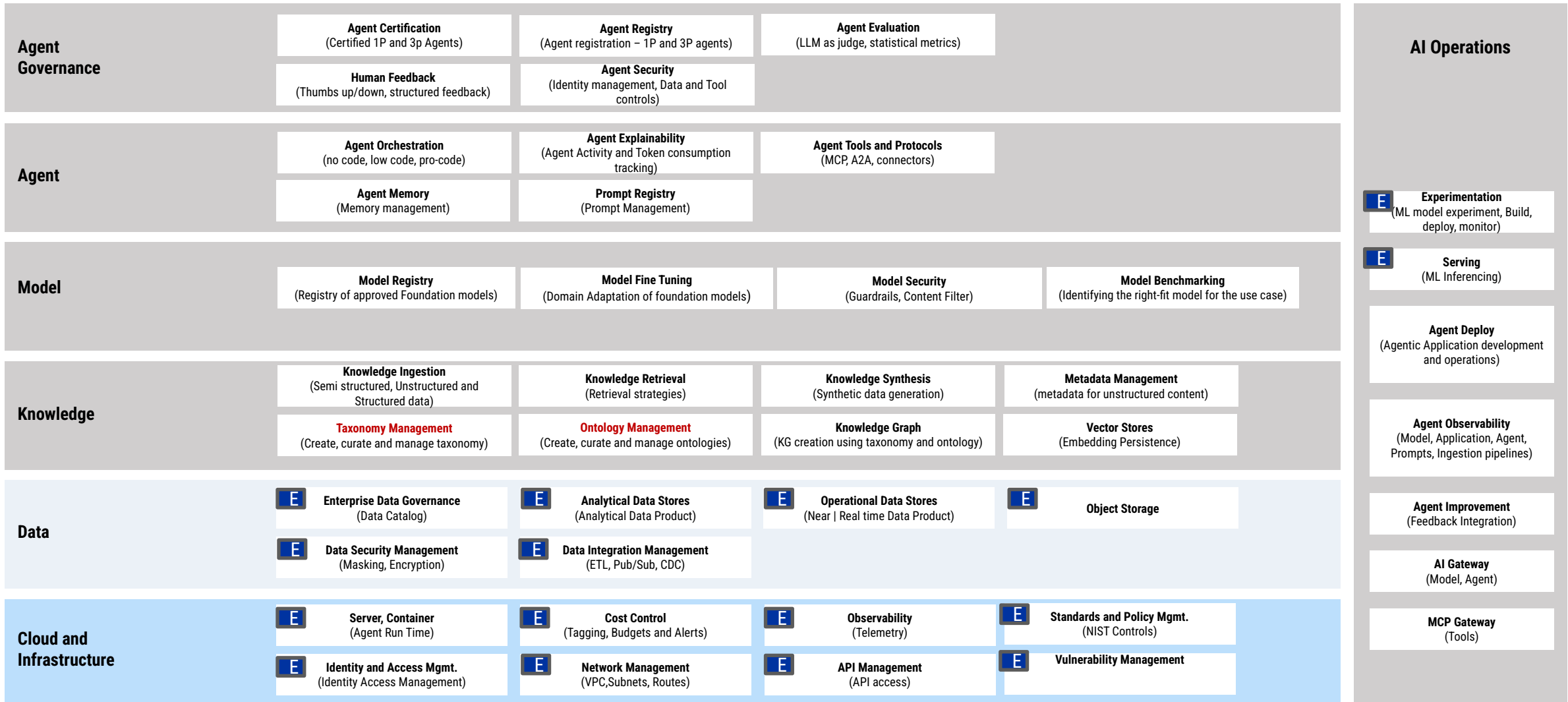
PLATFORM

DATA



Agentic AI Platform Capabilities - Level 3

*Existing capabilities will need enhancements during use case enablement



Level 2 Capability

Level 3 Capability

EXISTING CAPABILITY

CLOUD & INFRA

PLATFORM

DATA

Agentic AI Platform - Layer Maturity Stages

Knowledge

Model

Agent and Agent Governance

AI operations

Optimized(O)

- Rich semantic modeling leveraging ontology and taxonomy management tools
- Knowledge Layer acting as the central knowledge index for the organization

- Dynamic model routing
- Capability established for adapting foundational models for costco context

- Agent certification process has been established
- Agent onboarding is enabled through the certification process and agent registry
- Agents in the organization are discoverable through Agent registry

- Correlation of logs and agent activity traces to proactively identify agent activity anomalies
- Ability to apply kill-switch to proactively suspend an agent

Defined(D)

- Standard patterns established for ingestion and retrieval
- Automated and scalable ingestion and retrieval pipelines
- Unified knowledge serving layer with documented design choices
- Governance is in place for metadata curation, PII tagging

- Process established for onboarding models
- Unified model access layer via model gateway
- Monitoring and security controls applied

- Agent orchestration patterns are defined
- Agent activity is collected and analyzed via OTEL compliant solution
- Realization of Nexus architecture to orchestrate across 1st party and 3rd party agents

- A central control plane exists to monitor and track model and tool access by the agents
- Prompt and Tool deployment is integrated with CI/CD

Initial(I)

- No reference architecture or repeatable patterns
- Adhoc ingestion and retrieval
- Little to no governance: weak metadata and access controls

- No consistent model access layer via model gateway
- No model usage observability
- Minimal model security controls

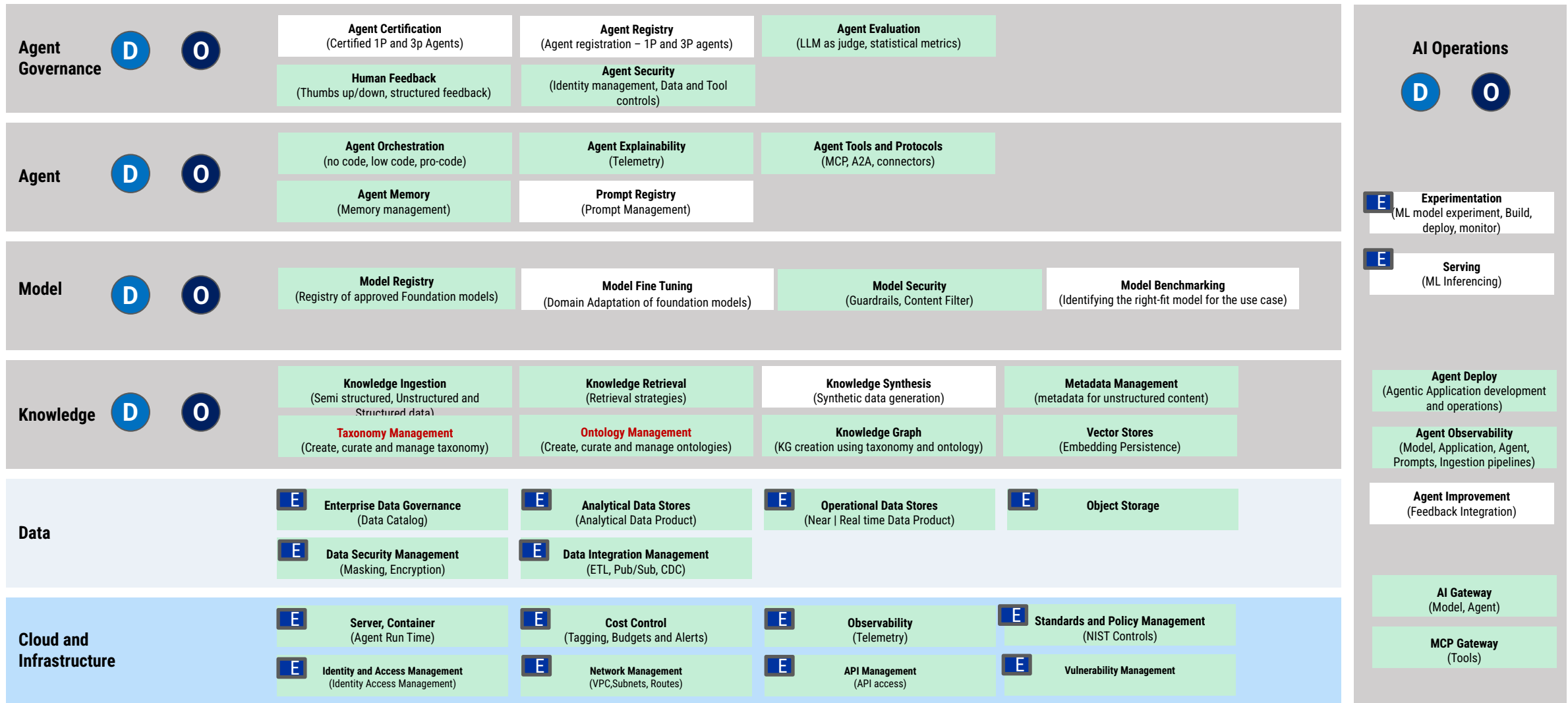
- Agent orchestration patterns are not defined
- Capability to collect and analyze agent activity not enabled
- Memory management patterns are not defined

- Limited visibility into Agent Activity
- No cost tracking of the agent activity
- No central control plane for model and tool access



Agentic AI Platform Capabilities - MVP

*Existing capabilities will need enhancements during use case enablement



Level 2 Capability

Level 3 Capability

EXISTING CAPABILITY

CLOUD & INFRA

PLATFORM

DATA

MVP Level 3 Capabilities



MVP



Target State

D - Defined

O - Optimized



Technology Considerations at each layer

Knowledge Layer | Technology Decisions (1/5)

Knowledge Ingestion

Scalable capabilities to process semi-structured, unstructured and structured enterprise data (documents, images, audio, video, relational data). Includes modules for extraction of key entities, metadata and data classification tags, chunking for embeddings, and enrichment for downstream retrieval.

GCP Services

1. Gemini Enterprise
2. Vector Search
3. Alloy DB
4. Cloud run



Non-GCP services

1. None

Knowledge Retrieval

Ability to optimize the search space by combining one or more retrieval/reranking strategy to identify the most optimal and relevant context to be passed to the language model for response generation

GCP Services

1. Gemini Enterprise
2. GKE
3. Redis Cache/Cloud memory store



Non-GCP services

1. None

Knowledge Synthesis

Services for generating, validating, and integrating synthetic data to support prompt tuning, scenario generation, and evaluation of agent behaviors. Provides broad coverage of diverse data types, including edge-case and safety scenarios.

GCP Services

1. None

Non-GCP services

1. Python
2. RAGAS/DeepEvals

Metadata Management

Defines, organizes, and governs metadata across knowledge assets. Covers data access rules, categories, timestamps, lineage, and quality attributes. Enables retrieval filtering and context isolation by supplying high-precision descriptors.

GCP Services

1. Dataplex



Non-GCP services

1. None

Taxonomy Management

Taxonomy is a structured classification system that organizes knowledge into categories, hierarchies, and relationships. Creates a consistent vocabulary that both humans and AI models can interpret reliably.

GCP Services

1. Dataplex



Non-GCP services

1. UI and Backend

Ontology Management

Ontology is a semantic representation of the business domain capturing entities, attributes, relationships, constraints, and interactions. Provides LLMs and agents with structural understanding of the domain to improve grounding and reasoning.

GCP Services

1. Alloy DB
2. Firestore (Optional)

Non-GCP services

1. UI and Backend

Knowledge Graph

A knowledge graph is a dynamic representation of knowledge that models the concepts within a particular domain and the relationships between those concepts. It acts as the digital brain of the AI agent.

GCP Services

1. None

Non-GCP services

1. Neo4J
2. UI and Backend



[Assessment](#)

Vector Store

Vector stores are specialized databases for storing and searching high-dimensional numerical representations of data, enabling AI systems to find semantically similar items.

GCP Services

1. Alloy DB

Non-GCP services

1. None



Model Layer | Technology Decisions (2/5)

Model Registry ✱

Model registry refers to the set of approved models from different providers which are approved to be used by the organization. The models in this registry are exposed via the AI gateway. Model registry provides scoped access to the approved models.

GCP Services	Non-GCP services
APIGEE	1. Kong 2. LiteLLM

Model Fine Tuning

Capability to train or adapt foundation models with domain-specific Costco data so the model internalizes the vocabulary, semantics, and constraints of the problem space.

GCP Services	Non-GCP services
Vertex AI Fine Tuning	None

Model Security

Mechanisms to enforce safety constraints, prohibited topics, refusal behavior, and output filtering at the model level

GCP Services	Non-GCP services
Model Armor	None

Model Benchmarking

Suite of services for testing and evaluating base models and custom models against a well- defined set of metrics and create benchmarks for business related functional areas.

GCP Services	Non-GCP services
Vertex AI Evaluation Service	1. Front end and backend service



Agent Layer | Technology Decisions (3/5)

Agent Orchestration

A highly customizable, low-code and pro-code , scalable framework with chain-of-thought reasoning, dynamic task decomposition and management. Agents are designed to collaborate seamlessly through an integrated memory system, and multi-agent collaboration is facilitated via a 3-layer orchestrator/super/utility agent topology.

GCP Services	Non-GCP services
1. Vertex AI Agent Engine	None
2. Google ADK	

Agent Explainability

The continuous stream of spans and traces that captures agent interactions, prompts, tool usage, latency, cost, errors and action outcomes from execution of the agentic flows providing observability into agent execution.

GCP Services	Non-GCP services
1. Cloud Trace	
2. Cloud Logging	
3. Cloud Monitoring	

Agent Tools and Protocols *

Agent tools and connectors provides pre-built services that allows the agents to integrate securely to enterprise data and systems ecosystem (CRM, ERP, ITSM tool etc.)

GCP Services	Non-GCP services
APIGEE	1. Kong
	2. LiteLLM

Agent Memory

A secure, governed and persistent layer that allows agents to store specific episodes of its interactions which can later be retrieved so that it can learn from past interactions. It stores key facts, preferences, actions an outcomes – across semantic, episodic and entity dimensions

GCP Services	Non-GCP services
Vertex AI Agent Engine Memory Bank	1. Langmem 2. mem0

All options will be available through a memory abstraction

Prompt Registry

A prompt registry is a centralized, version-controlled catalog where all prompt templates are managed and stored. It ensures that prompts are treated as first class artifacts, which are reviewed, tested, tagged and versioned. It becomes the single source of truth that tracks the evolution of prompts as they mature through review, testing and optimization

GCP Services	Non-GCP services
Vertex AI Prompt Management	GitHub/CICD



* Technology Decision | POC & KAD

GCP Native

Build

3rd party



Agent Governance | Technology Decisions (4/5)

Agent Certification

Agent certification is the process of assessing agents against a capability maturity and agent readiness dimension. The capability maturity define the autonomy/agency level of the agent, and the readiness dimension is measured by security, effectiveness and interoperability aspects.

GCP Services

None

Non-GCP services

Custom Developed
(Python + REACT)

Agent Registry

Agent registry is SOR for all certified agents-capturing their identity, owner, purpose, versions, allowed tools/data, and policy constraints. Each agent is documented through a A2A compliant Agent Card, a standardized profile that captures the agent skills, capabilities, its hosting location etc.

GCP Services

None

Non-GCP services

Custom Developed
(Python + REACT)

Agent Evaluation

Measurement systems that evaluate how well an agent reasons, retrieves, and acts. Evaluation ensures continuous reliability and tracks drift over time

GCP Services

Vertex AI Evaluation

Non-GCP services

1. RAGAS
2. Trulens

All options will be available through a abstraction

Human Feedback

A structured human-in-the-loop (HITL) mechanism that guides agent behavior toward safe, aligned outcomes. Human input—approvals, feedback, corrections, and reinforcement

GCP Services

None

Non-GCP services

Custom Developed
(Python + REACT)

Agent Security

Agent security ensures every AI Agent treated like non-human entity with strong control over what it can access and what it can do. It starts with agent identity - each agent has a unique, verifiable identity used for authentication, authorization, and full audit logging of actions and tool calls.

GCP Services

Vertex AI Agent Engine
Identity

Non-GCP services

None



AI Operations | Technology Decisions (5/5)

AI Gateway

AI Gateway is a centralized control plane that sits between agent applications, model providers and MCP servers. It enforces governance and operations at runtime - authentication, rate limits, policy checks, logging/tracing and spend/budget controls. It standardizes access to model and context which makes cost, security and reliability manageable at scale. In addition, it also enables semantic caching and usage analytics

GCP Services

APIGEE
[Assessment](#)

Non-GCP services

1. Kong
2. LiteLLM

MCP Gateway

A control plane or a proxy layer that manages how agents can securely access tools, data, and other resources through the MCP servers. It acts as the policy enforcing middle layer – validating requests, brokering capabilities, and ensuring that every tool invocation follows enterprise rules around safety, observability and authorization

GCP Services

APIGEE

Non-GCP services

1. Kong
2. LiteLLM

Agent Deploy

Agent Deploy enhances traditional DevOps practices by adding or enhancing checks and balances that are unique to developing Agentic systems. This includes adding prompt scanning, MCP tool scanning to the pipeline

GCP Services

Non-GCP services

GHEC, Github Actions

Agent Observability

Ability to collect, analyze and observe how agents behave in production. It captures end-to-end telemetry across agent runs, model calls, and tool interactions-including latency, errors, quality signals, and cost

GCP Services

1. GCS
2. AlloyDB

Non-GCP services

1. Arize
2. Dynatrace
3. Grafana

[Assessment](#)

Agent Improvement

Continuous cycle of making agents more accurate, safe, and cost efficient based on real production signals. It uses evaluations, user/human feedback to facilitate reinforcement learning for continuous agent improvement

GCP Services

Vertex AI Finetuning

Non-GCP services

None



★ Technology Decision | POC & KAD

GCP Native

Build

3rd party





Capability Design Principles

Knowledge Layer | Design Principles

Controlled Vocabulary

Agentic Systems need to operate on a controlled vocabulary – a standardized, domain-aligned set of terms, definitions and semantic boundaries that describes the concepts, relationships and constraints of the concepts for a particular problem domain

Small World Approach

Agents need to operate on scoped world models – compact, goal-aligned representations of the specific problem space they are designed to solve-rather than relying on a large, all-encompassing world model.

Metadata curation/management

As part of context engineering, metadata must be curated, standardized and governed across knowledge assets with “domain experts(knowledge managers)” in loop.

Context Isolation and Retrieval

Once the scoped world model is defined, the retrieval layer must be designed to deliver precise, high-relevance context to the agents – selecting and tuning retrieval mechanisms that best match the domain and the knowledge structure. This layer will also enable a controlled Knowledge caching - enforcing TTLs, provenance, and invalidation rules so cached context remains accurate, auditable, and aligned to the domain’s knowledge structure

Context Shaping

The context served to the agents must be shaped-filtered, structured and reformatted, so that the final context window reflects only the most relevant tools, conversation history, memory fragments. The context window should not be overloaded with information that may cause context poisoning

Knowledge Management LC

Enterprise knowledge(a major portion is unstructured) need to follow a knowledge management lifecycle – transforming human aligned content into AI aligned assets through systematic curation, metadata enrichment, normalization and context-overlap reduction before making it available for agentic systems

Incremental Knowledge Ingestion

While developing Agentic systems, it is critical to design for incremental knowledge. The system must intercept change triggers and re-ingest the modified knowledge into vector stores. It is also important to log and version such changes for auditability

Evaluation and Guardrails

Agentic systems must include guardrails, evaluation pipelines, and human/automated feedback loops to continuously refine reasoning, detect drift, and enforce safety & compliance.

Additional Details



Model Layer | Design Principles

Model selection based on use case

Design must consider model routing that routes tasks to the most appropriate model based on domain, task complexity, language, and performance constraints.

Model inference acceleration

Uses quantization, batching to reduce latency and compute cost.

Model distillation

Compresses large models into smaller ones without major loss in capability, improving performance and ability to deploy

Quotas and rate limits

A governance layer that enforces execution boundaries such as quotas, rate limits, refusal behavior, and policy controls to keep model usage predictable, compliant, and safe.

Benchmarking recipes

A structured evaluation framework with standardized metrics, datasets, and scorecards to assess model quality, performance, and safety before deployment

Additional Details



Agent Layer | Design Principles (1/2)

Role Based Agents

When translating a traditional workflow into an agentic flow:
Agent boundaries must be clearly defined around roles or sub-roles (not around individual tasks). Each agent should embody a coherent functional responsibility that mirrors how work is naturally partitioned among humans or system roles.

Agent Identity

Every agent must possess:
A verifiable identity (issued, signed, and managed through trusted identity infrastructure) to ensure that all interactions, delegations and executions can be authenticated and trusted end-to-end. Intra-domain agents are first-party identities; Inter-domain agents require explicit trust agreements and no implicit trust across domains

Input and output guardrails

Agents must operate within clearly defined input and output guardrails(or the constitution of work) – validating, constraining, and shaping data entering and leaving the agent.
This ensures safe, predictable, and policy-aligned behavior across the agentic system.
Guardrails should be applied based on agent risk tier

Stateless Agent Services with Externalized Memory Management

Agent services should remain stateless delegating all forms of memory, transient state, checkpoints and conversational history externalized in a durable persistence layer. It is therefore critical to design the state model of the agents to ensure only relevant information is persisted

Memory as a Service

Agentic systems must externalize long term memory into a dedicated “Memory-as-Service” layer – providing standardized APIs for storing, retrieving and managing agent interactions, user preferences rather than embedding memory logic directly within individual agents

Own Your Prompts

Prompts must be treated as first-class, governable assets – authored, versioned, evaluated and maintained by the respective project team – rather than being embedded ad hoc within application code or use abstraction layers like DSPY

Interoperability across Multi-Agent Systems

Agentic applications must be designed to interoperate seamlessly across heterogeneous multi-agent systems-supporting shared protocols, common schemas, standardized messaging patterns, and portable capability definitions to ensure agents can collaborate seamlessly across frameworks and platforms

Integrated Telemetry

Telemetry must be a native, first-class capability of the agent framework – automatically capturing reasoning traces, tool calls, latency, cost and state transitions – so developers are not responsible for manually instrumenting agents through decorators. Manual instrumentation should be an exception when needed

Additional Details



Agent Layer | Design Principles (2/2)

Domain driven MCP Development

MCP servers must be designed and deployed using Domain driven Design(DDD) principles – where capabilities are exposed along bounded contexts rather than as monolithic endpoints . This will enable multi-tenancy, capability isolation and seamless cross domain reuse

Data to Model and Model to Data

In Agentic Architecture, we must accommodate both “data to model” and “model to data” patterns. Data to model is when we ingest the data from source systems and vectorize them in vector stores. Model to Data is when the model retrieves the data from its source.

Configurable Agents

Agent behavior must be configurable – allowing prompts, tools , policies and underlying models to be externally defined and dynamically adjusted – so that the agent can evolve without significant code changes as the AI landscape and enterprise requirements continue to evolve

Protocol Aware Agents

Agent frameworks must natively support standardized interaction protocols such as A2A, so that agents automatically produce protocol-compliant messages, actions, status and tasks without developers manually instrumenting the protocol logic. Where native support is not there, reusable protocol adapters to be provided.

Additional Details



AI Governance | Design Principles

Certification as Pipeline

Agent governance should be a pipeline (dev->stage->prod) with clear “promotion” criteria. Certification is the gate that confirms an agent met required security and evaluation thresholds for its risk tier

Risk-tiered Controls (Policy Profiles)

Not all agents need the same rigor. Define tiers and scale requirements for evaluation coverage, approval depth and security restrictions accordingly.

Standardized Artifacts via Agent Card

Every agent must ship with an Agent Card that documents its purpose, intended users, tool/data access, guardrails, evaluation results, known

Registry as the Single Source of truth with Immutable Versioning

The Agent registry should hold the canonical, versioned definition of each agent. Treat production agents as immutable releases-changes require a new version + recertification

Identity-first Security with least Privilege by Default

Each agent has a unique agent identity used for authN/authZ, policy enforcement, and audit logs. Grant least privilege(scoped tools, minimal data, time bound credentials) and prefer just-in-time access for sensitive actions

Full Traceability and Auditability End-to End

Every run should produce a trace: inputs, decisions, tool calls, outputs and policy decisions, linked to the agent version in the registry.

Operational Guardrails + Kill Switches

Governance should include runtime controls, rate limits, spend/usage budgets, escalation rules and the ability to immediately disable an agent or revoke credentials if something goes wrong.

Additional Details



AI Operations | Design Principles

Prompt as Code

Prompt templates must live in source control with change history

Model Routing

Route by use-case and risk tier: cheaper models for low-risk tasks, stronger models for complex/high impact tasks with automatic fallback

Cost Controls

Budgets, rate limits, caching and unit cost tracking are built into platform observability so that spend scales predictably

Feedback Driven Iteration Loops

Capture human feedback and production traces, convert them into test cases, and continuously improve prompts and agent output efficiency

Feedback Driven Iteration Loops

Capture human feedback and production traces, convert them into test cases, and continuously improve prompts and agent output efficiency

Centralized Policy Enforcement

All model traffic flows through the gateway so auth, content/PII policies, tool/MCP allowlists, and tenant controls are enforced consistently

Additional Details



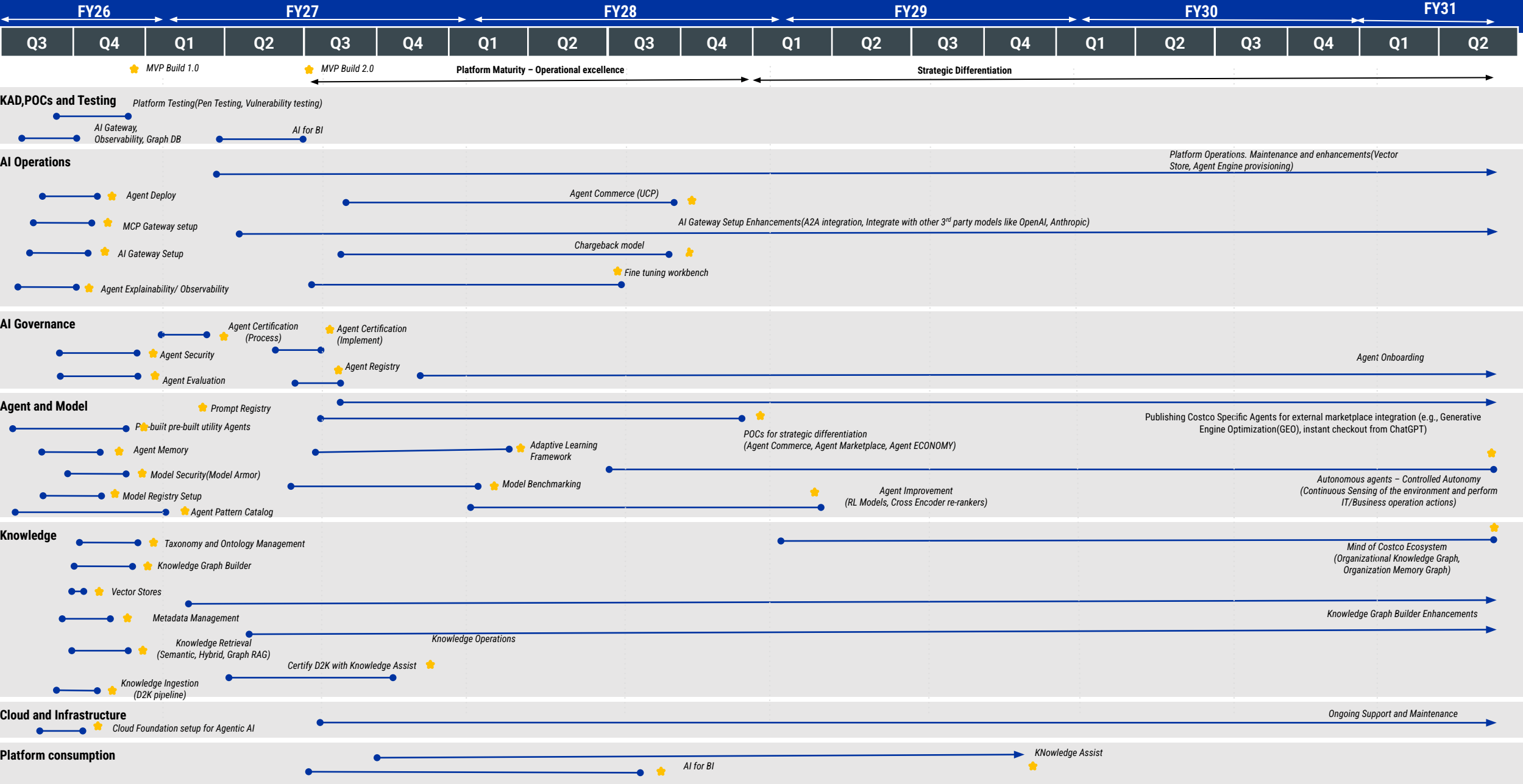


Agentic AI Platform Roadmap

Platform Capabilities Roadmap

Agentic capabilities with repeatable patterns go into the platform

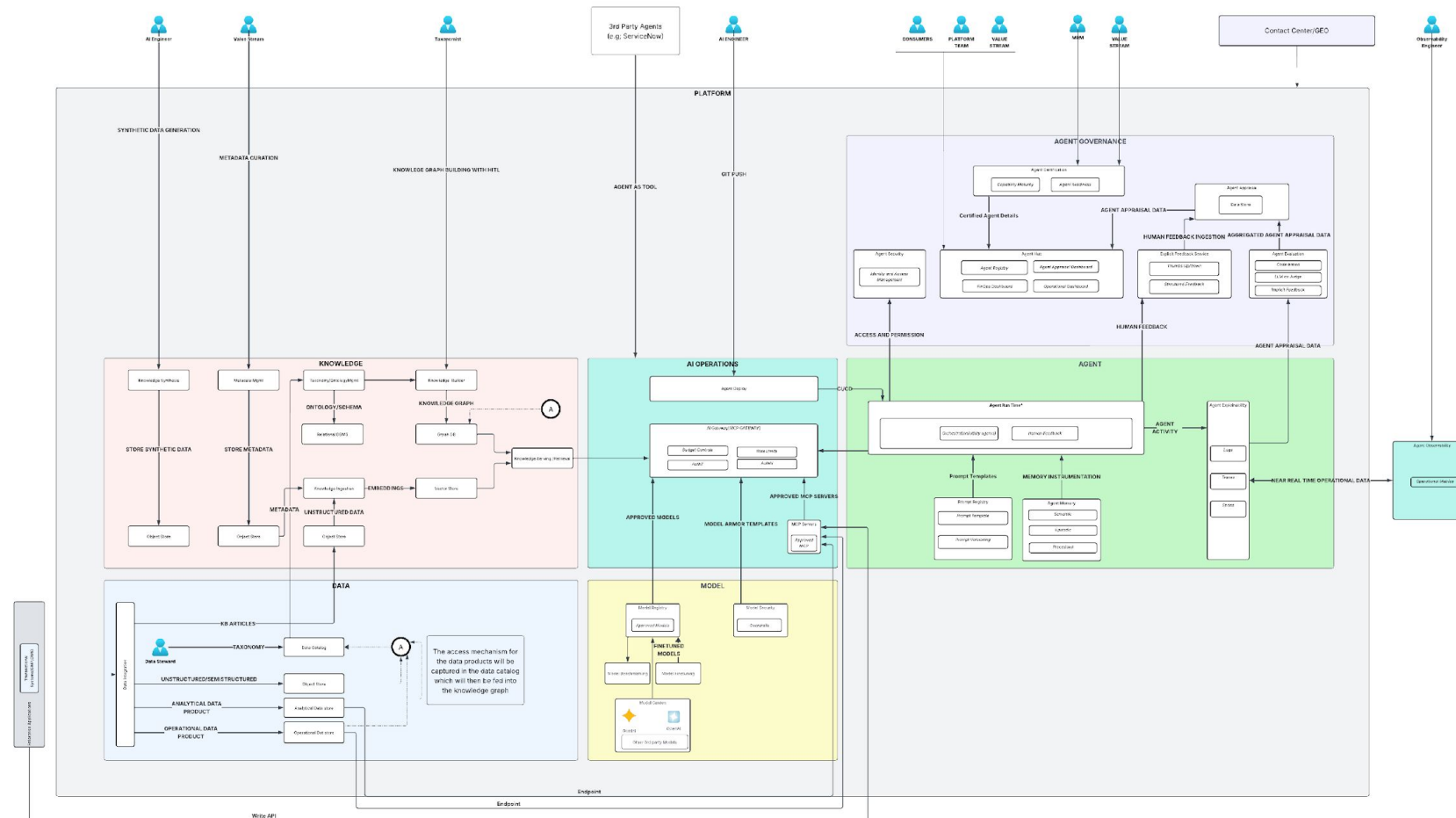
★ Milestones



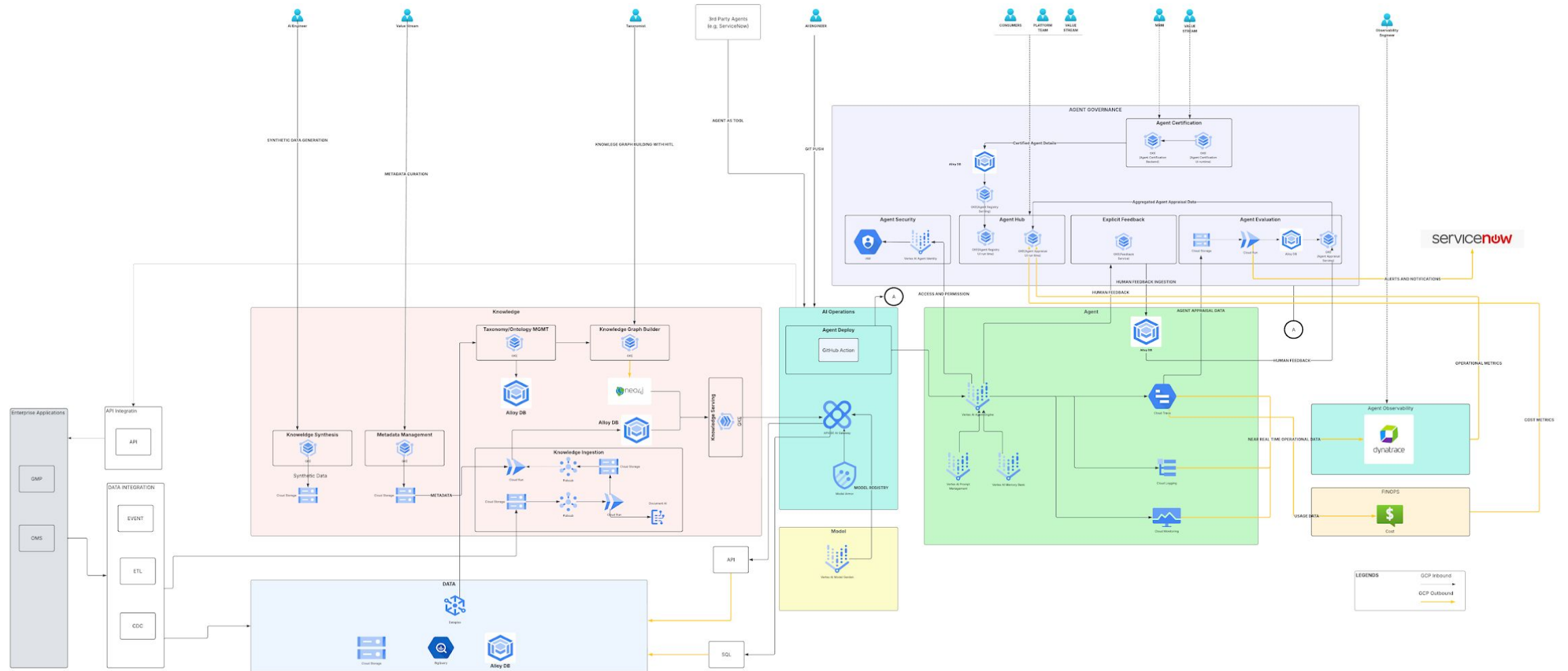


Detailed Architecture

Enterprise AI Agentic Platform | Logical Architecture



Enterprise AI Agentic Platform | Technical Architecture





Architecture Pattern Catalog

Pattern Catalog (1/2)



Knowledge (2)

Data to Knowledge

Knowledge to Data



Model Deployment (2)

Public

Private



Context Caching (2)

Explicit

Implicit



Agent Orchestration (3)

Deterministic(RPA)

Controlled Flow

Controlled Autonomy



Prompt (6)

Basic

Reasoning

Interactive

Structural

Quality

Output



Memory (3)

Semantic

Episodic

Procedural



Gateway (4)

External - Public

Internal Core - Internal

External - Confidential

Internal Core - Restricted



Agent Registry (2)

1st Party Agents

3rd Party Agents



Evaluation (3)

Code

Model

Human



Agent Observability(2)

Operational

Agent Appraisal



Pattern Catalog (2/2)



Agent Security (14)

[Prompt Injection](#)

[Insecure Model output](#)

[Inaccurate Model output](#)

[Sensitive Data Disclosure](#)

[Denial of Service](#)

[Privilege Escalation](#)

[Rogue Actions](#)

[Insecure Integration](#)

[Model Source Tampering](#)

[Model Reverse Engineering](#)

[Model Exfiltration](#)

[Model Deployment tampering](#)

[Unauthorized Training Data](#)

[Data Poisoning](#)

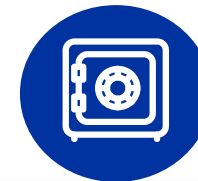


Multi Tenancy Patterns

[Vector Store](#)

[Knowledge Store](#)

[AI Gateway](#)



Defensive UI (7)

[Plan Preview](#)

[Bounding controls](#)

[Auto-Pause checkpoints](#)

[Evidence Chips and Citations](#)

[Live thought trace](#)

[Interruptible Generation](#)

[Turn Lock \(Single In-Flight request\)](#)

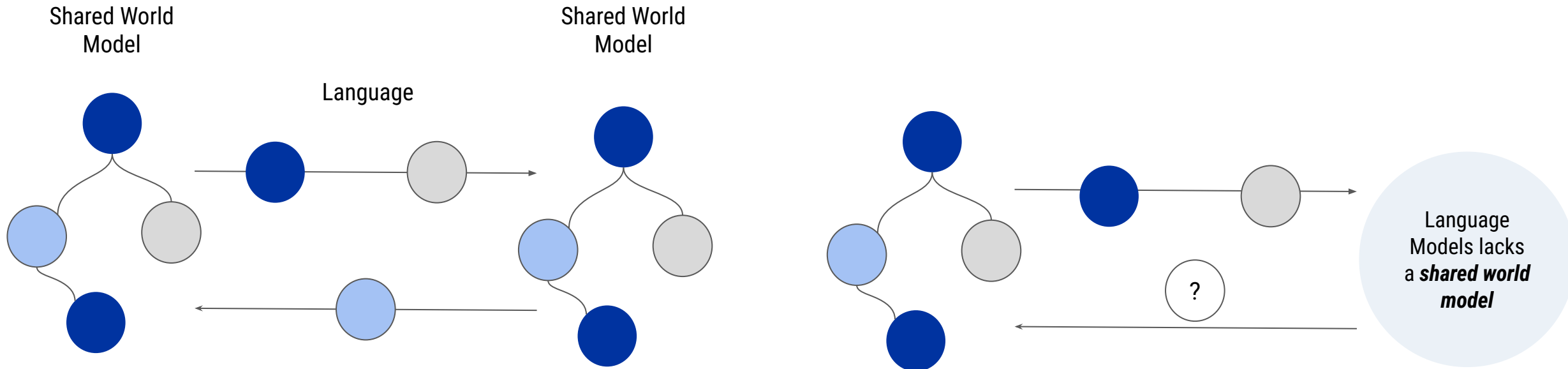
RAI Controls

[RAI Controls](#)



Knowledge Layer Patterns

Knowledge Layer | Small World Model for the Agents



When two human interact with each other both have the same shared world model. Hence they can communicate efficiently

Language models lack that world model. **Knowledge engineering** is the technique through which we provide that small world model to the agents powered by the language models

Knowledge Layer | Knowledge Integration Patterns



Data to Knowledge(D2K)

Source data is extracted, chunked, contextualized and embedded into a vector store to enable semantic retrieval for grounding the language model with organization specific knowledge. This pattern requires to build an incremental ingestion pipeline to ensure any change in the source data is reflected in the vector store.



Knowledge to Data (K2D)*

Data remains in the source systems, with only the knowledge representation of the data in the form of metadata, knowledge graph is persisted in the knowledge layer. Retrieval returns references and query intent, with required data fetched from the source at run time.

* **K2D Pattern** leverages Graph Database for capturing the knowledge representation. It acts as the Knowledge Index for the Agents as a service. **It is not a “Graph Database” as a service for the organization.**

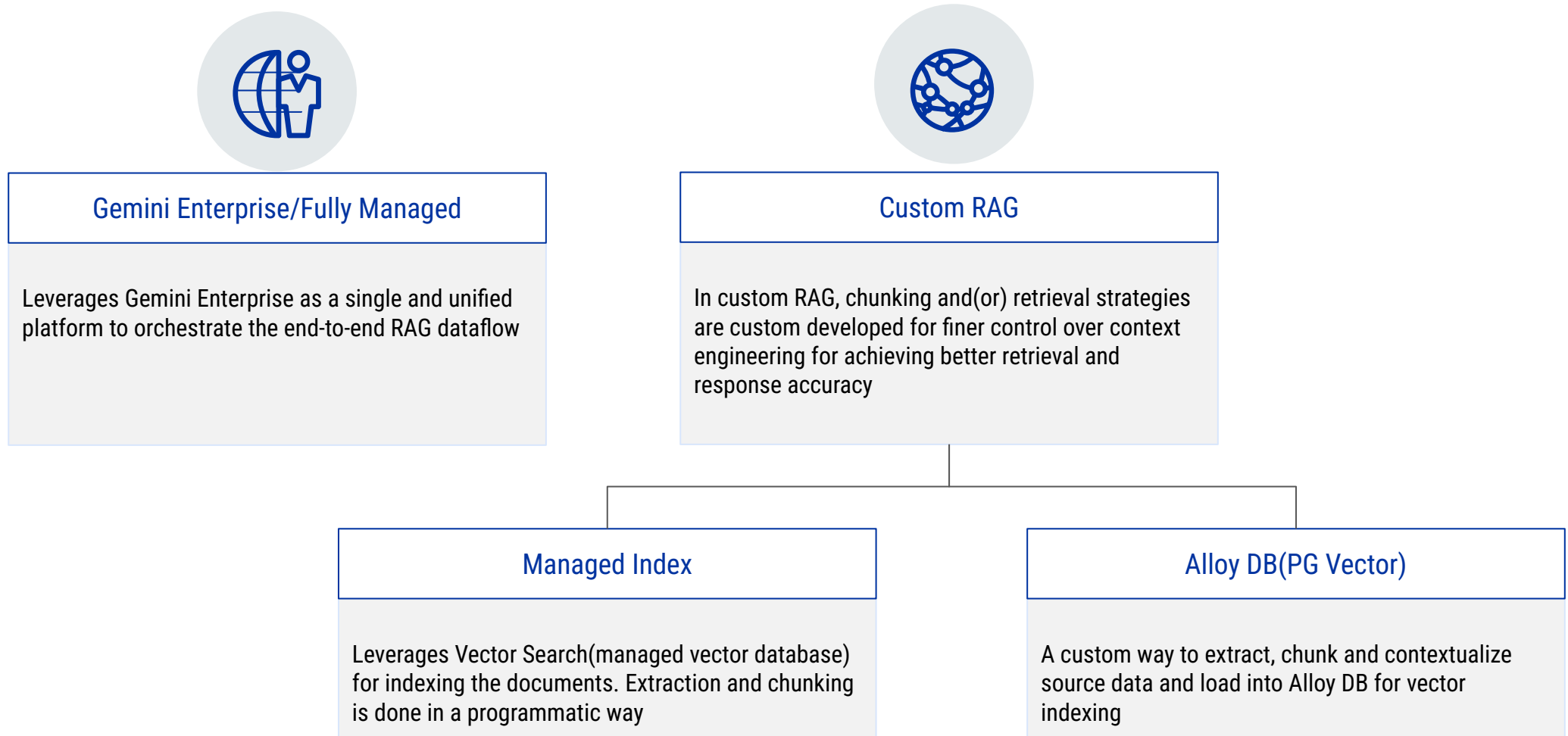




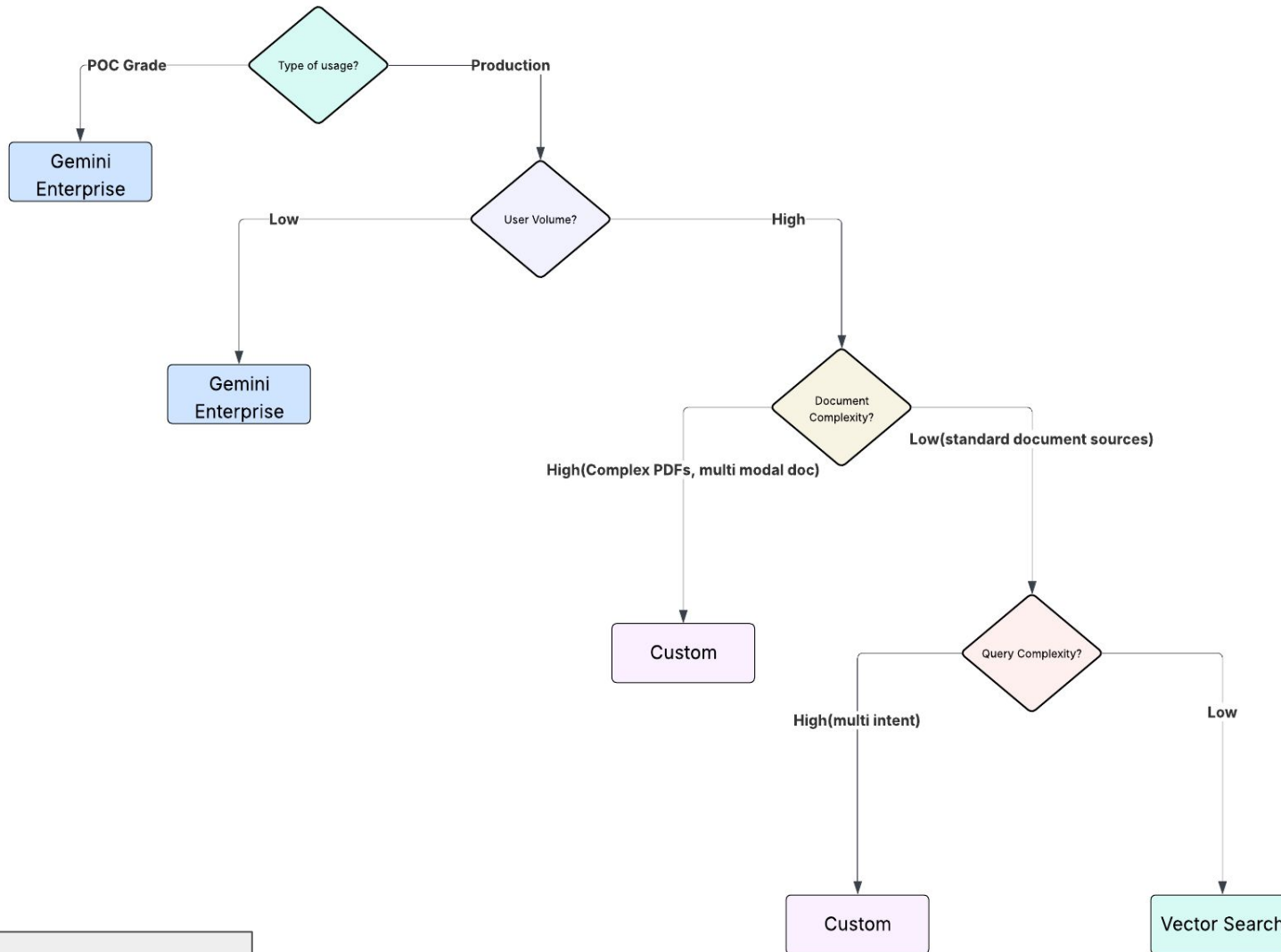
Data to knowledge

[> Return To Catalog](#)

Data to knowledge | Two patterns



Data to knowledge | Decision Criteria



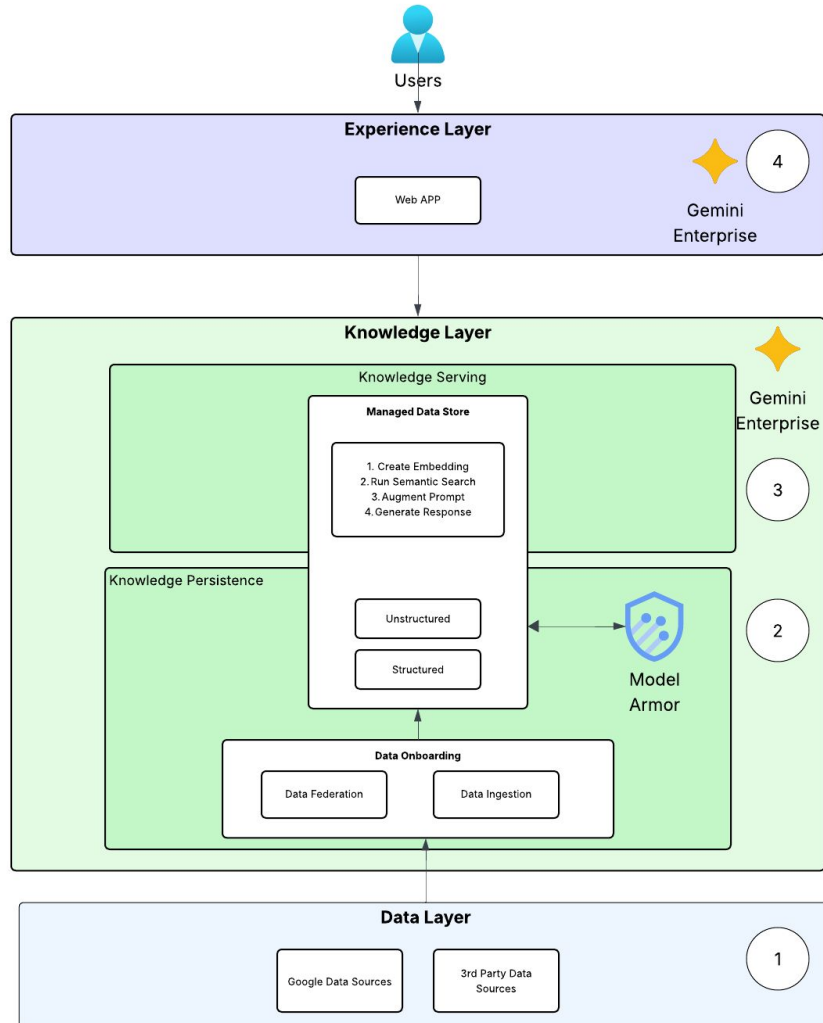
[Go to Lucidchart](#)

This is an **initial decision criteria** to select the D2K pattern. However, based on the accuracy of the response, **multiple experiments with all the patterns** may need to be done to reach the goldilocks zone of cost , accuracy and latency



Managed RAG

Data to knowledge | Gemini Enterprise - Managed RAG



This pattern is a **fully managed RAG solution** which is a no-code and configuration driven approach to access 3rd party or Google data sources either through data federation or data ingestion. Built in data connectors allows to bring in the data (ingestion) or the identity data such as roles, permissions or users (federation) into the managed data store of Gemini Enterprise. The manage data store also provides semantic retrieval services to retrieve contextual data based on user query

1

Platform support team configures the required data sources based on user requirements

2

Platform support team configures model armor guardrails for input and output moderation

3

Data is automatically ingested or federated in the managed data store

4

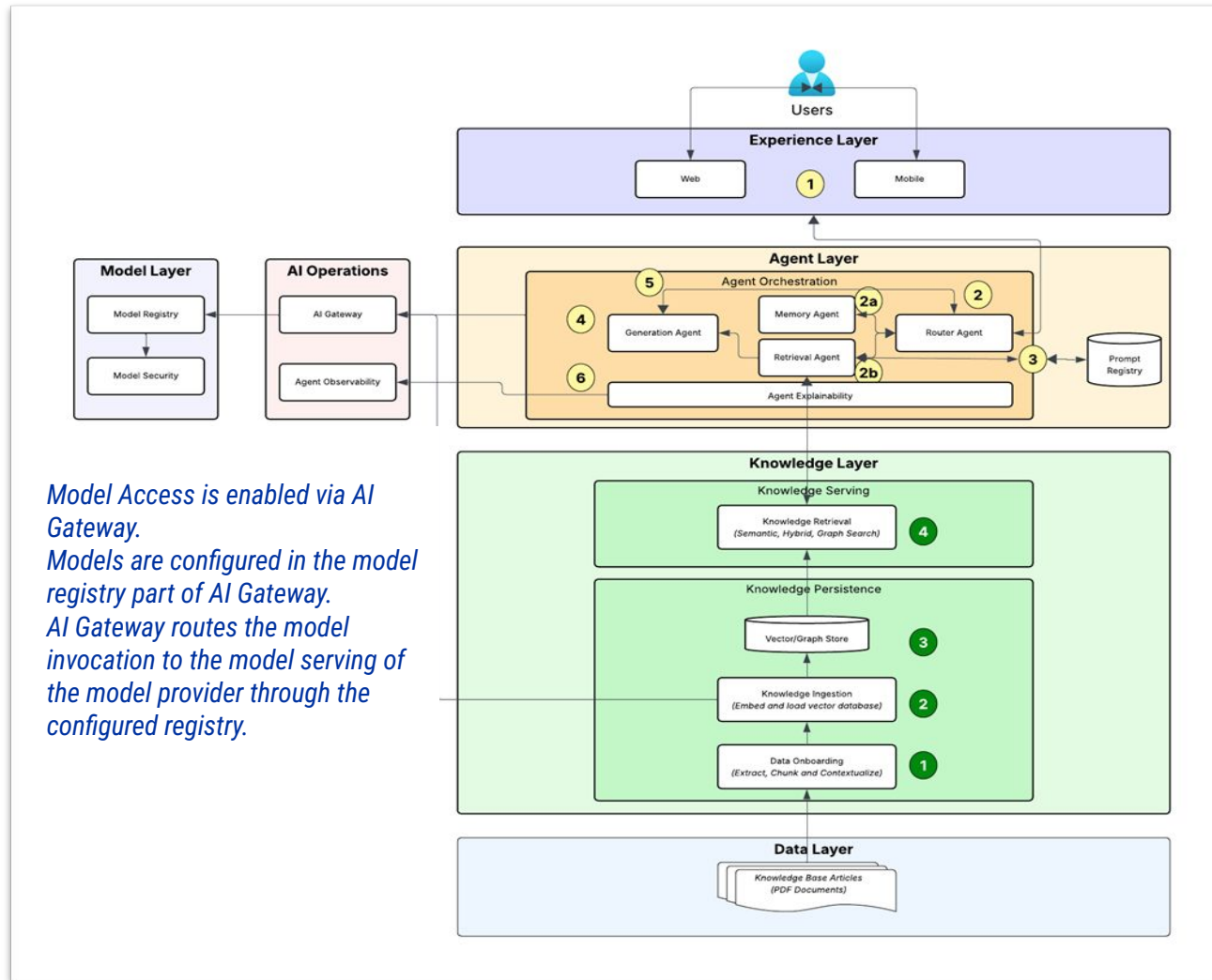
The solution is wrapped as a web app and exposed to user to talk to the data

[Go to Lucidchart](#)



Custom RAG

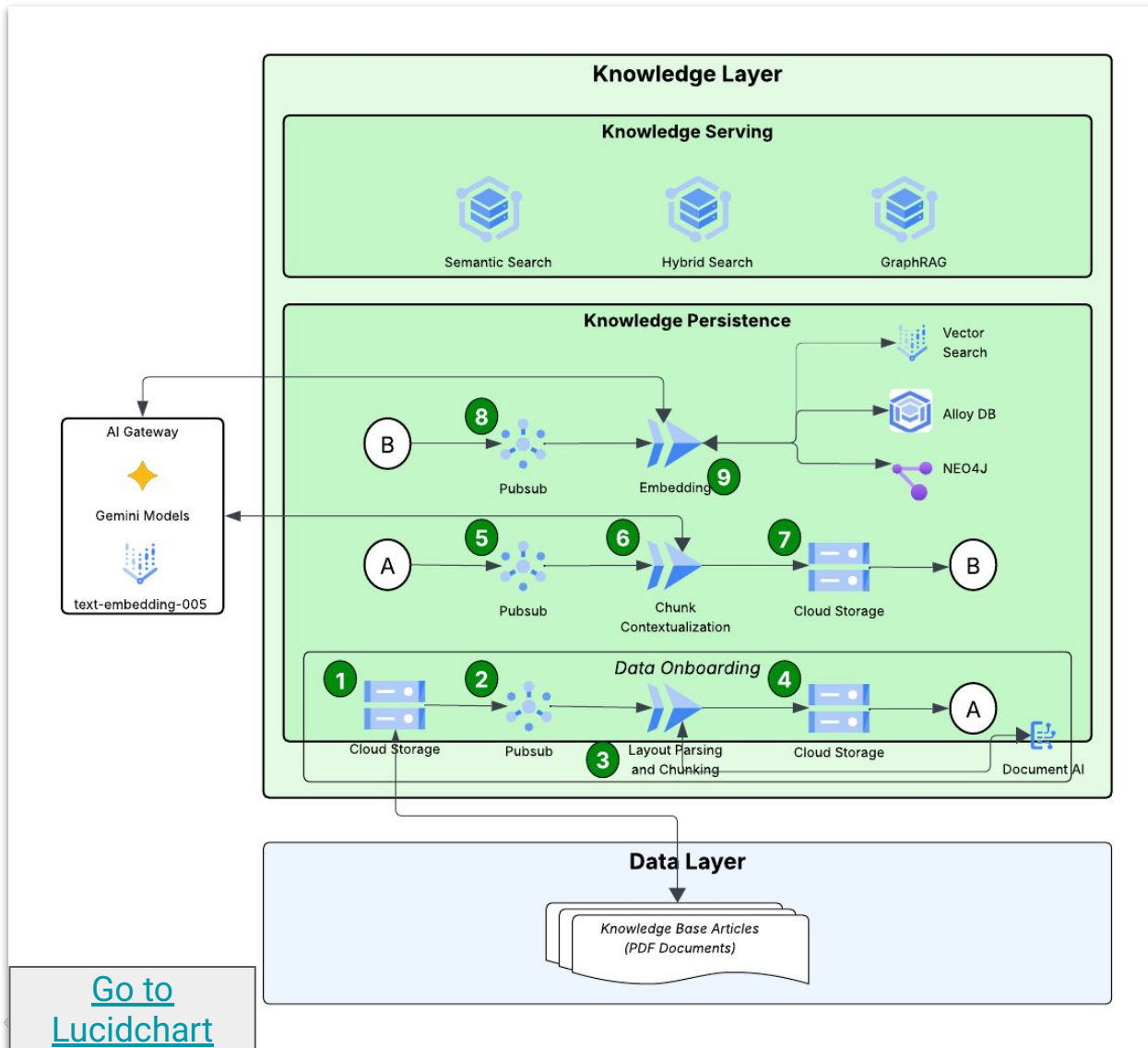
Knowledge Layer | Data to Knowledge | Logical Architecture



[Go to Lucidchart](#)

- 1 User logs on through the experience layer which can be either a web or a mobile channel
 - 2a Router agent routes the user query to Memory Agent – Memory Agent accesses memory store and determines if response can be retrieved from memory. If yes, provides the response to router, which then sends back to the user
 - 2b If response is not available in memory, router routes the query to retrieval agent which retrieves the relevant context from the knowledge layer through the retrieval services
 - 3 The retrieval agent extracts the prompt from registry and augments it with retrieved context, sends to Generation agent
 - 4 The Generation agent calls the language model through AI gateway to get the response
 - 5 Response is sent to the router which then sends the response back to the user
 - 6 Agent Explainability captures telemetry information and routes to Agent Observability
- 1 Data Onboarding process extracts, chunks and contextualizes the raw data for AI consumption
 - 2 Knowledge Ingestion process embeds the contextualized data
 - 3 The embedded and contextualized data is stored in a vector or a Graph store
 - 4 The ingested contextualized data (knowledge) is exposed in the serving layer through knowledge retrieval process

Knowledge Layer | Data to Knowledge | Technical Architecture(1/2)



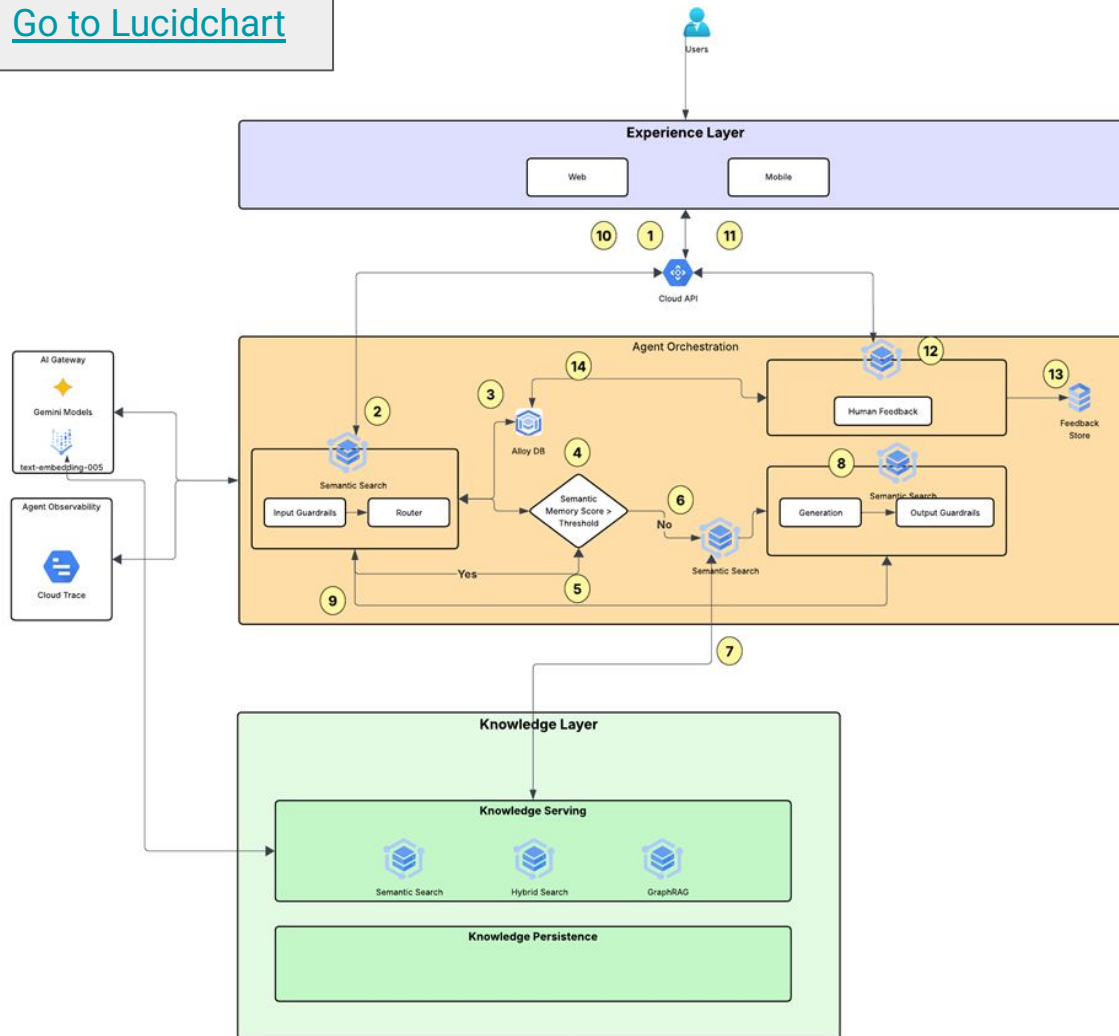
The knowledge serving Layer exposes the embedded knowledge in Alloy DB or Neo4J through retrieval services (Semantic Search, Hybrid Search and Graph Search)

- 1 Raw PDF based documents are pushed to cloud storage
- 2 Pub/Sub triggers layout parsing and chunking process
- 3 Layout chunking and parsing process uses Document AI for layout parsing and chunking
- 4 The chunks are stored in cloud storage bucket
- 5 Pub/Sub triggers the embedding process
- 6 Chunk contextualization process calls Gemini model to create the context summary for the chunk*
- 7 Chunk context are stored in Cloud storage bucket
- 8 Pub/Sub triggers the embedding process
- 9 Embedding process embeds each chunk and stores in Alloy DB. For Graph RAG, the embedding process uses the layout configuration to create a document graph and then creates embedding for each document node. The chunks can also be ingested into Vertex AI Vector Search which is a managed vector store

[Go to Lucidchart](#)

Knowledge Layer | Data to Knowledge | Technical Architecture(2/2)

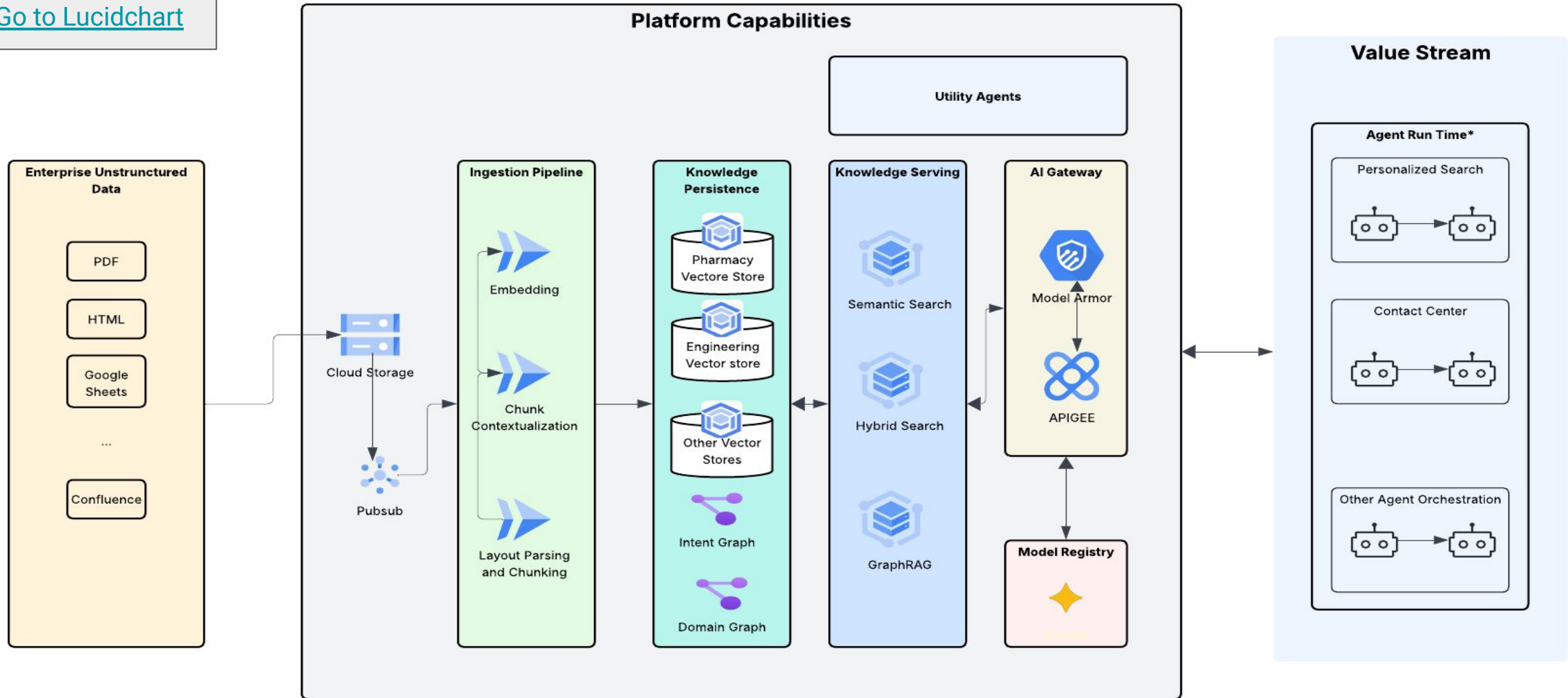
[Go to Lucidchart](#)



- 1 User logs in through web or mobile channel and sends a query which is routed to Cloud API. Cloud API routes the request to the Agent Orchestration layer
- 2 The router process applies any domain specific input guardrail and then routes the query to memory store to retrieve a matching response from memory store
- 3 If yes, response is sent to router agent
- 4 Memory store returns matching responses with a similarity score
- 5 If no, the query is routed to the retrieval agent
- 6 Checks if the similarity score is more than the configured similarity score
- 7 Retrieval agent calls the configured knowledge service (semantic, hybrid or Graph search) to get relevant context/documents from the knowledge layer
- 8 The relevant context is passed to the generation agent which calls Gemini language model to get the response. The response is passed through an output guardrail
- 9 The response (if passes output guardrail check) is then sent to the router agent
- 10 Router Agent sends back to the API layer and then to the user through the experience layer
- 11 User provides feedback to the response
- 12 The feedback is processed by the Feedback agent
- 13 The feedback is stored in the feedback store
- 14 The feedback is also stored in the memory (in case of thumbs up feedback) to be used for subsequent similar queries

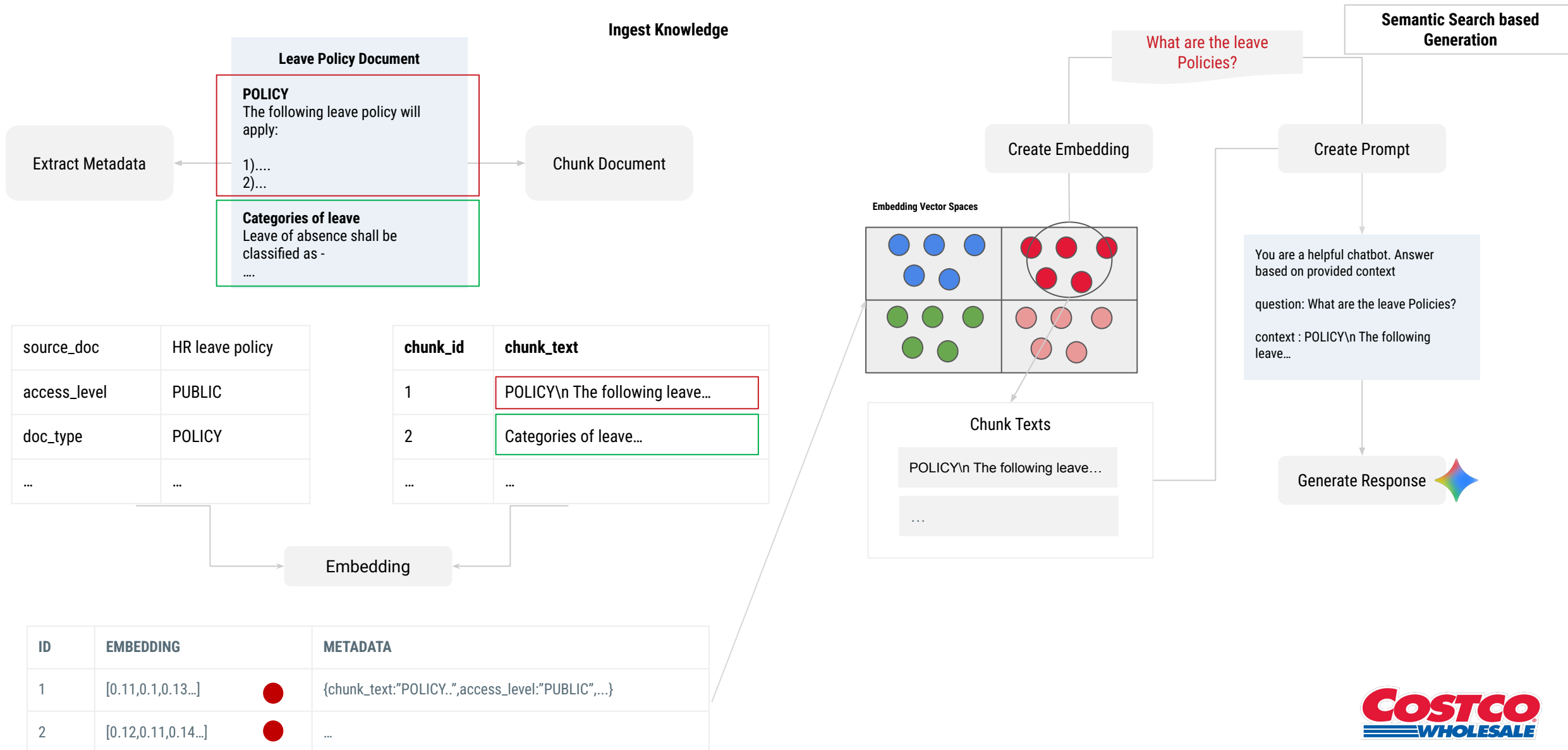
Hub and Spoke Model

[Go to Lucidchart](#)



** Agent Runtime is part of the platform. The provisioning will come from the value stream*

Semantic Search : Conceptual View



Hybrid Search : Conceptual View



Hybrid Search : Conceptual View

Weighted RRF Score = $\sum [w_r * (1 / (k + \text{rank}_r))]$

K(Smoothing Constant) = 60

Weight of Semantic Search = 0.7

Weight of Keyword Search = 0.3

Document	RRF Score	Weighted RRF Score
C1	$= 1 / (60 + 1) = 0.0164$	$= 0.7 * 0.0164 = 0.0115$
C2	$= 1 / (60 + 2) = 0.161$	$= 0.7 * 0.161 = 0.113$
C5	$= 1 / (60 + 3) = 0.159$	$= 0.7 * 0.159 = 0.011$

Document	RRF Score	Weighted RRF Score
C5	$= 1 / (60 + 1) = 0.0164$	$= 0.3 * 0.0164 = 0.0049$
C2	$= 1 / (60 + 2) = 0.161$	$= 0.3 * 0.161 = 0.0048$
C3	$= 1 / (60 + 3) = 0.159$	$= 0.3 * 0.011 = 0.0048$

Document	Final Score	Rank
C1	$0.0115 = 0.0115$	3
C2	$0.113 + 0.0048 = 0.0161$	1
C3	$0.0048 = 0.0048$	4
C5	$0.011 + 0.0049 = 0.0160$	2

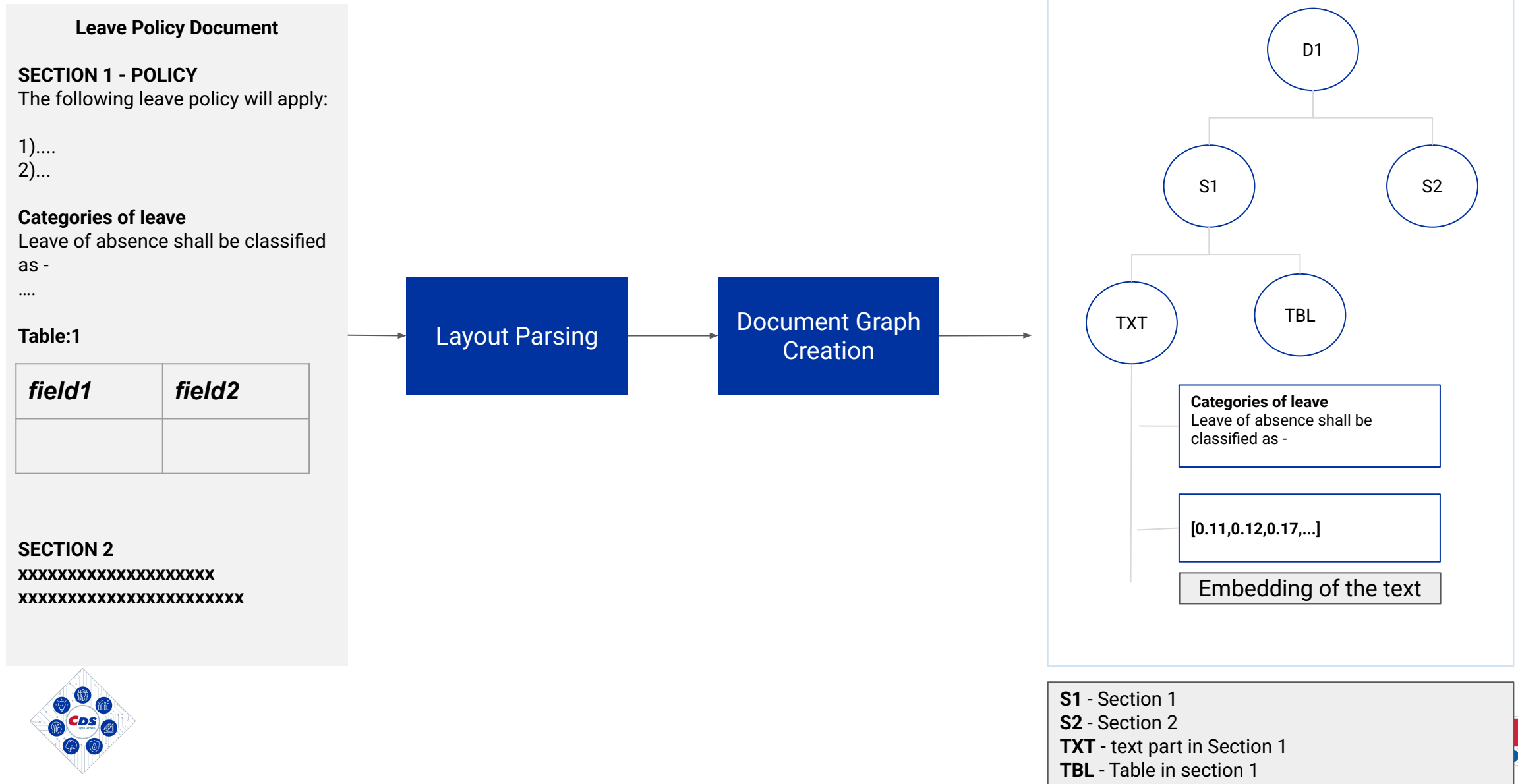
PG Vector – Hybrid Search Query

```

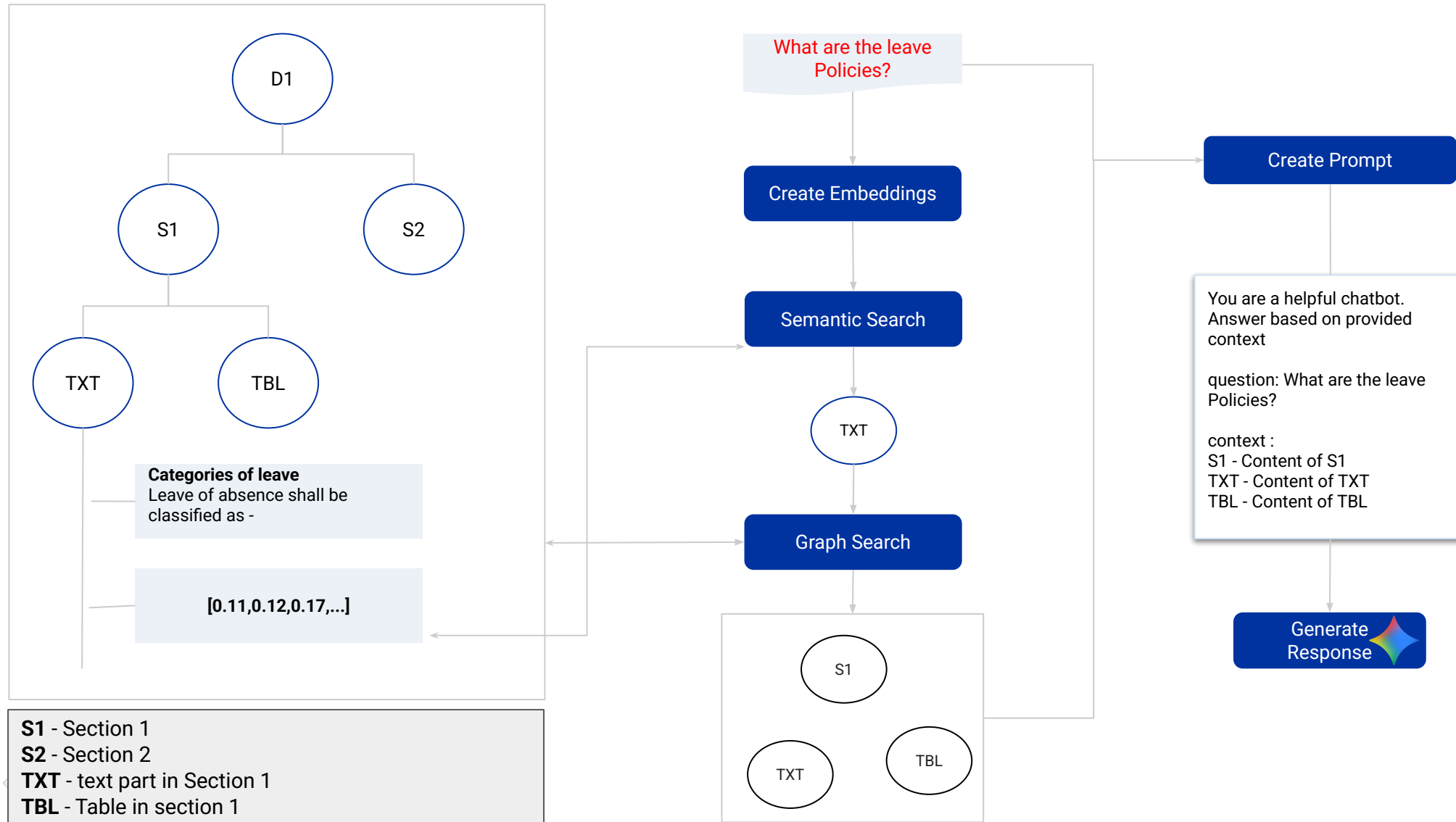
WITH semantic_search AS (
  SELECT id, RANK () OVER (ORDER BY embedding <=> %(embedding)s) AS rank
  FROM documents
  ORDER BY embedding <=> %(embedding)s
  LIMIT 20
),
keyword_search AS (
  SELECT id, RANK () OVER (ORDER BY ts_rank_cd(to_tsvector('english', content), query)
  DESC)
  FROM documents, plainto_tsquery('english', %(query)s) query
  WHERE to_tsvector('english', content) @@ query
  ORDER BY ts_rank_cd(to_tsvector('english', content), query) DESC
  LIMIT 20
)
SELECT
  COALESCE(semantic_search.id, keyword_search.id) AS id,
  COALESCE(1.0 / (%(k)s + semantic_search.rank), 0.0) +
  COALESCE(1.0 / (%(k)s + keyword_search.rank), 0.0) AS score
FROM semantic_search
FULL OUTER JOIN keyword_search ON semantic_search.id = keyword_search.id
ORDER BY score DESC
LIMIT 5
  
```



Graph RAG : Conceptual View | Ingestion



Graph RAG : Conceptual View | Retrieval

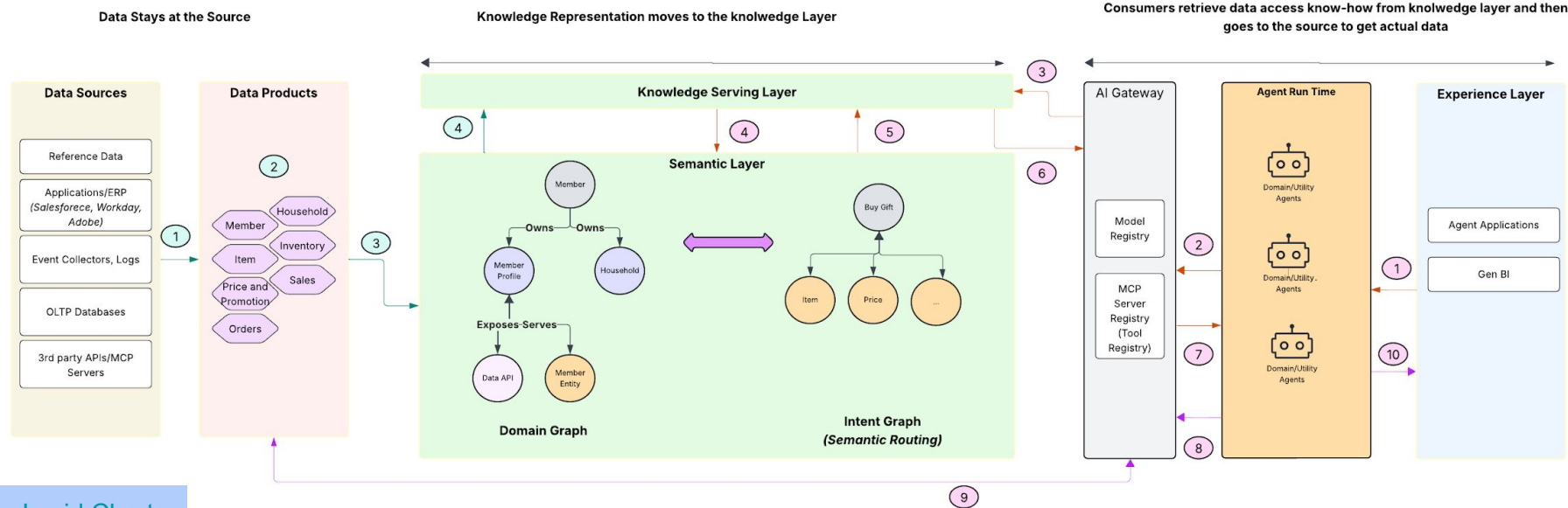




Knowledge To Data

[> Return To Catalog](#)

Knowledge Layer | Knowledge to Data : Logical Architecture



[Go to Lucid Charts](#)

1 From the experience layer Agentic Applications or Users send a request to the Agents running in Agent run time

2 Agents connect to the knowledge serving layer to access intent and domain graph through the AI gateway

3 The knowledge serving layer exposes the services to connect to the semantic layer

4 Knowledge layer connects to the semantic layer to read the intent graph and then connect to domain graph

1 Data from system of records are pushed into the data lake

2 The data from source data goes through data transformation to create the data products

3 The semantic layer is created by extracting the relationships between the data products.

4 The semantic layer is exposed via the knowledge serving layer

5 The information on how to access and retrieve data from domain data products is sent to the serving layer

7 The information on how to access the data products is sent to the agents

9 The required data is retrieved and sent to the agents as context to create the response

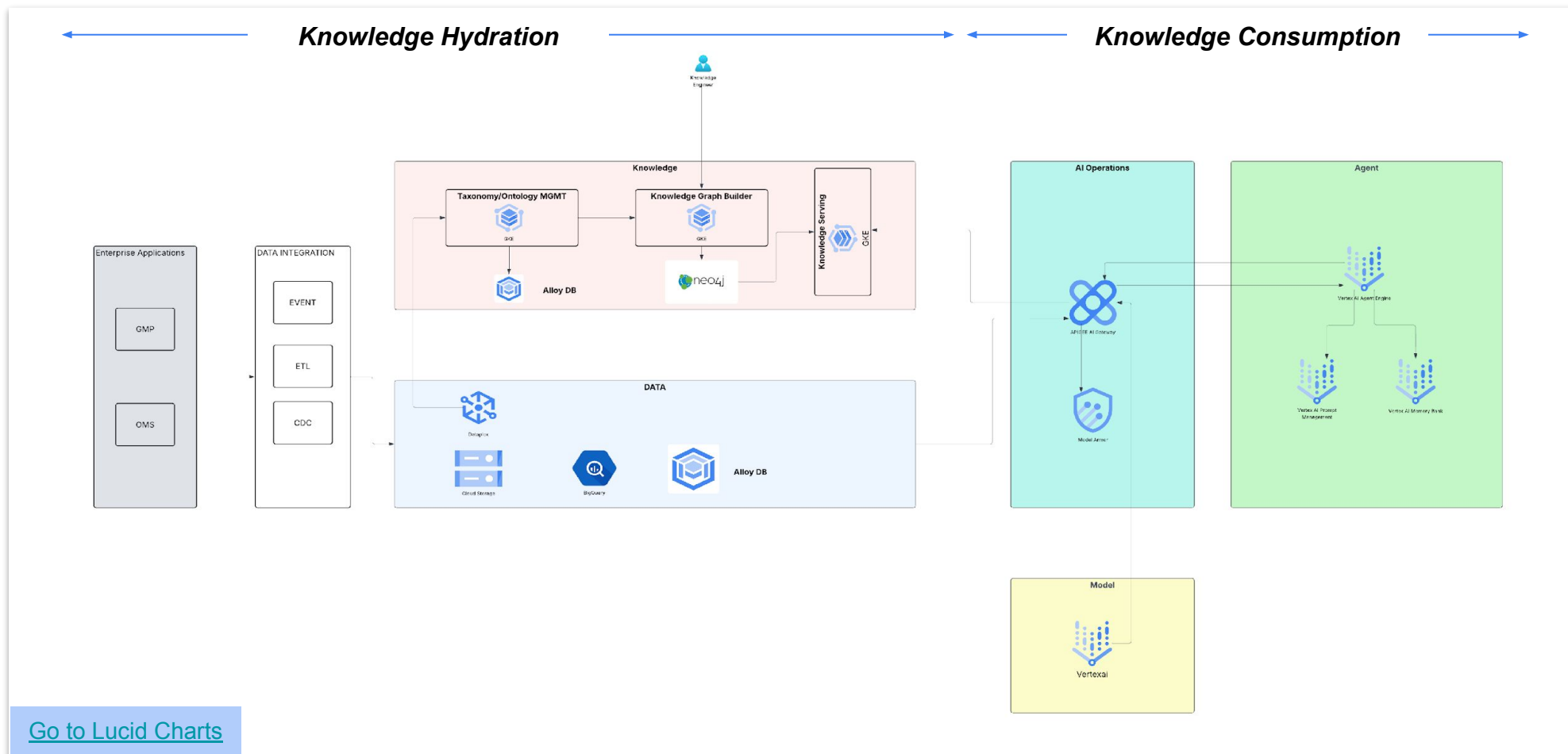
6 Serving layer sends the information to the agent layer through AI gateway

8 Agents use the information to access the data from the data products

10 Generated response is sent back to the agentic applications running in the experience layer



Knowledge Layer | Knowledge to Data : Technical Architecture



The knowledge to data pattern requires the knowledge representation of the underlying data products to be extracted and brought to the knowledge layer as a graph. The semantic layer where the knowledge graph sits is semi-automated by providing taxonomy and ontology of the domain as input to the KG creation solution. The KG creation solution creates the initial graph which is tuned by knowledge manager and then persisted in a Knowledge Graph

Knowledge Graph Studio - Wireframe

Extraction Settings

Upload Data

Generate Graph

Select Domain

Member

Available Taxonomies

Data Products

- Member Profile
- Household
- ...

Entity Type

- Member
- Items

Selected Taxonomies

Data Products

- Member profile

Entity Type

- Member

Available Ontologies

- Personalized Search
- Use case # 1
- Use case # 2

Selected Ontologies

- Personalized Search



Knowledge Graph Studio - Wireframe

Extraction Settings

Upload Data

Generate Graph

Drag and drop
Or browse
Documents, Images, unstructured text

From GCS

Other Connectors

File name	Upload Status	Size	Source/Type
service_chat.txt	Uploaded	132 KB	GCS



Knowledge Graph Studio - Wireframe

Extraction Settings

Upload Data

Generate Graph

Knowledge Graph Studio

XX nodes

XX edges

Navigate

Zoom

←

→

↑

↓

🏠

Add Node

Node name

Description

Add

Add Edge

Source Node

Connects

Target Node

Connect

Edit Node

Select Node name

Description

Add

Edit Relationship

Source -> Target

Description

Add

AI Assistant – Conversational Graph Editor

Try: "Add node <node name>"

Send

Clear

Persist Graph in Neo4J



Model Deployment

[> Return To Catalog](#)

Model Deployment Patterns

Public Deployment

- Cloud hosted services
- API based access
- Managed infrastructure
- Scalable on-demand

When to Use?

- 1 Cost-effective scaling
- 2 Quick deployment
- 3 Non-sensitive data
- 4 Standard compliance needs

Private Deployment

- On-premises infrastructure
- VPC/isolated networks
- Self-hosted models
- Full control

When to Use?

- 1 Sensitive/regulated data
- 2 Strict compliance(HIPAA, GDPR)
- 3 Data sovereignty requirements





Context Caching

[> Return To Catalog](#)

Context Caching Patterns

Implicit Caching

Enabled by default on most Gemini models; cache hits reduce cost, but caching is not guaranteed for implicit caching

Explicit Caching

Explicit caching is enabled through Gemini explicit caching feature which allows to pass some content to the model once, cache the input tokens and then refer to the cached tokens for subsequent requests

Scenarios when Explicit Caching is recommended

- 1 • Chatbots have extensive system instructions
- 2 • There is a need to repetitive analysis of lengthy video files
- 3 • Recurring queries against large document sets
- 4 • Frequent code repository analysis or bug fixing



[Context caching overview](#) | [Generative AI on Vertex AI](#) | [Google Cloud Documentation](#)





Orchestration

Design Patterns

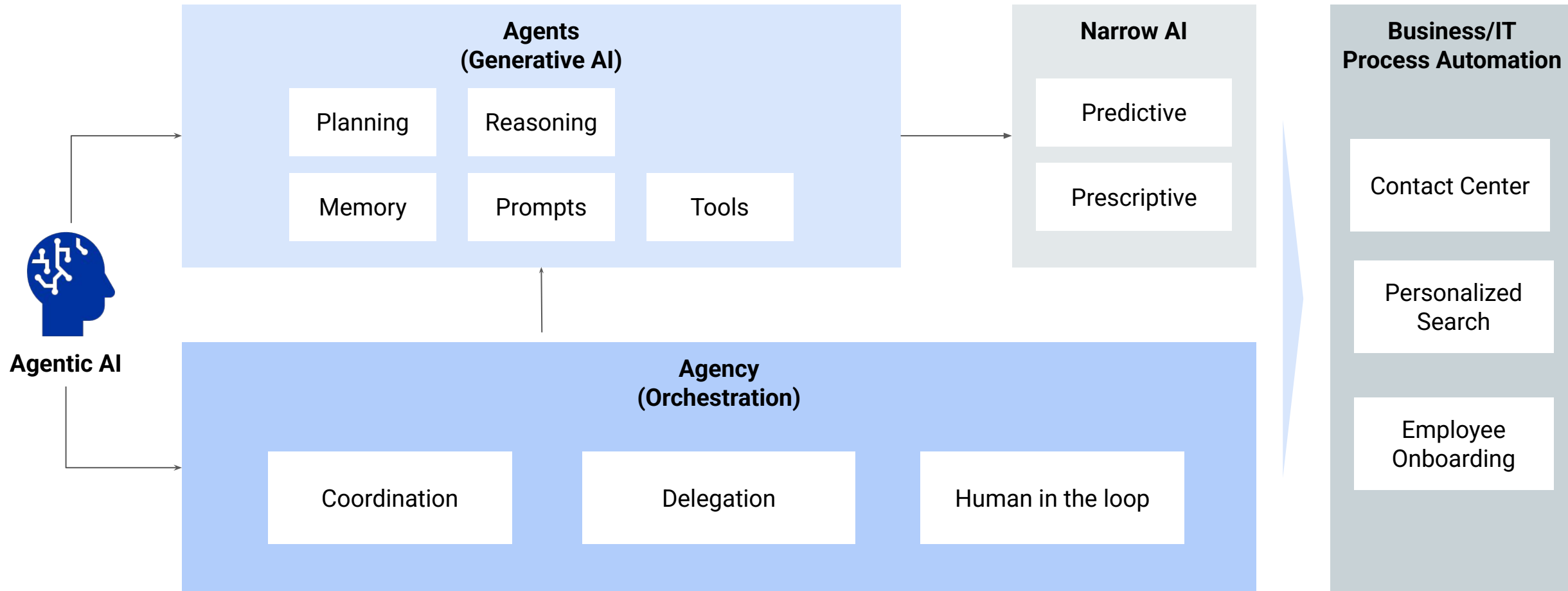
[> Return To Catalog](#)

Narrow Vs Generative Vs Agentic AI

Feature	Narrow AI	Generative AI	Agentic AI
Primary Goal	Predictive/Prescriptive	Content Creation(discrete, single tasks)	Act and Execute
Action Type	Recommendation only	Generative	Controlled Autonomy
Adaptability	Low(needs a retraining loop)	Medium(prompt engineering)	High(Context Engineering)
Workflow	Single Step	Prompt chaining	Multi-step
Example	Fraud detection, Recommender system	Summarization, Q&A Chatbot	Employee Onboarding, IT Incident management, Personalized Search, Call Center



How do these three AIs come together



Agentic AI Orchestration | Orchestration Patterns

Determinism

Autonomy

RPA Based Workflow

Scripted, rule-based bots that follow predefined steps and assume structured and consistent input and output. It is best for repetitive, high-volume work with little ambiguity, but becomes brittle when anything changes

ADK has **two types of agents** to realize controlled flow and controlled Autonomy patterns

Controlled Flow Engineering

Human defines the adaptive flow of the agents. The flow is pre-defined, but within the flow, agents can decide the next step based on predefined conditional paths

1

Workflow agents are specialized components in ADK designed purely for **orchestrating the execution flow of sub-agents**. Their primary role is to manage *how* and *when* other agents run, defining the control flow of a process.

Controlled Autonomy

In this pattern, the root or the supervisor agent determines the next state or step in the flow, however its autonomy is still constrained using a defined knowledge model, prompt and set of purpose driven tools

2

The LlmAgent (often aliased simply as Agent) is a core component in ADK, acting as the "thinking" part of your application. It leverages the power of a Large Language Model (LLM) for reasoning, understanding natural language, making decisions, generating responses, and interacting with tools.

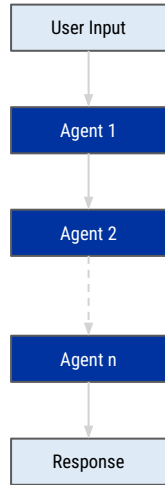
Note:

1. Favor deterministic behavior, where possible
2. Avoid leveraging fully autonomous agents



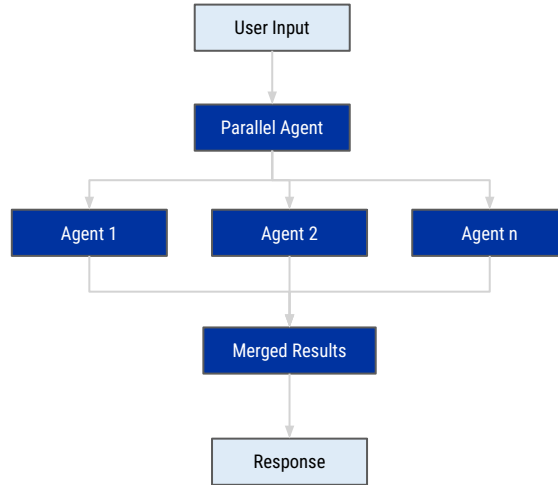
Agent Orchestration | Workflow patterns

Sequential



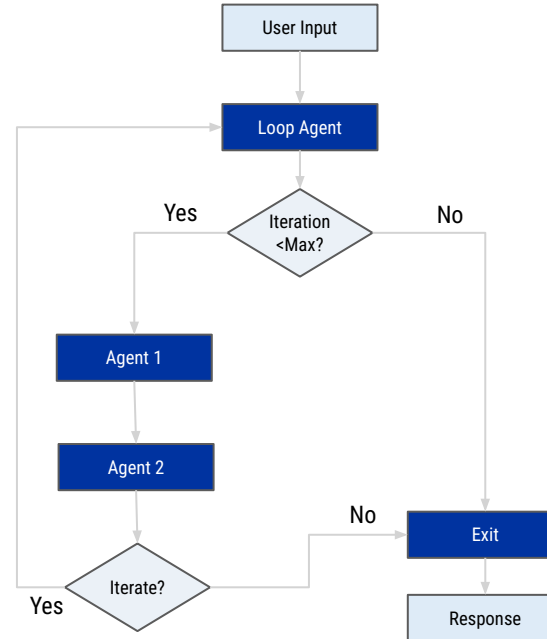
Executes sub-agents one after another in a defined order. Each agent completes before the next starts

Parallel



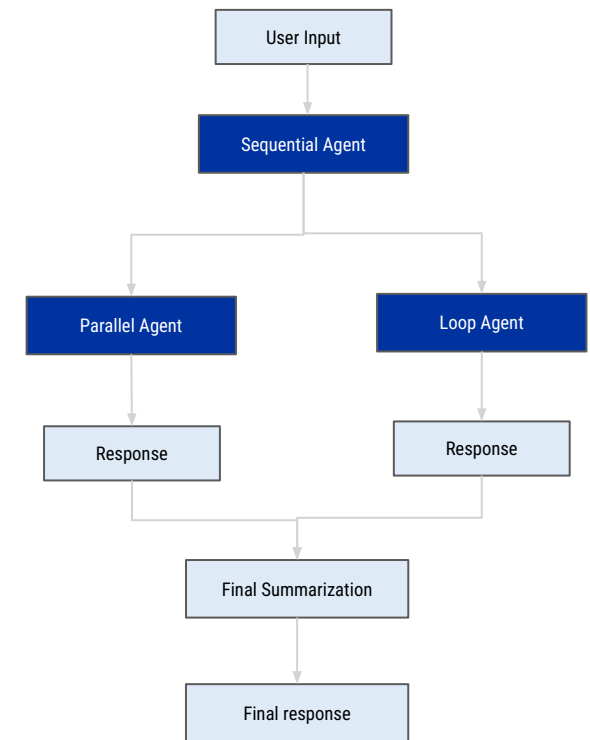
Executes multiple sub-agents concurrently with isolated contexts. Each agent works independently and cannot see others output

Loop



Executes sub-agents repeatedly in a loop until termination condition is met (escalation or max iterations)

Nested/Custom

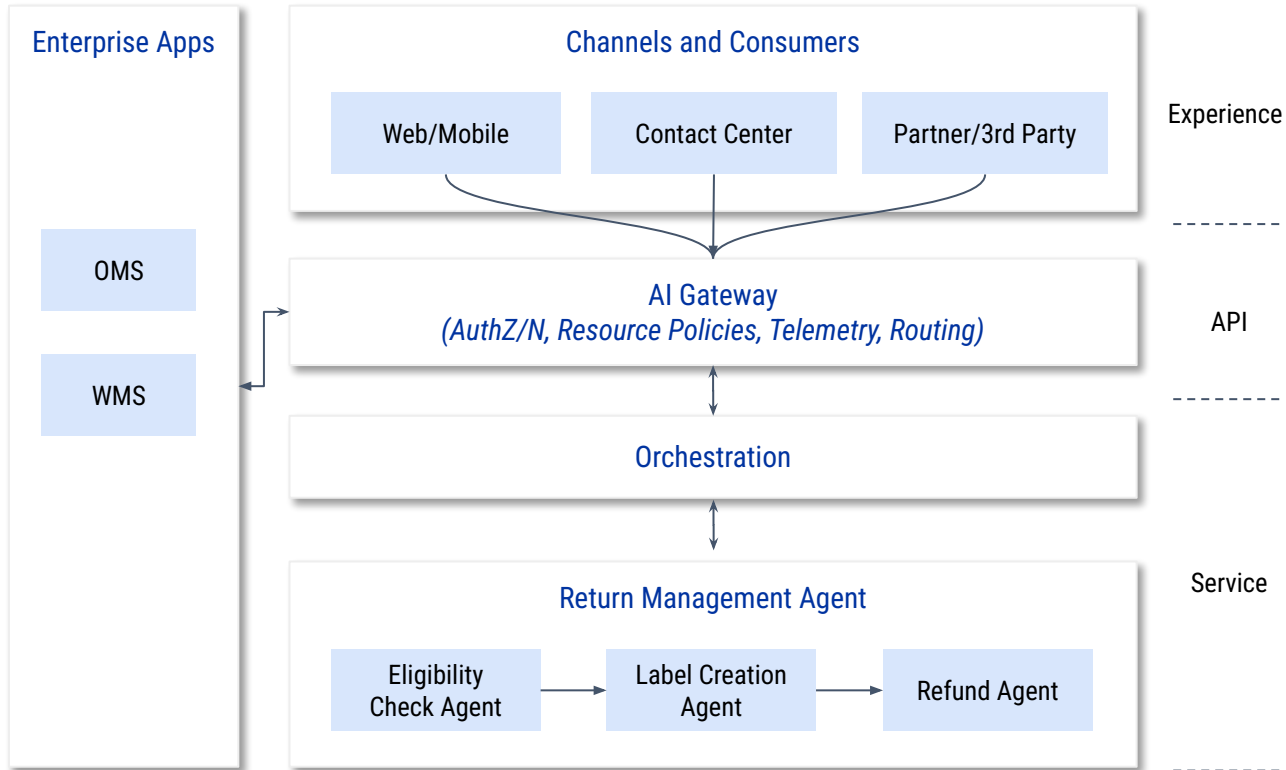


Combines multiple orchestration hierarchically to create sophisticated multi-stage workflows. Any pattern can contain other patterns as sub-agents, enabling complex logic



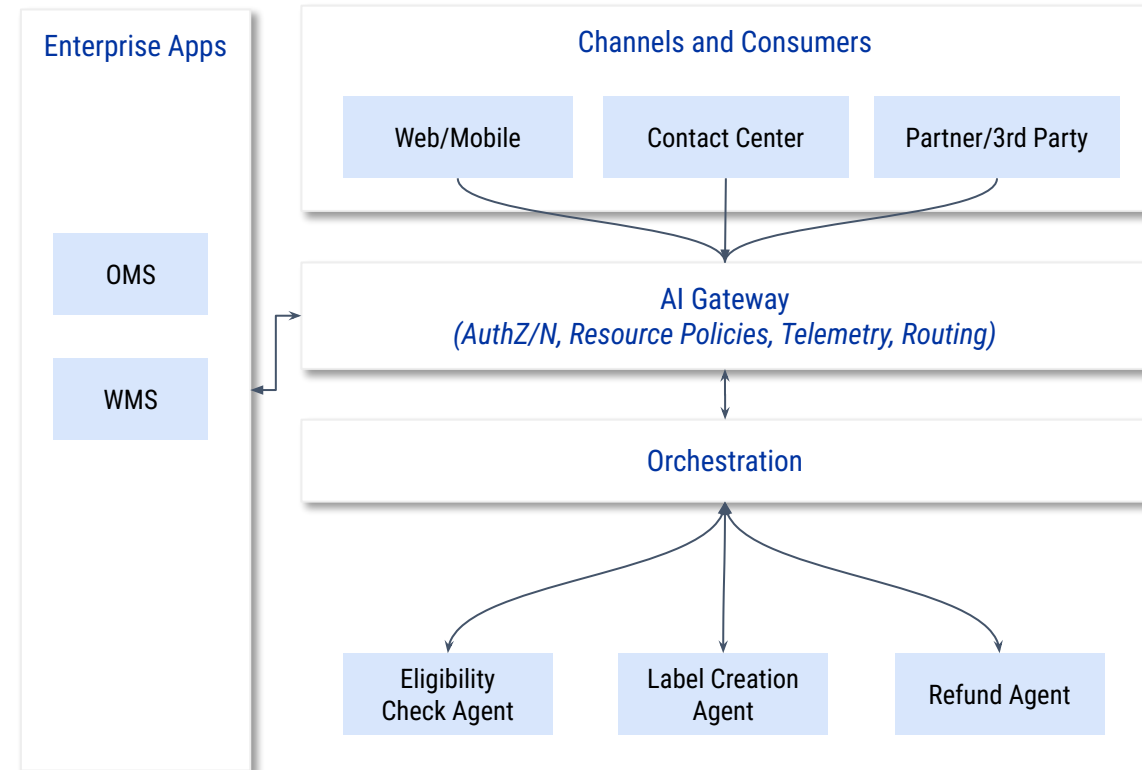
Agent Orchestration | Macros Vs Micro Agent Pattern

Macro Agent Pattern



1. **Stable contract** : Consumers integrate to one outcome-driven API, so we can change internal decomposition without breaking clients
2. **Efficient Governance**: Easier to enforce least privilege, policy guardrails, audit logs and telemetry instead of replicating controls across many endpoints
3. **Better reliability and debugging**: End to end tracing is simpler, reduces hand-offs, improves latency

Micro Agent Pattern



1. **Endpoint Sprawl**: Clients must chain multiple micro calls, increasing coupling and chattiness across the network
2. **Inefficient controls**: Guardrails, auth, rate limits, and logging drift across services, expanding attack surface and weakening compliance policies
3. **Higher latency/more failure points**: Each hop adds latency, small changes cascade across handoffs, making the system brittle.



Prompt Patterns

[> Return To Catalog](#)

Anatomy of a Prompt

	Role/Persona	Defines who the AI should act as(e.g., a data analyst) to establish the tone and expertise	Role/Persona sets the expertise level
	Context	Provide background information, relevant details, target audience, or the 'why' behind the request to guide the Ais' understanding	Context provides necessary background
	Task/Instruction	Clearly state the primary goal or action required(e.g. "Write a blog post", "Analyze the dataset"). Be specific and direct	Task is the clear instruction
	Constraints	Set limitations, rules, or boundaries, such as word count, specific keywords to include or exclude, tone to avoid	Constraints define boundaries
	Format	Specify the desired output structure (e.g.; bullet points, a structured report, a code snippet, a table , a script, or a specific file format)	Format structures the output
	Examples	Offer concrete examples of the desired output style, structure, or content to give the AI a clear pattern to follow, enhancing accuracy	Examples provide guidance

Prompt Patterns

Basic Patterns

Zero-Shot

Few-Shot

Reasoning Patterns

Chain of thought

Tree of thought

Interactive Patterns

ReAct

Flipped interaction

Structural Patterns

Template

Persona

Quality Patterns

Self consistency

Refinement

Output Patterns

Automator

Context Manager



Prompt Patterns



Zero Shot Prompting

Translate this into French: "Hello how are you"



Chain of Thought

Solve this problem step by step:
If a train travels at 60 mph for 2.5 hours, how far does it go?
Let's think through this:



Few Shot Prompting

Classify the given sentiment. Here are few examples to follow:

Examples

"I love this product" – Positive

"This is terrible" – Negative

"Best purchase ever" - ?



Tree of Thought

Consider three different approaches to solve X:

Approach 1: ...

Approach 2: ...

Approach 3: ...

Evaluate each approach and choose the best one



Prompt Patterns

ReAct

Task: Find the population of the capital of France

Thought: I need to identify France's capital first

Action: Search for "capital of France"

Observation: Paris

Thought: Now I need Paris population

Action: Search for "Paris Population"

Template

Analyze [TOPIC] from the perspective of [ROLE]

Focus on [ASPECT 1], [ASPECT 2] and [ASPECT 3]

Flipped interaction

I want to learn about machine learning.

Ask me question to understand my background and goals, then create a personalized learning path.

Persona Pattern

You are a senior security engineer with 15 years of experience.

Review this code for vulnerabilities...



Prompt Patterns

Self Consistency

Solve this problem in 3 different ways and verify if they agree

Note: This ensemble method can also be leveraged by asking the question to three different models and use majority vote

Refinement

Generate a marketing email.

Now make it more concise.

Now make it more persuasive.

Now add urgency.

Automator

Generate a Python script that I can run directly. Include all imports and make it executable.

No explanations, just the code.

Context Manager

Context: I am building a React app with Typescript.
I will ask multiple questions. Keep this context in mind.

Question 1: How do I create a custom hook?

Question 2: How do I test it?





Agent Memory Management

[> Return To Catalog](#)

Agent Memory | Types of Memory

01

SEMANTIC

- Semantic memory **captures durable facts** and their relationships that anchor an agent's response
- It helps the agent **retain important context** that is not baked into the model from pre-training, and that cannot reliably be pulled from the web or a generic retrieval system

Purpose: Facts and Knowledge

Storage Pattern: Profile or collection

02

EPISODIC

- Episodic memory **preserves records or past interactions** so an agent can **recall specific experiences**. Unlike procedural memory, it does not encode general rules of behavior
- It **captures "cases" the agent lived through**. And unlike semantic memory, it is not about timeless facts or general knowledge. It is about what happened and how the agent handled it, **answering "how did we solve this?"** rather than only "what is the answer?"
- In practice, episodic memory is often **stored as distilled few shot examples** extracted from long conversations

Purpose: Past Experience

Storage Pattern: Collection

03

PROCEDURAL

- **Captures "how to do things"** – the internalized know-how an agent uses to carry out tasks
- Unlike episodic memory, which recalls specific past experiences, procedural memory **encodes generalized skills, rules, and behavioral routines that apply across situations**

Purpose: System Behavior

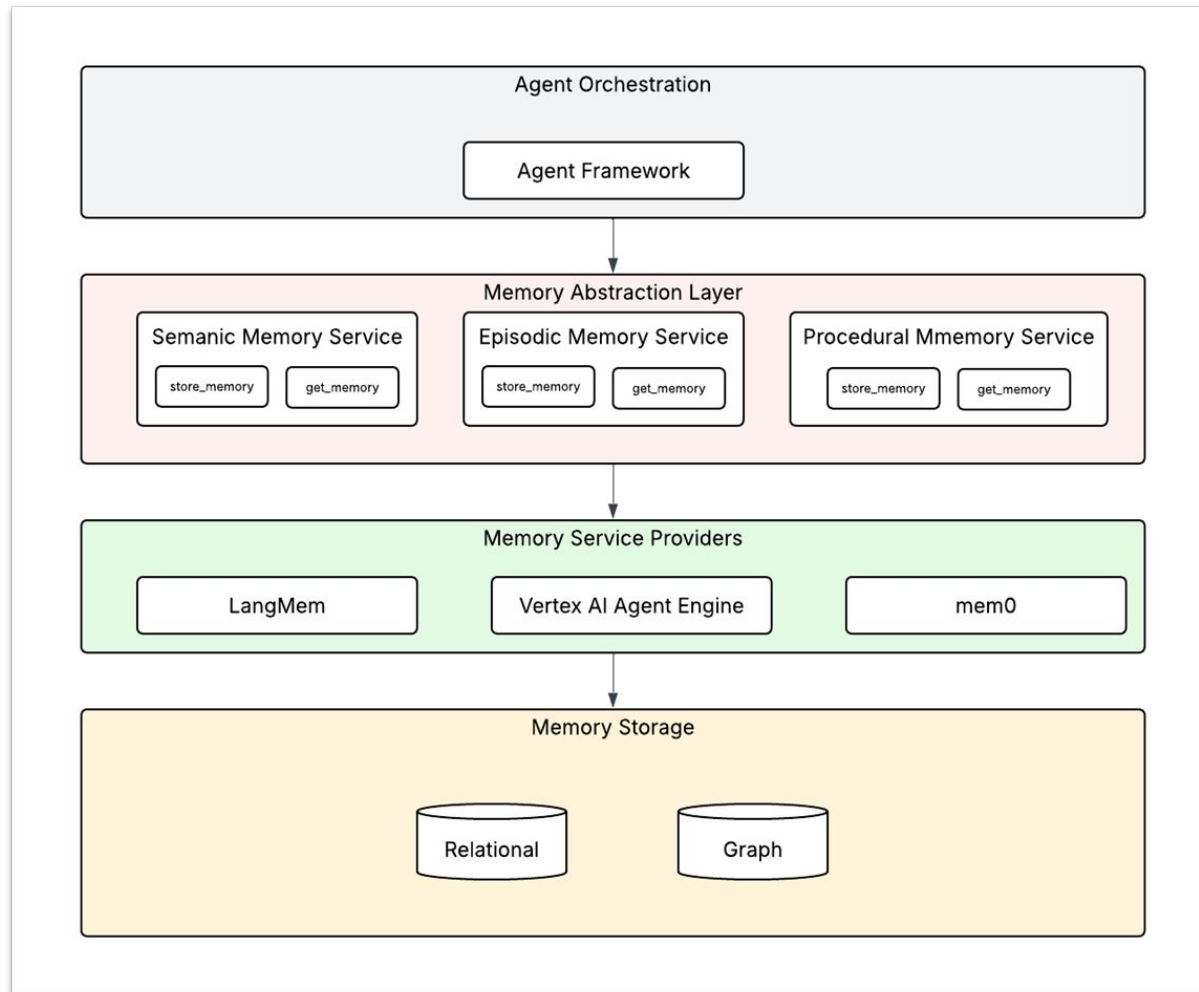
Storage Pattern: Collection

Semantic and Episodic memory are the most used memory



Agent Memory | Logical Architecture

This architecture separates agent orchestration from memory implementation so teams can evolve memory capabilities without rewriting the agents. By introducing a memory abstraction layer, we standardize how semantic, episodic and procedural memory are stored and retrieved in the organization. This will drive common standards and governance for the memory layer. The architecture has the below layers



Agent Orchestration

This is the run time where agents live and where the orchestration framework coordinates planning, tool calls, retries and multi agent collaboration. It decides when memory is needed, what context to request, and how memories influence the agent's reasoning and actions

Memory Abstraction Layer

This layer exposes standard and consistent API for memory operations, so agents don't depend on any specific vendor or storage design. It provides purpose-built services for semantic, episodic and procedural memory – each with clear “store” and “get” primitives, plus optional policies like TTL and scoping. The abstraction layer also standardizes memory schemas

Memory Service Provider Layer

This is the adapter/integration layer that plugs the abstraction APIs into specific memory frameworks or managed services like Vertex AI Memory Bank. It translates standardized memory calls into the provider specific capabilities and constraints

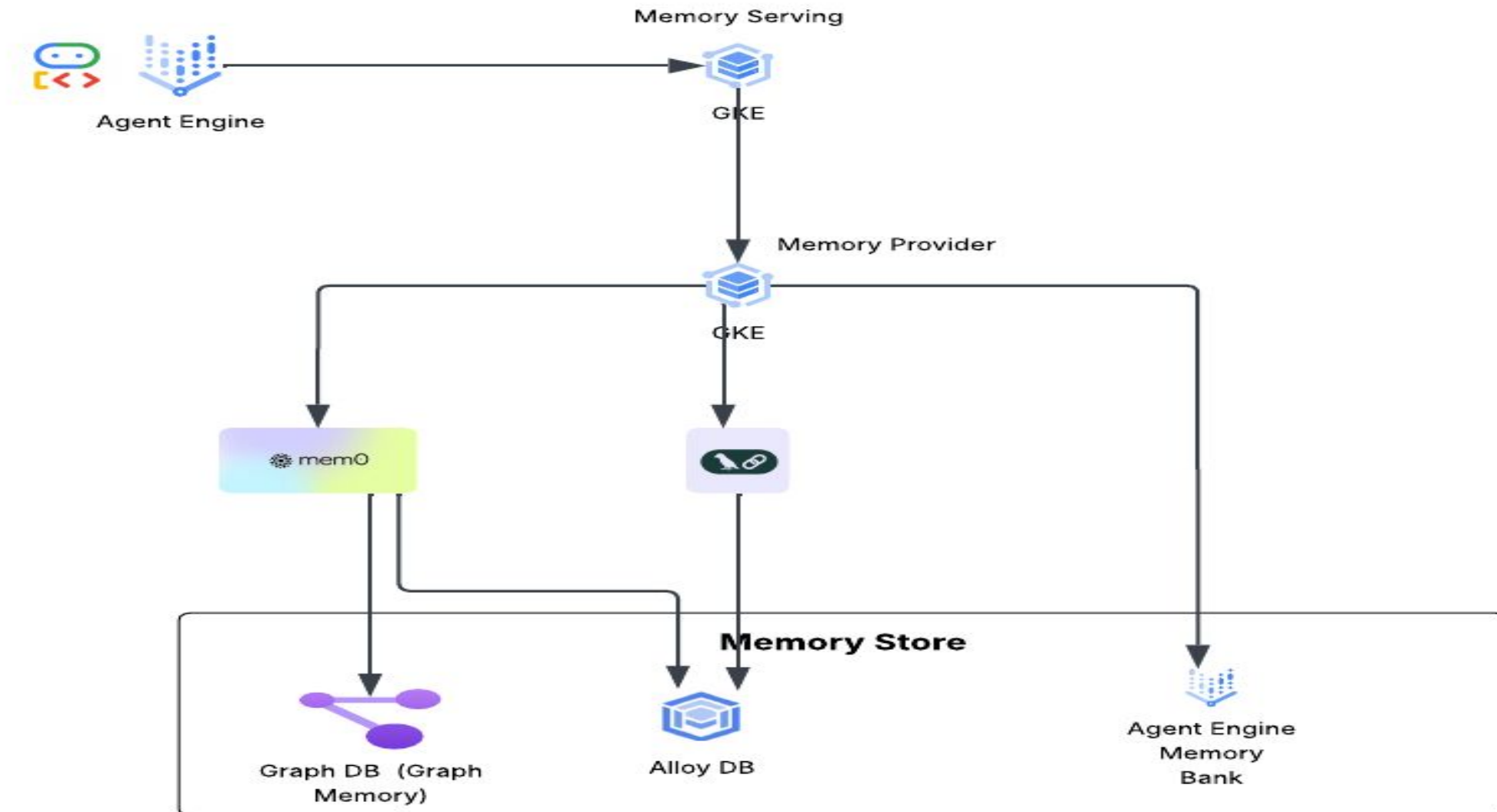
Memory Storage Layer

This is where memories persist over time. Depending on the memory type and retrieval needs, storage may be relational(strong structure and constraints), graph(relationships and traversals)

[Go to Lucidchart](#)

Agent Memory | Technical Architecture

[Go to Lucidchart](#)



Agent Memory | Examples

Memory Type	Purpose	Examples	Storage Pattern
Semantic	Facts and Knowledge	• Customer prefers eco-friendly brands and avoids leather • Shipping preference: always choose the fastest option under \$10	Profile or Collection
Episodic	Past Experiences	• Episode: Customer chatted on Jan 5 about a delayed package(Order # 48392). Agent issued a \$10 credit after carrier delay was confirmed; customer was satisfied	Collection
Procedural	System Behavior	• Returns workflow : confirm item eligibility -> offer exchange/refund options-> generate label->summarize next steps	Collection



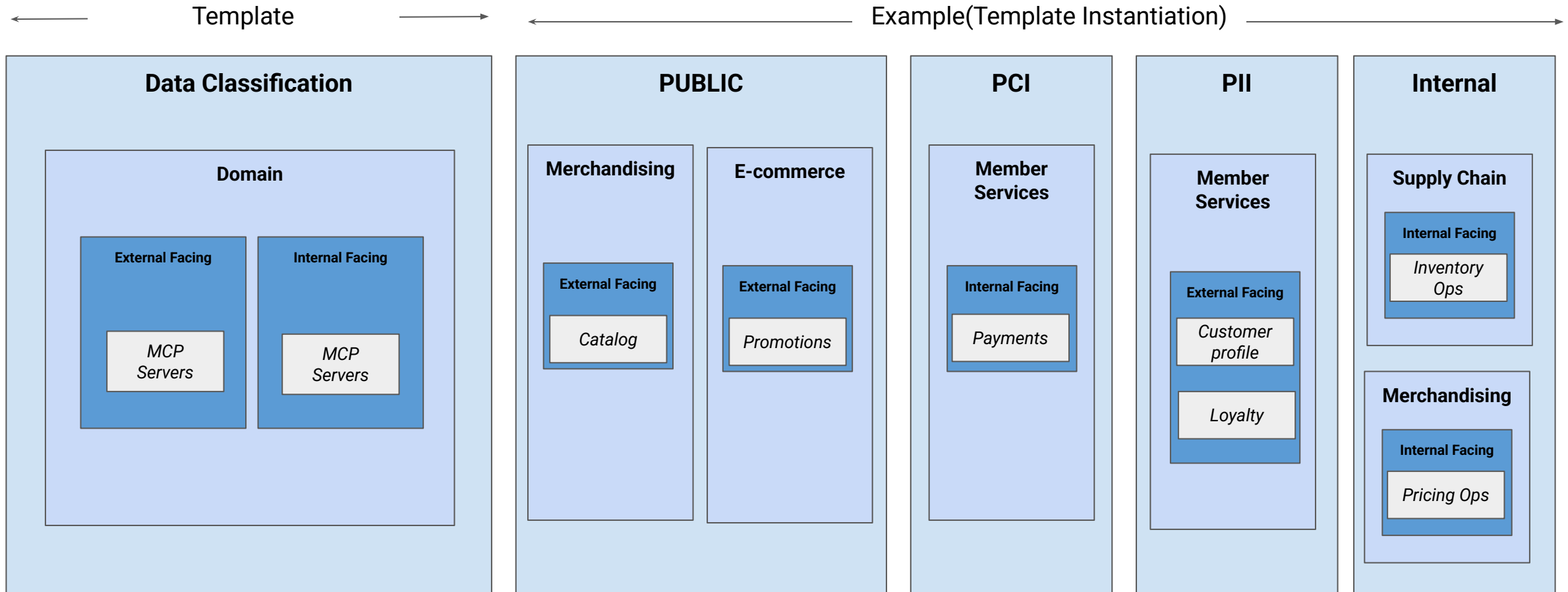


Gateway Patterns

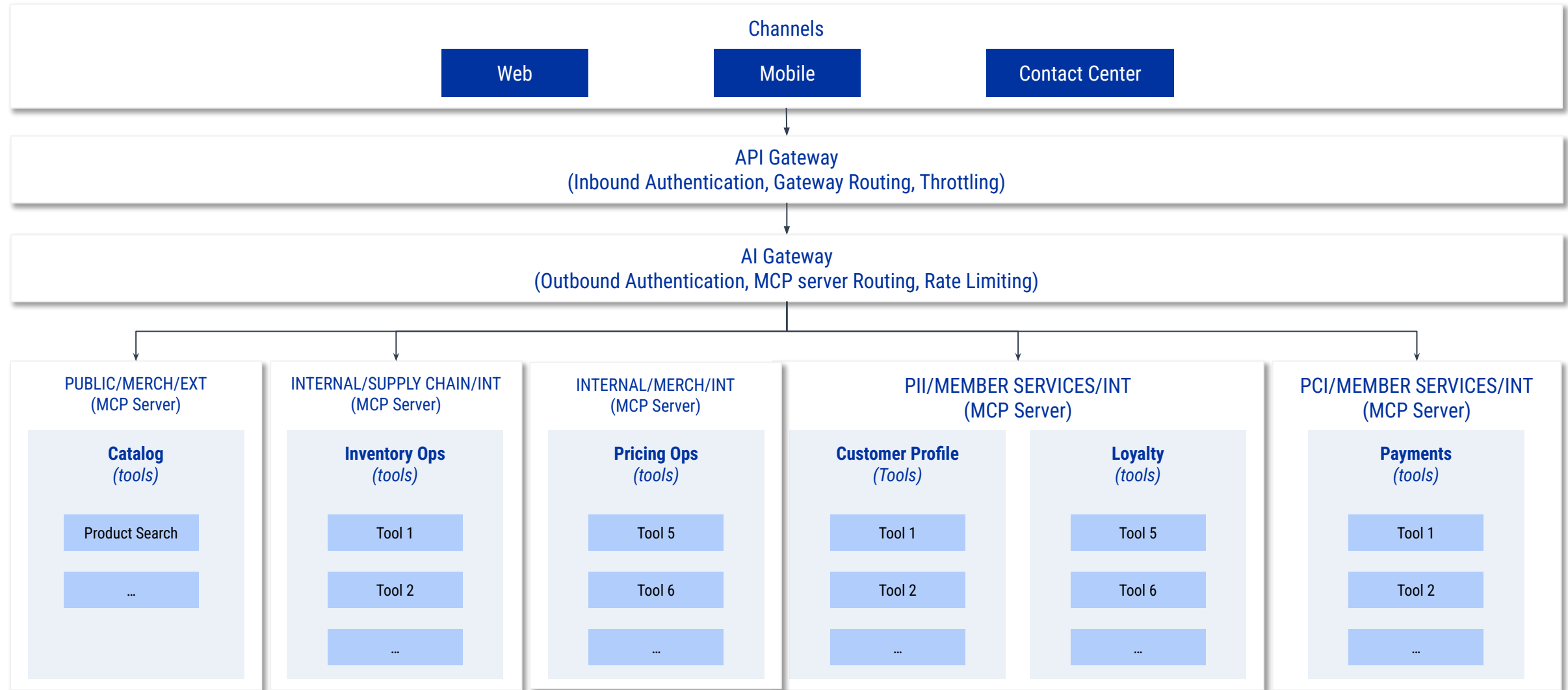
Design Patterns

[> Return To Catalog](#)

Gateway Pattern | MCP Deployment Strategy (1/2)



Gateway Pattern | MCP Deployment Strategy (2/2)



Gateway Pattern | MCP Transports

STDIO

The stdio transport is the most straightforward way to connect MCP hosts and servers locally. It relies on the standard system streams - stdin for sending messages and stdout for receiving them- to facilitate full-duplex communication. Because it operates via command line execution, it is ideal for local integrations where the server runs as a child process of the host. It is highly efficient for development because it requires no network configuration or port management; however, it is strictly limited to local environments where the host has direct execution access to the servers binary or script

Streamable HTTP

In streamable http transport, the server runs as an independent process that can handle multiple client connections via a single HTTP endpoint path that supports both POST and GET. Clients send each JSON-RPC message as a new HTTP POST to the MCP endpoint, and the server can reply either with a single JSON response or by opening an optional Server Sent Events(SSE) stream to deliver multiple server to client messages

While using streamable HTTP, it is recommended to use stateless streamable http approach. If we keep the server stateless, we get much easier horizontal scaling(no sticky session) , simpler load balancing, better resilience



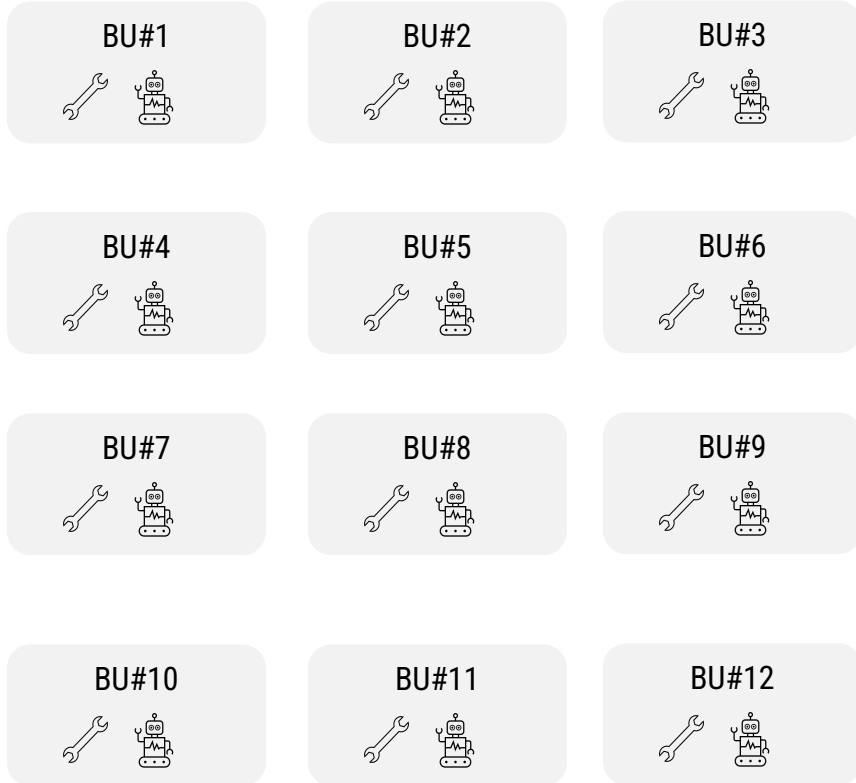


Agent Registry

[> Return To Catalog](#)

Agent Registry | Need for a registry

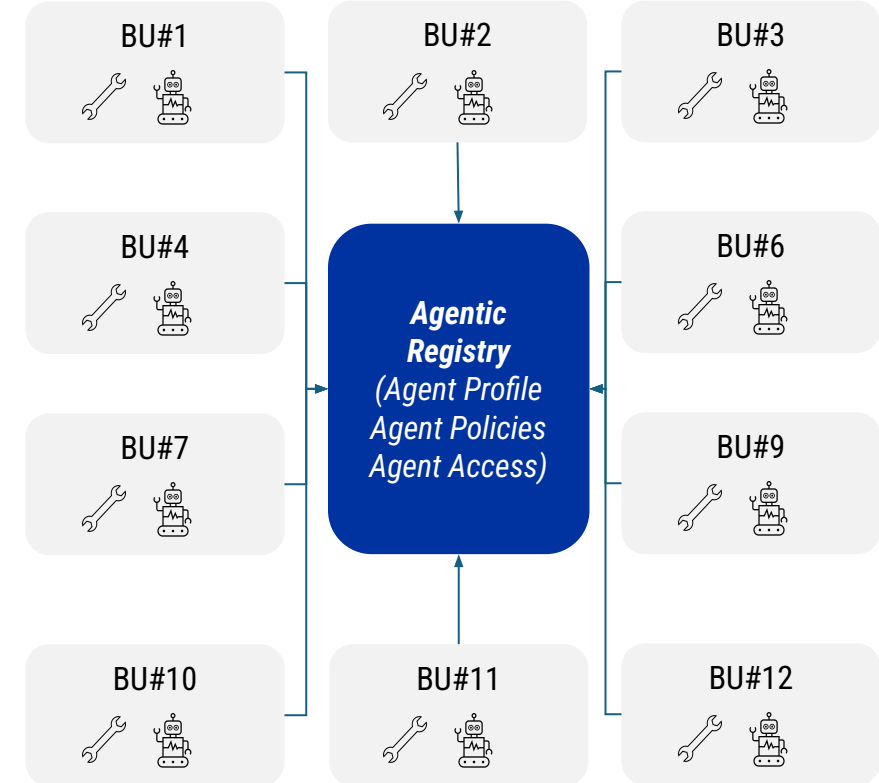
Organization Siloed Agentic Capabilities



- Lead to agentic proliferation
- Limited or no reuse of capabilities
- Limited or no governance of capabilities

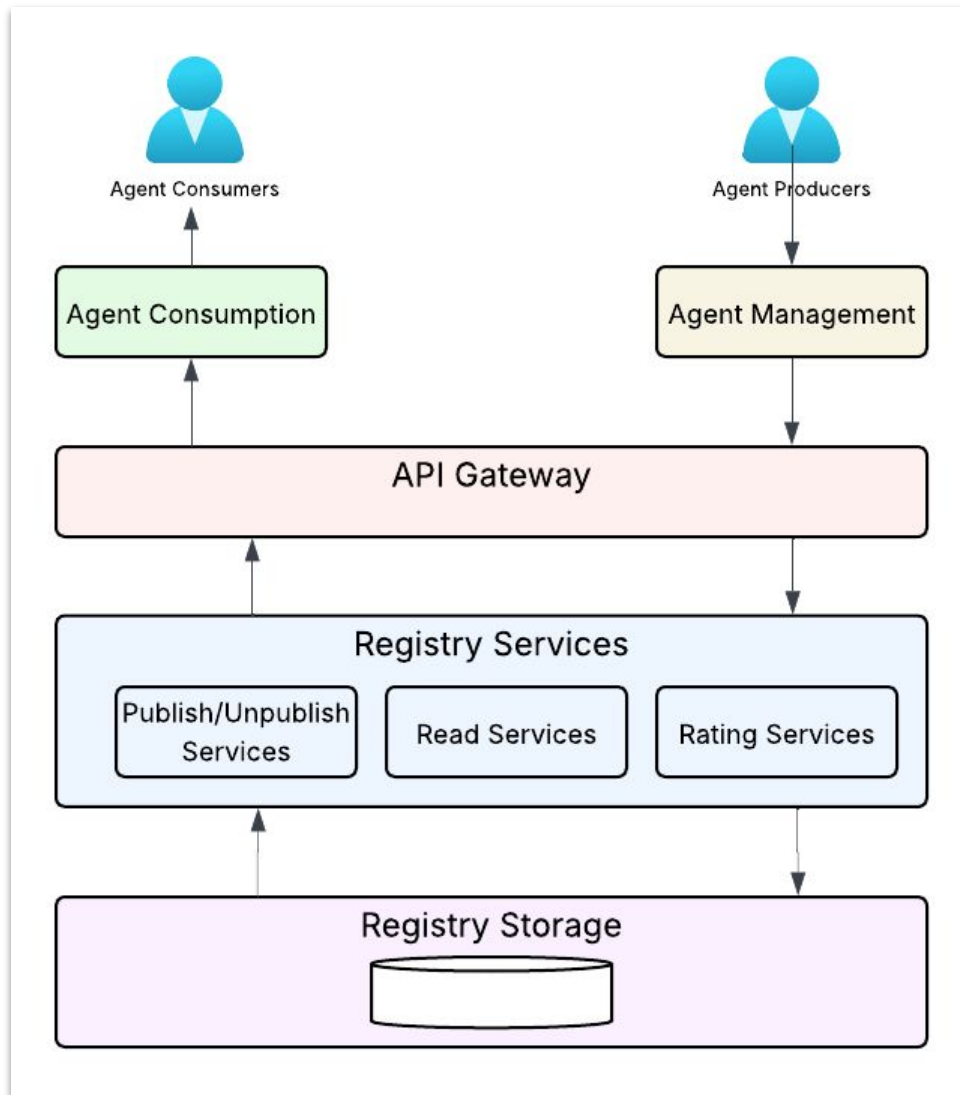
*Agent Registry enables
**discoverability, reuse
and a centralized
governance** for
responsible use of
agentic capabilities*

Organization Distributed Intelligence governed and consolidated with registry



- Rationalization of agentic capabilities and skills
- Promotes reuse of capabilities and skills
- Agentic capabilities governed through a common registry

Agent Registry | Logical Architecture



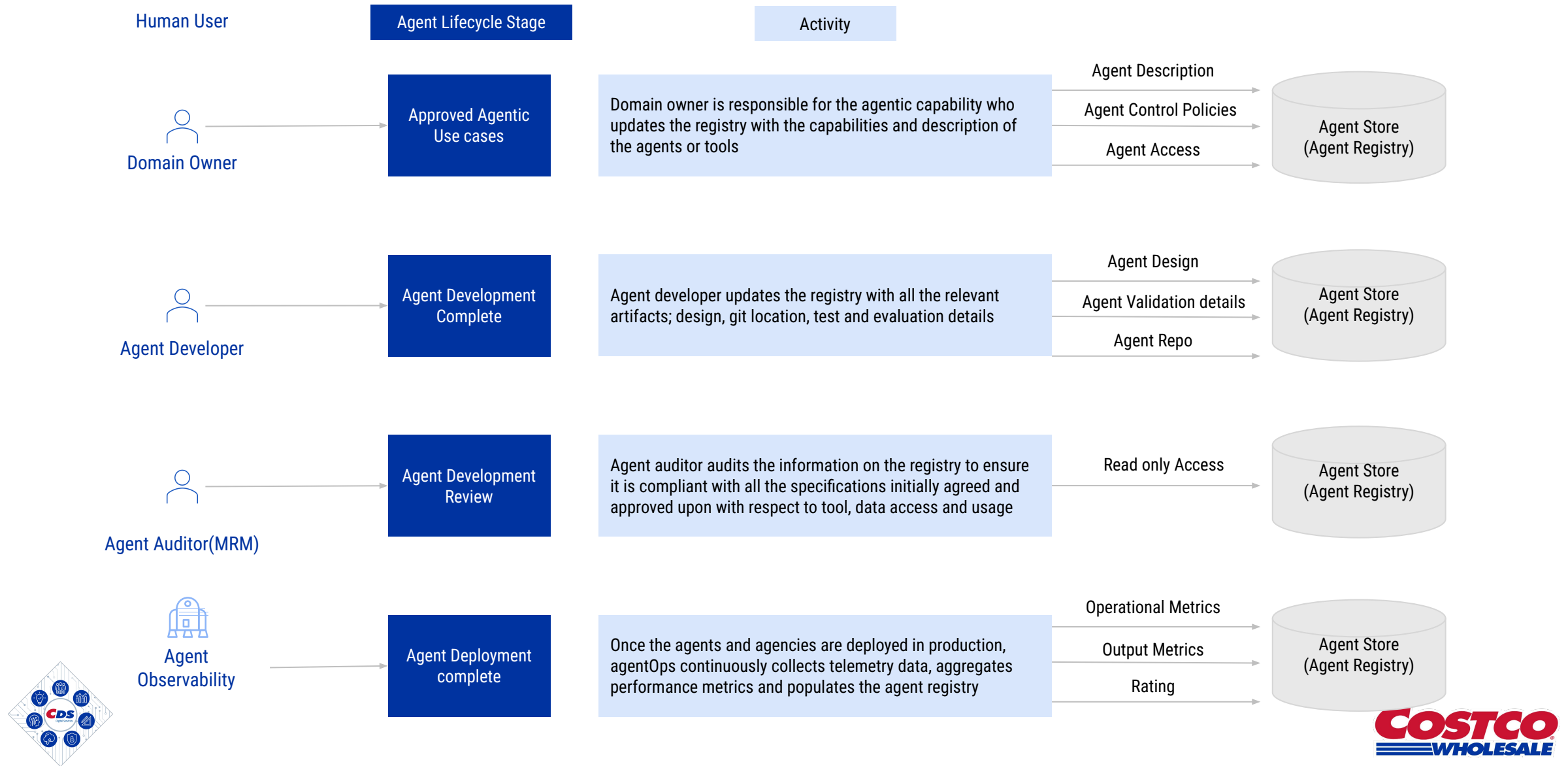
Agent registry enables both consumption and management

Agent management ensures a governed way of designing, development, deploying and continuous monitoring of agents

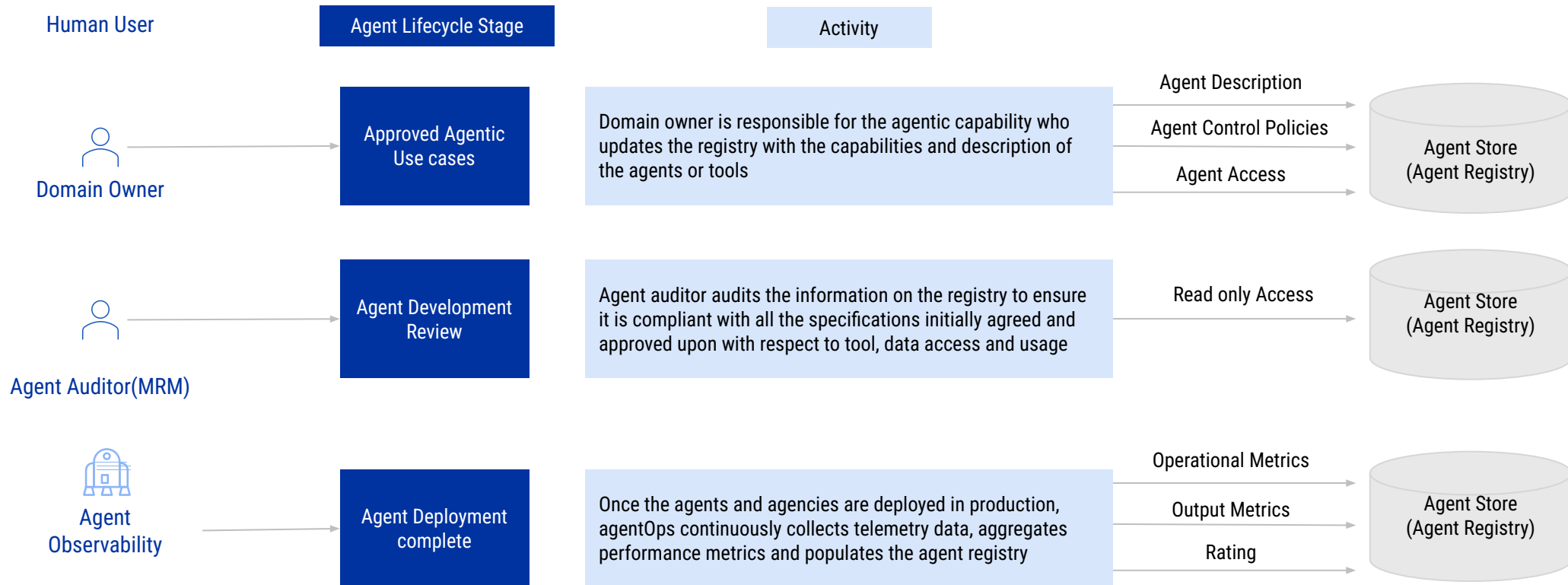
Four Core Roles

- 1 Domain Owner – The agent steward who sets the agent profile, agent policy and agent access details
- 2 Developer/Architects – Designs and develops based on the specification
- 3 Auditor – MRM (model risk management) team certifies for compliance and deployment
- 4 Agent Observability – Continuously appraises the deployed agents in production

Agent Certification-Onboarding Flow - 1st Party Agents



Agent Certification-Onboarding Flow - 3rd Party Agents

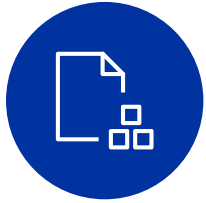




Agent Evaluation

[> Return To Catalog](#)

Agent Evaluation | Types of Grading



1. Code Based

- Deterministic programs that score an agent run by checking objective signals like exact outputs, tests, constraints, logs, tool-call schemas, or end-state invariants
- They return repeatable, deterministic results and are best when you can define clear correctness checks



2. Model Based

- Another model (often an LLM) that evaluates the agent's output and/or full trajectory using a rubric, checklist, reference answer, or pairwise comparison producing a score or preference
- They are useful for open-ended tasks where "correctness" is nuanced, but they can be noisy and should be calibrated/validated periodically



3. Human Graders

- People (SMEs or end users) who evaluate agent behavior and results using rubrics, preference judgments, or real-world A/B testing
- They are the highest fidelity signal for subjective or high stakes domains, but slower and more expensive – often used for calibration and spot-checking



Agent Evaluation | Code Base : Pattern Catalogs

Pattern	Name	Description
P01	String Match Checks	Compares the agent's output to expected text patterns-exact quality, regex rules, or approximate(fuzzy) similarity
P02	Binary tests	Run automated tests against the agent-produced artifact or the system state it changed; grade is pass/failed
P03	Static analysis	Analyze produced code/config without(or before) running it –style linting, type checking, dependency check, security scanners
P04	Outcome verification	Check whether the agent achieved the intended end result by verifying the final state of the environment (not the exact wording of the response)
P05	Tool call verification	Validate the agent used the correct tools(or avoided forbidden ones) and that tool parameters are well formed within allowed bounds
P06	Transcript analysis	Score the agents behavior using telemetry from the interaction tarce-how long it took, how much steps, how much it spent and how often it retried



Agent Evaluation | Model Based | Pattern Catalogs

Pattern	Name	Description
P01	Rubric based scoring	An LLM grades an agents output against a defined rubric with criteria like correctness, completeness, reasoning quality, tone, safety, and tool use. It returns per-criterion scores and an overall pass/fail or weighted score
P02	Natural Language assertions	An LLM evaluates a checklist of yes/no statements about the output or transcript, such as “mentions key risks”, “cites sources”
P03	Pairwise comparison	An LLM is shown two candidate outputs for the same task and chooses the better one based on a rubric
P04	Reference based evaluation	An LLM compares the candidate outputs to a provided reference(gold answers) and score similarity/correctness with semantic similarity
P05	Multi judge consensus	Run multiple LLM grading passes(ensemble) and combine them to reduce variance and single-judge bias

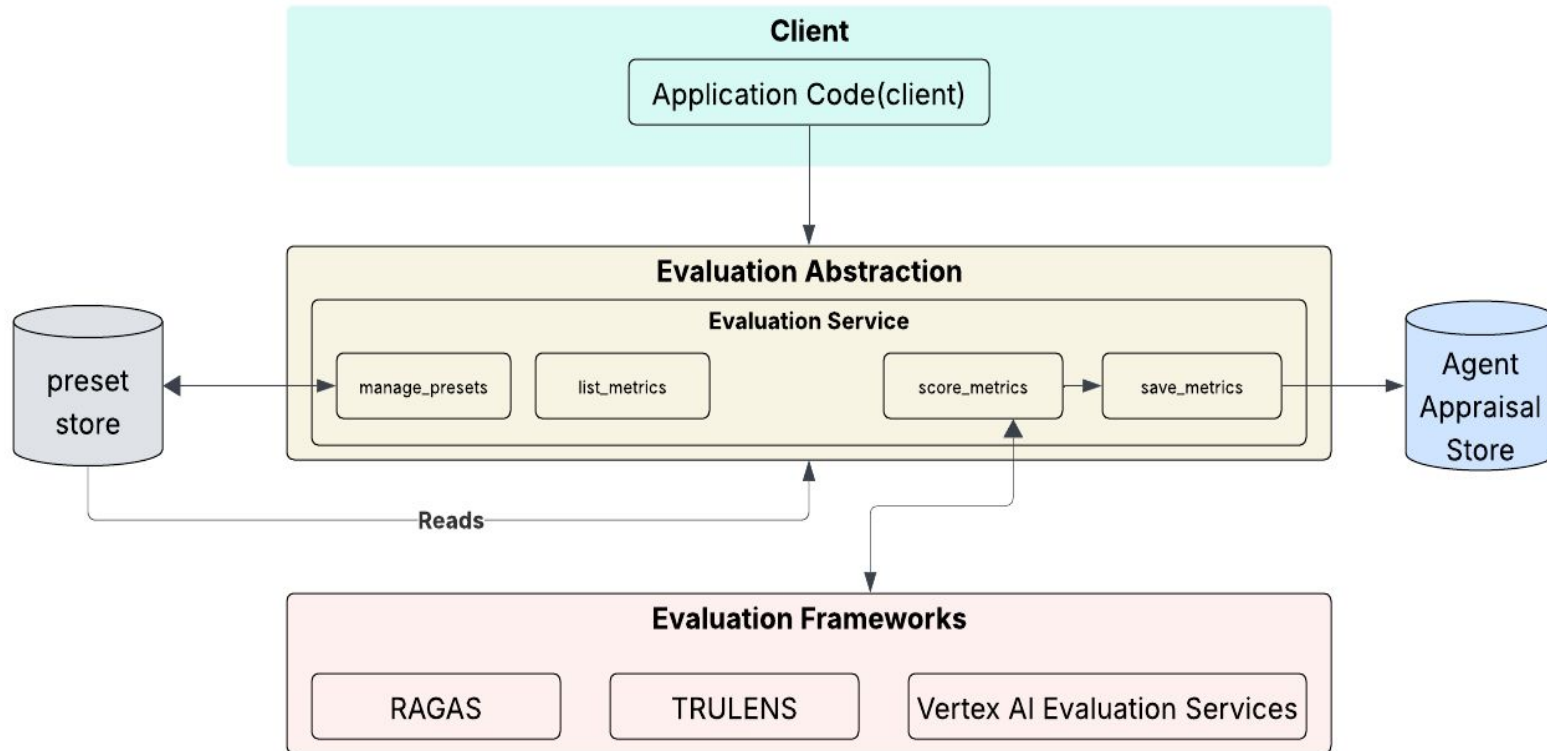


Agent Evaluation | Human Graders | Pattern Catalogs

Pattern	Name	Description
P01	SME Review	Subject matter experts evaluate agent outputs and/or full transcripts using a rubric, often adding qualitative notes on correctness, risk and domain appropriateness
P02	Crowdsourced judgement	Trained crowd workers rate outputs using simplified rubrics or preference tests; quality is maintained with training, gold questions and audits
P03	Spot check sampling	Humans review a strategically selected subset of runs to validate automated graders, catch new failure modes and monitor drift over time
P04	A/B testing	Real users are split between two agent variants (A and B) in production (or a pilot). Compare outcome metrics such as task completion rate, time to resolution, escalations, user satisfaction, error rates



Agent Evaluation | Evaluation Framework



Evaluation Abstraction - The evaluation abstract layer abstracts the functionalities of the underlying evaluation frameworks and exposes a standard set of evaluation services

- **manage_presets** - This function helps create, update and manage a set of metrics for a particular use case
- **list_metrics** - This function will list all available metrics in this layer
- **score_metrics** - This function will instantiate the respective evaluation framework and will calculate the score of the given metrics
- **save_metrics** - The calculated metrics is stored in the metrics store

Evaluation frameworks - This layer will have the actual implementation of the evaluation metrics which will be exposed in a standard way in the evaluation abstraction layer



[Link](#)



Agentic AI System Performance Metrics*

Multi Agent System Metrics

- Number of steps to complete a workflow
- Number of conversation turns between agents
- Agent Planning Accuracy

Agent Metrics

- Successful task completion rate
- Time to complete task
- Number of turns/steps to complete a task
- Tool invocation accuracy
- Token usage per task

Model Metrics

- Number of rate limiting errors(429)
- Total number of model API access errors
[errors\(4xx\)](#)

RAG Metrics

- Context relevance
- Answer Relevance
- Groundedness
- Query, context and response token count

** Non functional metrics related to Agentic AI system performance.*



Agentic AI Evaluation Best Practices

Eval Preset for each Agentic System

A pre-packaged set of evaluations specific to the agentic system that ships with the system and gets automatically executed as part of CI/CD and also on a continuous basis as the agentic system executes in production. For example, for a summarization agent, an eval preset will include ***adherence_score, coverage_score***

Use a mix of Code, Model and Human grading

Leverage a layered evaluation stack-code based checks for deterministic failures(schemas, tool-call validity, safety rules), model-based judges for rubric scoring at scale(helpfulness, reasoning, quality, trajectory quality) and targeted human review to calibrate the rubric and audit high-impact edge cases. This mix provides fast CI gates, broad coverage and human aligned trust in scores

Prefer deterministic over non-deterministic judge

Prefer deterministic code-based grading and human review over LLM-judging whenever the criterion can be measured objectively

Drift watch per agentic system

Continuously track each agentic system's responses and behavior over time using a fixed set of "golden" scenarios plus live sample traffic. Alert on semantic and output changes so that silent regressions for model updates, prompt tweaks, toll/schema changes can be caught before users do

Sampling Based LLM-Judge evaluation

Leverage sampling-based evaluation for LLM or Model graded evaluation to reduce the cost of evaluation. Samples can be based on high-risk intents, key workflows.

Independent Judge Model

It is recommended to use a different model than the one powering the agent to act as LLM-as-judge. This reduces self-grading bias and model blind spots

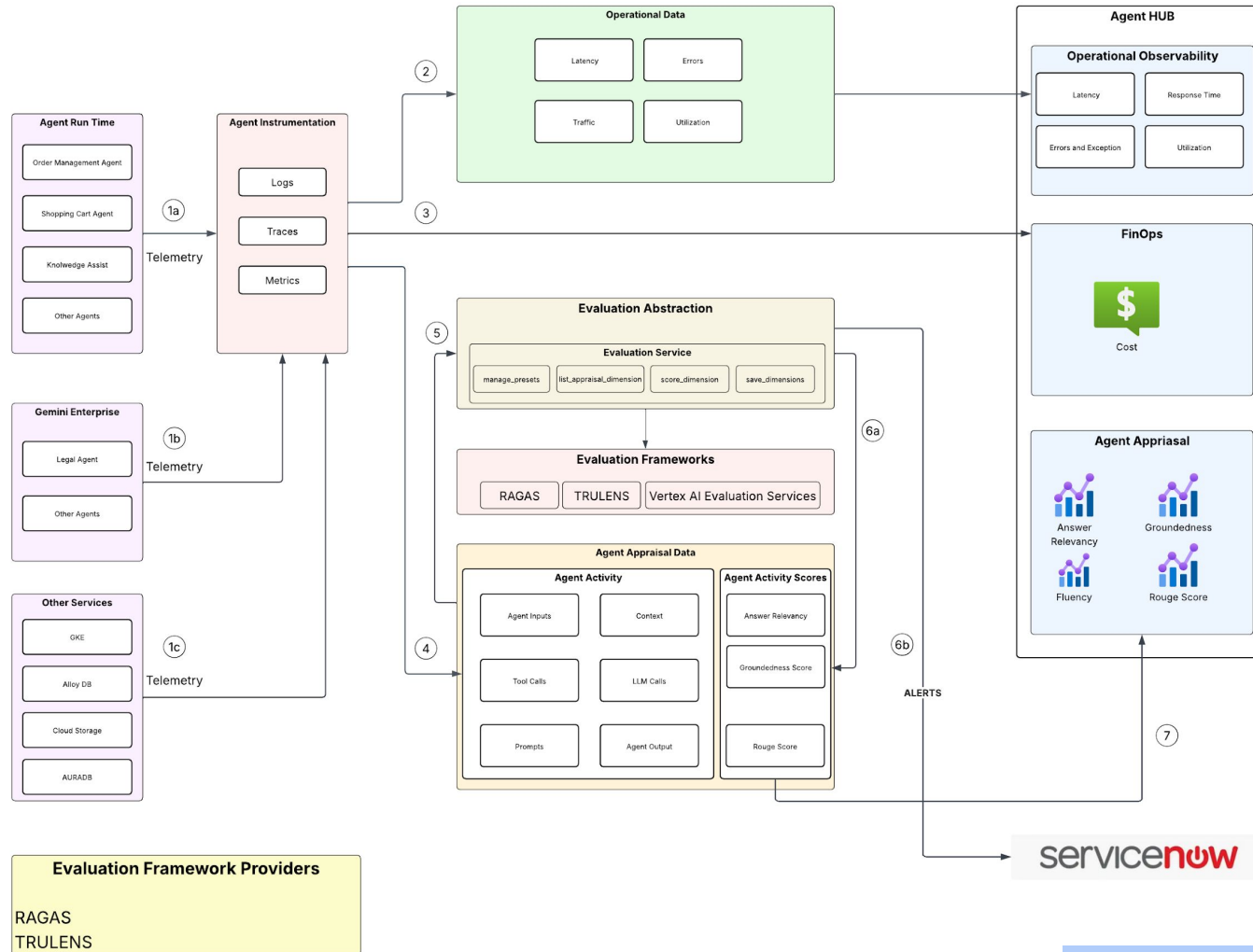




Agent Observability

[> Return To Catalog](#)

Agent Observability | Logical Architecture



1

Agents running in agent run time and other services pushes agent activity details to the Agent Activity Layer(Telemetry)

2

Operational metrics from the Agent Activity layer is collected and pushed to operational observability portal

3

Token usage and cost related information is pushed to FinOps Hub

4

Agent appraisal data in the form of agent input, output, prompt etc is collected

5

Evaluation framework leverages the appraisal data to calculate the appraisal scores

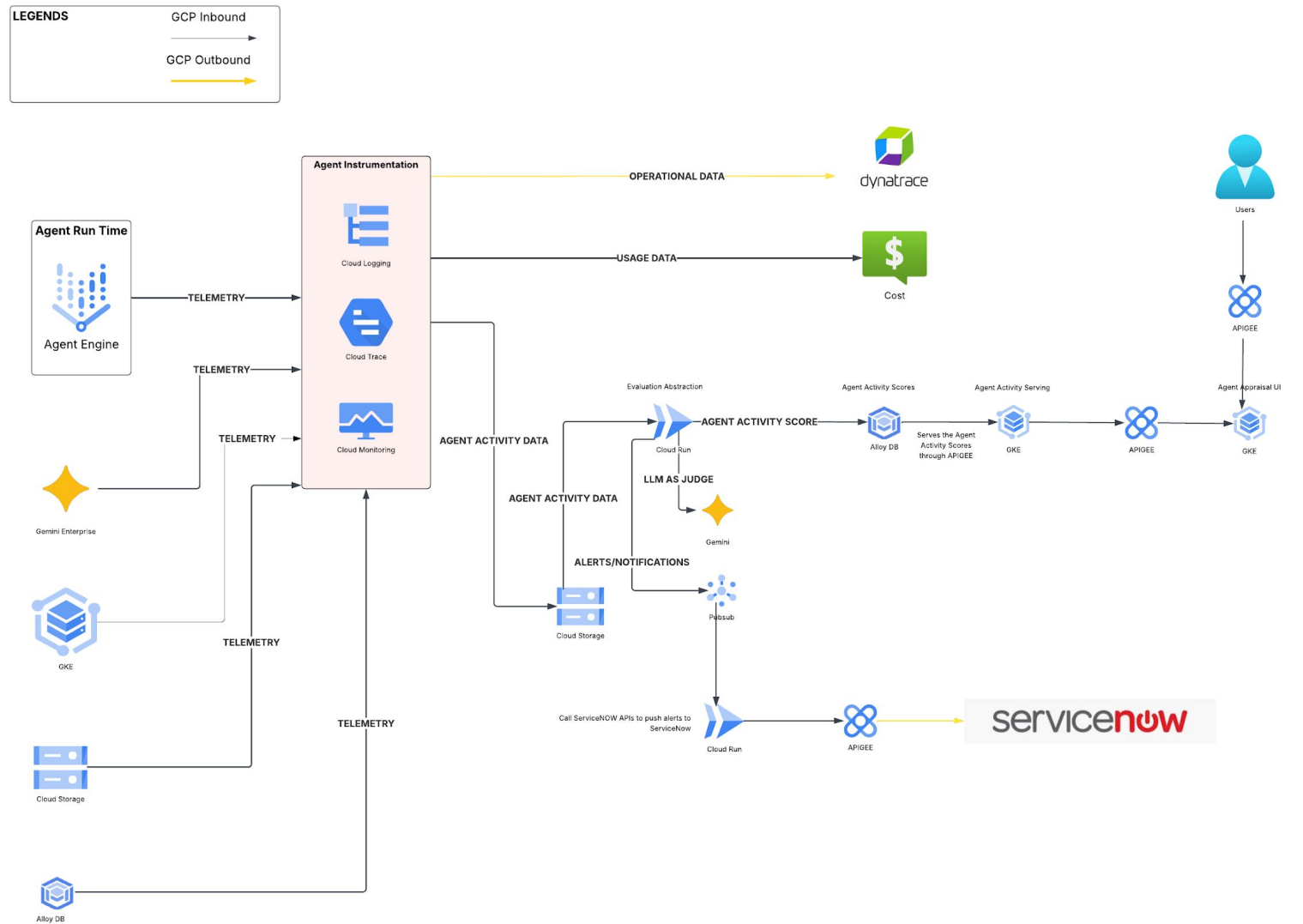
6

- a. Agent Appraisal scores are calculated and stored in Agent Appraisal store
- b. Any alerts notification is sent to service now

7

Agent Appraisal scores are visualized through the Agent Appraisal dashboard within the Hub

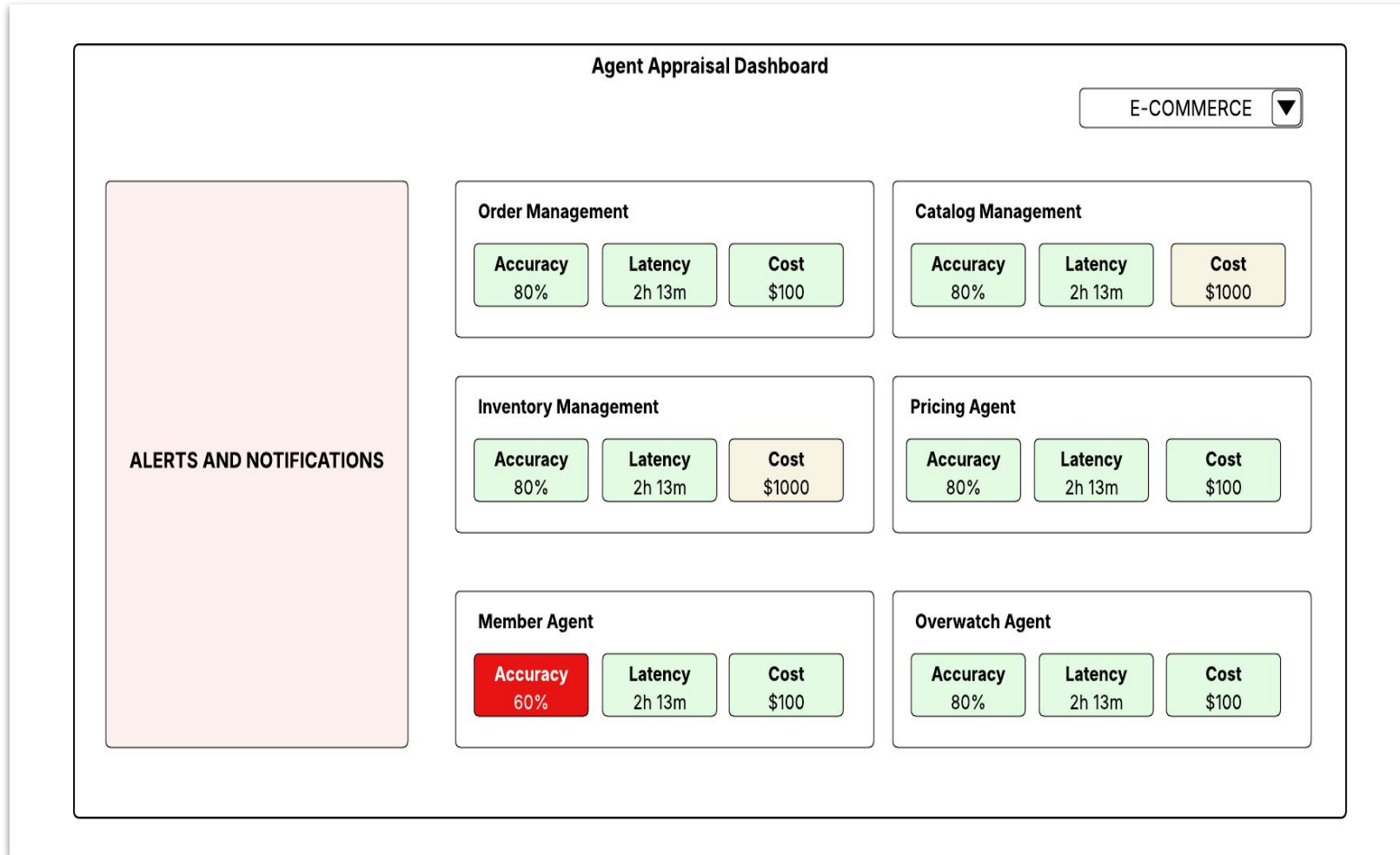
Agent Observability | Technical Architecture



[Go to Lucidchart](#)



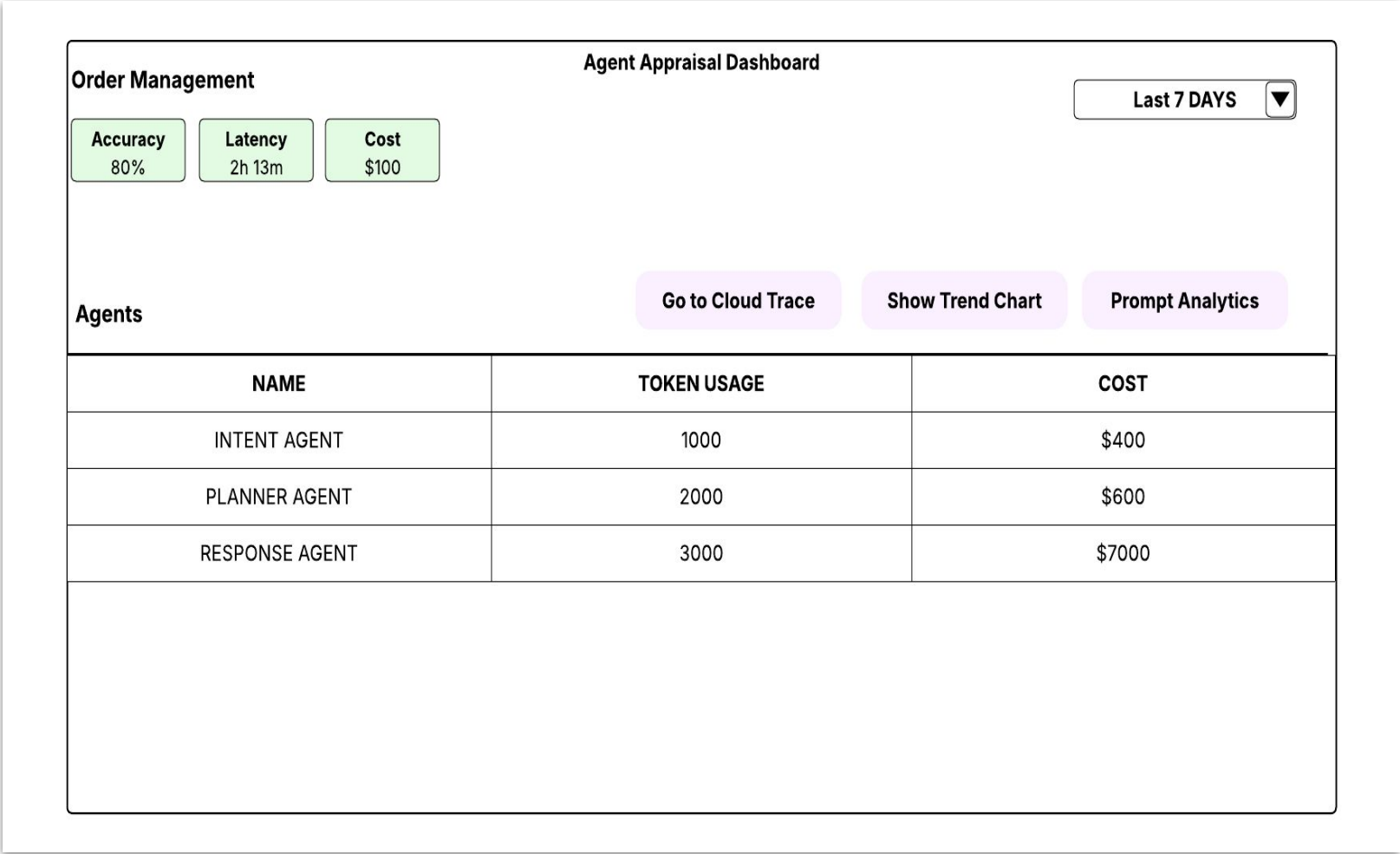
Agent Observability | Illustrative Wireframes



A consolidated view of all the active Agentic flows running in the organization



Agent Observability | Illustrative Wireframes



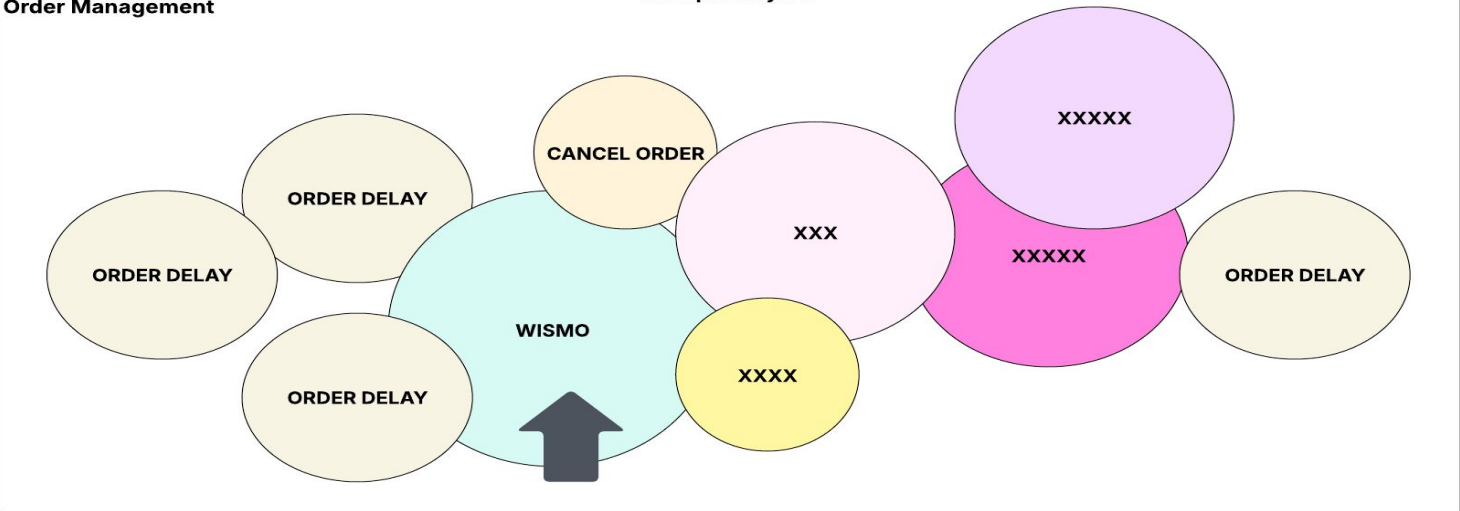
A Drill down view of a particular agentic workflow



Agent Observability | Illustrative Wireframes

Order Management

Prompt Analytics



Prompt	Number of tokens	Date
Where is my order	1000	09-09-1025
I am not able to find my order	10000	09-09-1025
My order is not traceable	2000	09-09-1025
I am not finding my order	4000	09-09-1025
I am not finding my order	4000	09-09-1025
I am not finding my order	4000	09-09-1025
I am not finding my order	4000	09-09-1025

Prompt Analytics for a particular Agentic flow

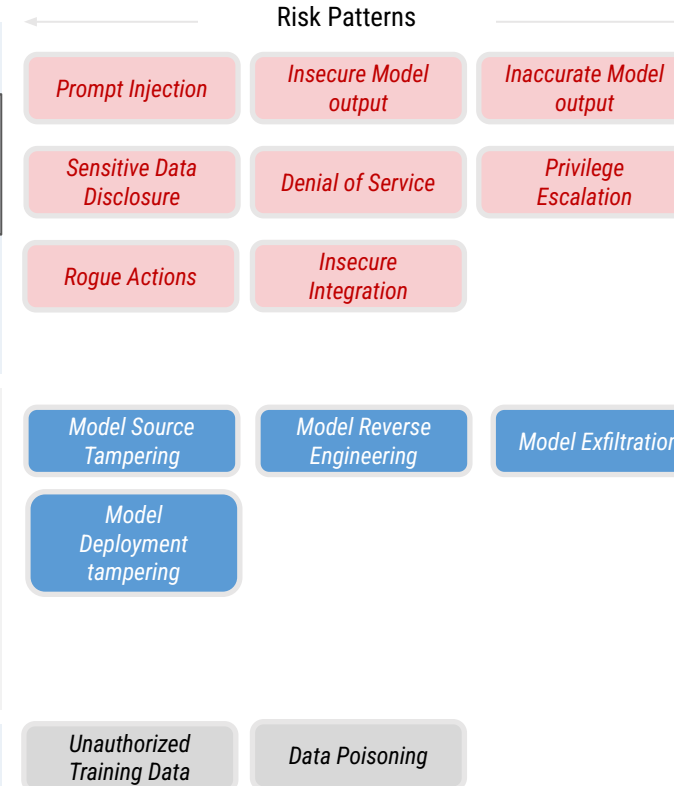
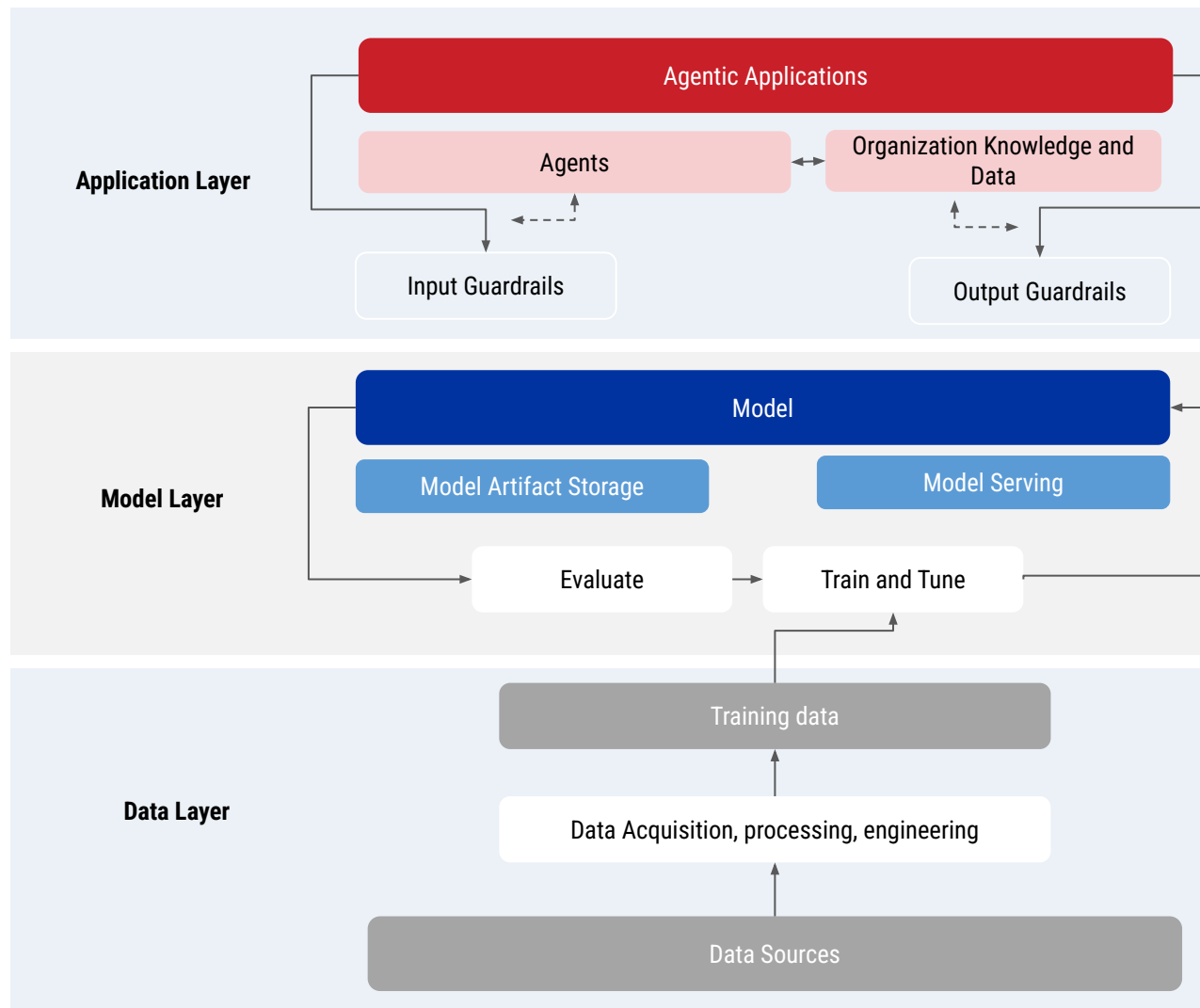




Agent Security

[> Return To Catalog](#)

Agent Security | Risk Patterns



Direct Control

This risks can be directly controlled by the consumers through secure application development

Indirect Control

Consumers do not have direct control on these risks since the models are not trained by consumers. Providers train and expose the models as hosted services. These types of risks are mitigated through adversarial testing





Risk Patterns ***Application Layer***

Agent Security | Risk Patterns & Mitigations (1/2)

Risk	Description	Mitigation
Prompt Injection	Prompt Injection occurs when a user(external content) manipulates an agent's instructions so that it ignores its original goals, rules or safety boundaries. There are primarily three types of prompt injection pattern 1. Direct Prompt Injection : Malicious instructions are directly entered into the prompt 2. Indirect Prompt Injection: Malicious instructions read from external data sources(e.g. documents, web pages) 3. Jailbreaks : Malicious instructions cause models to act in ways they were trained to avoid	• This risk can be mitigated through following prompt engineering best practices and leveraging input guardrails. • Input Guardrails: Configure input guardrails to detect and de-identify sensitive data. Use Model Armor to configure prompt injection and jail break detection • Separate Instructions from Inputs : Separate System Prompt from user prompt. Avoid to add any PLACEHOLDERS within system prompt • Human in the loop for high impact actions : High impact actions should not be auto approved, those should implement human in the loop • Explain-before-Act: Use chain of thought prompting to be able to monitor the agents' thought traces. • Source Attribution: Always engineer the system to show source attributions for response provided by agents
Sensitive Data Disclosure	This risk occurs when an agent exposes private, confidential, or regulated information to the wrong person, system or context	• Mitigate this risk by following: • Data minimization by default : The agent is provided access to data sources following least privilege principle • Data Sensitivity Classification: Capture data classification as metadata and use the metadata to filter relevant context for the agents • Output Guardrails: Configure output guardrails to detect and de-identify sensitive data. Use Model Armor templates with Advance detection type to detect sensitive data and prevent accidental sensitive data disclosure
Rogue Actions	Rogue actions occur when an agent takes actions that were not explicitly authorized, expected, or intended by the user, even if the action appears logical from the agent's internal reasoning	• Explicit permission boundaries: The agent is enabled with an approved set of tools following least privilege principle • Preview before execute workflow: For any high-risk actions, the agent should show the plan and list of tasks before executing them.



Agent Security | Risk Patterns & Mitigations (2/2)

Risk	Description	Mitigation
Insecure Model Output	Insecure model output occurs when an agent produces responses, recommendations, or actions that are unsafe, misleading or inappropriate for the user's context, even though the system itself has not been directly compromised	Content Filters: All the foundation models can be enabled with content filters. Content filters must be enabled before using any foundation models. Output Guardrails: Implement output guardrails to ensure any domain specific insecure data is detected by the guardrails before sharing the response/output with stakeholders.
Denial of service	Denial of service occurs when an agent or agentic system becomes unavailable, unresponsive or degraded due to attackers either overwhelm the model with excessive calls or use carefully crafted "sponge examples" that take advantage of model weakness to degrade performance	Mitigation must be applied at the application level using input guardrails and employing rate limiting and load balancing mechanisms to control the volume of calls to the agents
Insecure Integration	This risk occurs when an agent is integrated with tools/plugins that are not authorized and approved which then permits manipulation of inputs and outputs	This risk must be mitigated through use of only approved and authorized tools from the tool registry. Any tool access is routed through AI gateway
Inaccurate Model output	This risk occurs when the agent responds with output that are not grounded to the context of the use case. It provides hallucinated response	This risk must be mitigated at both knowledge and application engineering stages. The agent must be constrained to a decision boundary through a domain specific knowledge model, optimized prompt and fit-for-purpose tools
Privilege Escalation	This risk occurs when an agent gains or exercises permissions beyond what the users or system explicitly granted, allowing it to access data, tools or actions it should not be able to perform	Least privilege by default: The agent is provided minimal access to data sources and tools based on its role boundaries Time bound and task bound privilege: All agents are provided an identity and the permissions to those identities are managed through IAM. In addition, enable time bound privileges



Risk Patterns

Model Layer

Agent Security | Risk Patterns & Mitigations (1/1)

Risk	Description	Mitigation
Model Source Tampering	This risk occurs when the model code, training frameworks, or model weights are not hardened against supply chain attacks and tampering	The direct mitigation needs to be at the model provider end. At the consumer end, deploy evaluations to monitor response drift from the models
Model reverse engineering	This risk occurs when attackers send excessive request to the model and leverage the responses to reverse engineer the weights	The direct mitigation needs to be at the model provider end. At the consumer end, control access through API and employing rate limiting
Model exfiltration	This risk occurs when storage or serving infrastructure does not have adequate security against attacks which lets attackers target vulnerabilities in serving or storage systems to steal model code or weights	The control need to be at the provider end
Model Deployment Tampering	This risk is introduced when the model serving infrastructure is vulnerable to manipulations	The control need to be at the provider end





Risk Patterns

Data Layer

Agent Security | Risk Patterns & Mitigations (1/1)

Risk	Description	Mitigation
Unauthorized training data	This risk is introduced during model development when training data is not properly filtered out during data ingestion, processing and model evaluation during training	The control need to be at the provider end
Data poisoning	This risk occurs when the data intentionally contaminated causing the model to behave incorrectly, unfairly or unsafely after deployment	At the consumer end, this risk needs to be controlled through extensive adversarial testing and evaluation

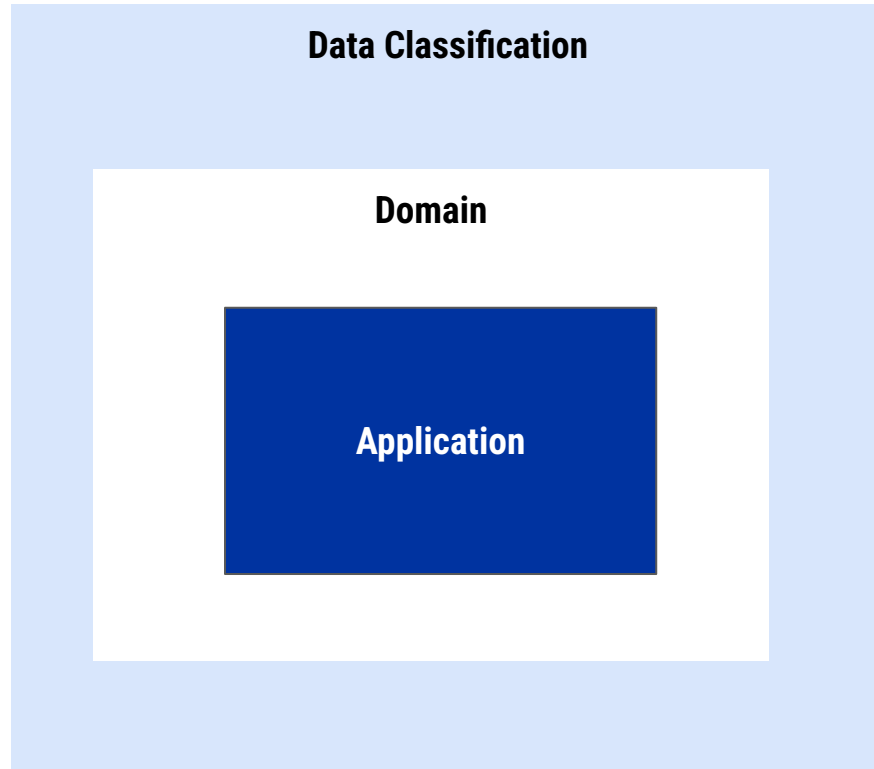




Multi Tenancy

[> Return To Catalog](#)

Multi Tenancy Approach



Multi-tenancy Approach will follow the below hierarchy

Data Classification - There will be a physical isolation at data classification level

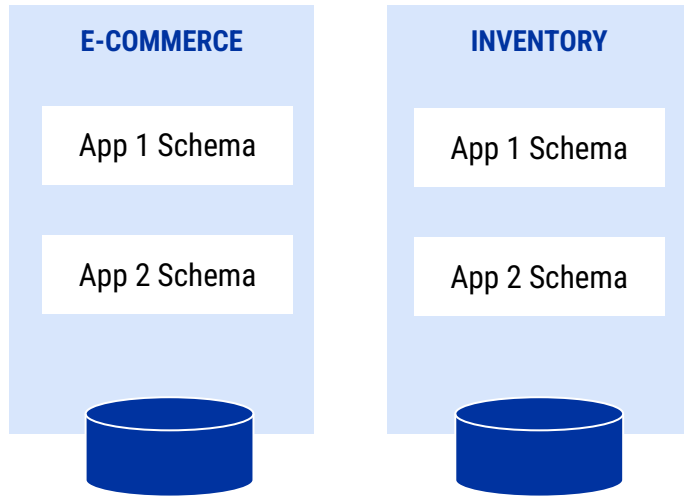
Domain - There will be a physical isolation of the set of services at the domain level

Application - Within a particular data classification and domain, services will be further logically isolated using schema-based separation

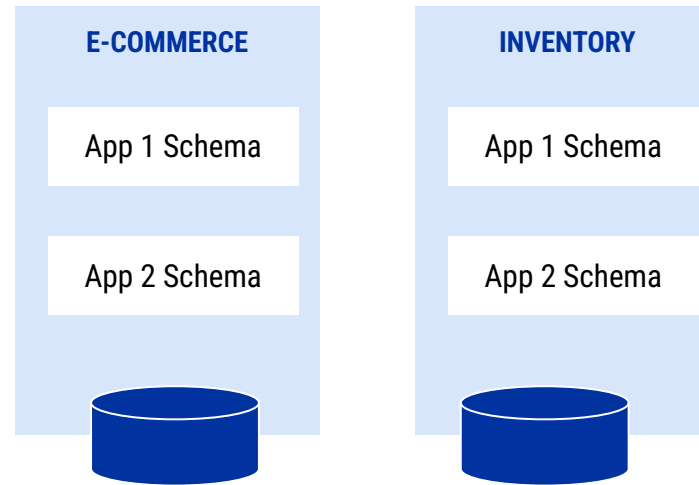


Multi Tenancy Approach | Vector Stores

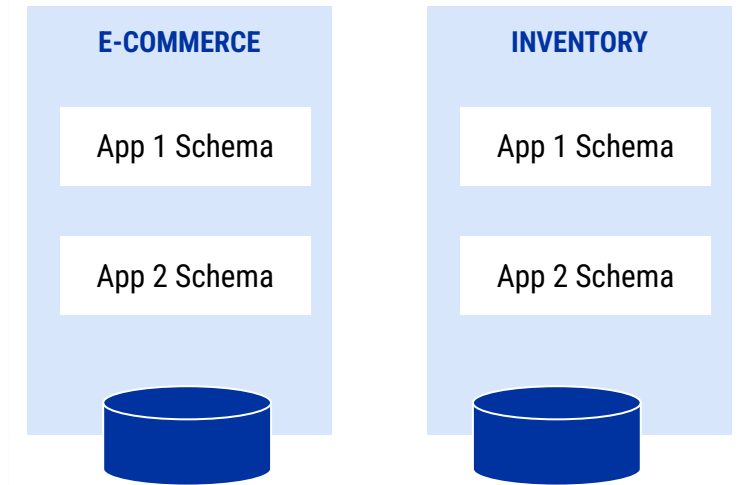
PII



PCI



PUBLIC

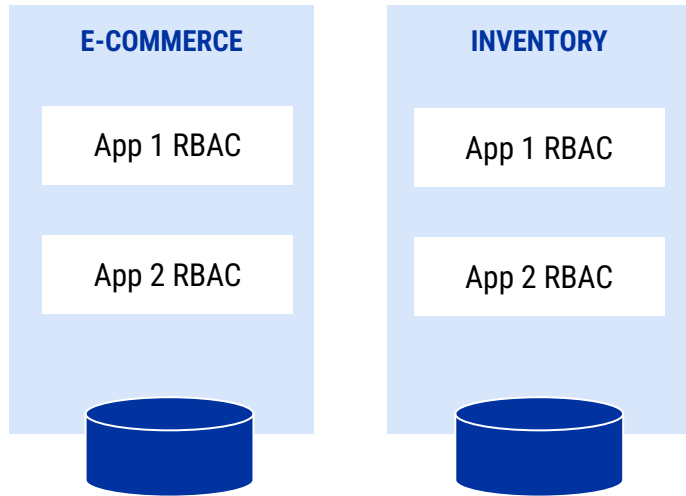


Each data classification has vector stores that are physically isolated. Within the data classification vector stores are physically isolated by domain. With a given domain, vector stores are logically isolated per application

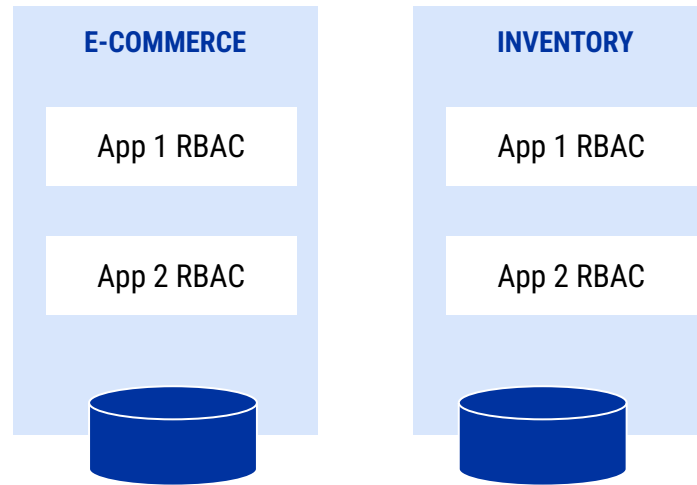


Multi Tenancy Approach | Knowledge Stores

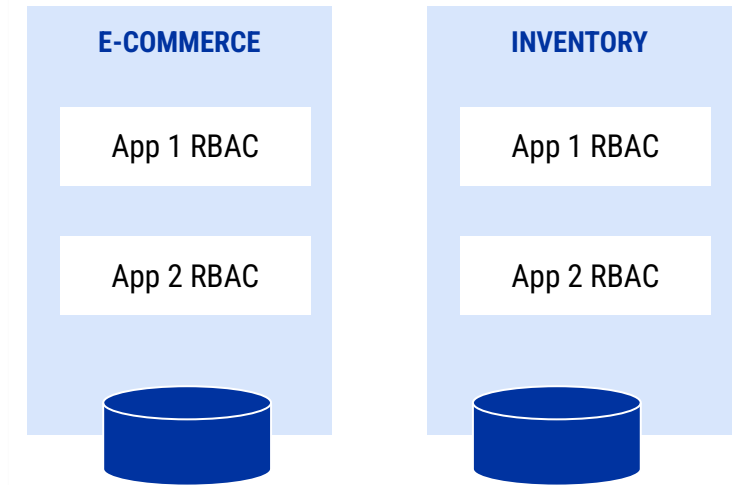
PII



PCI



PUBLIC



Aura DB can have one DB per Instance. For MVP, we plan to create one Instance for each data classification and domain group. Multi DB creation feature within an instance will be available end of June or September. We will consider moving to multi-DB pattern when the feature will be available in Aura DB



Multi Tenancy Approach | AI Gateway

APIGEE ORGANIZATION

ENVIRONMENT GROUP-1 (Non Prod)

Data Classification

Domain

ENVIRONMENT GROUP-2 (Prod)

Data Classification

Domain

- A dedicated AI gateway will be stood up for Agentic A
- Environment groups will be used for physically isolate production and non-production
- Within each environment group, logical isolation will be achieved through proxy and base path for isolation based on data classification and domain





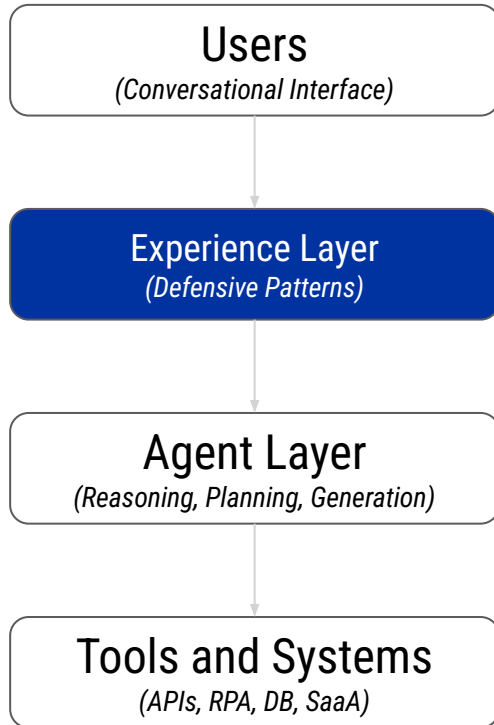
Defensive UI

Interface Patterns

[> Return To Catalog](#)

Defensive UI | Where does it fit?

Conversational Agentic Applications need an experience layer that mediates autonomy, risk and user control



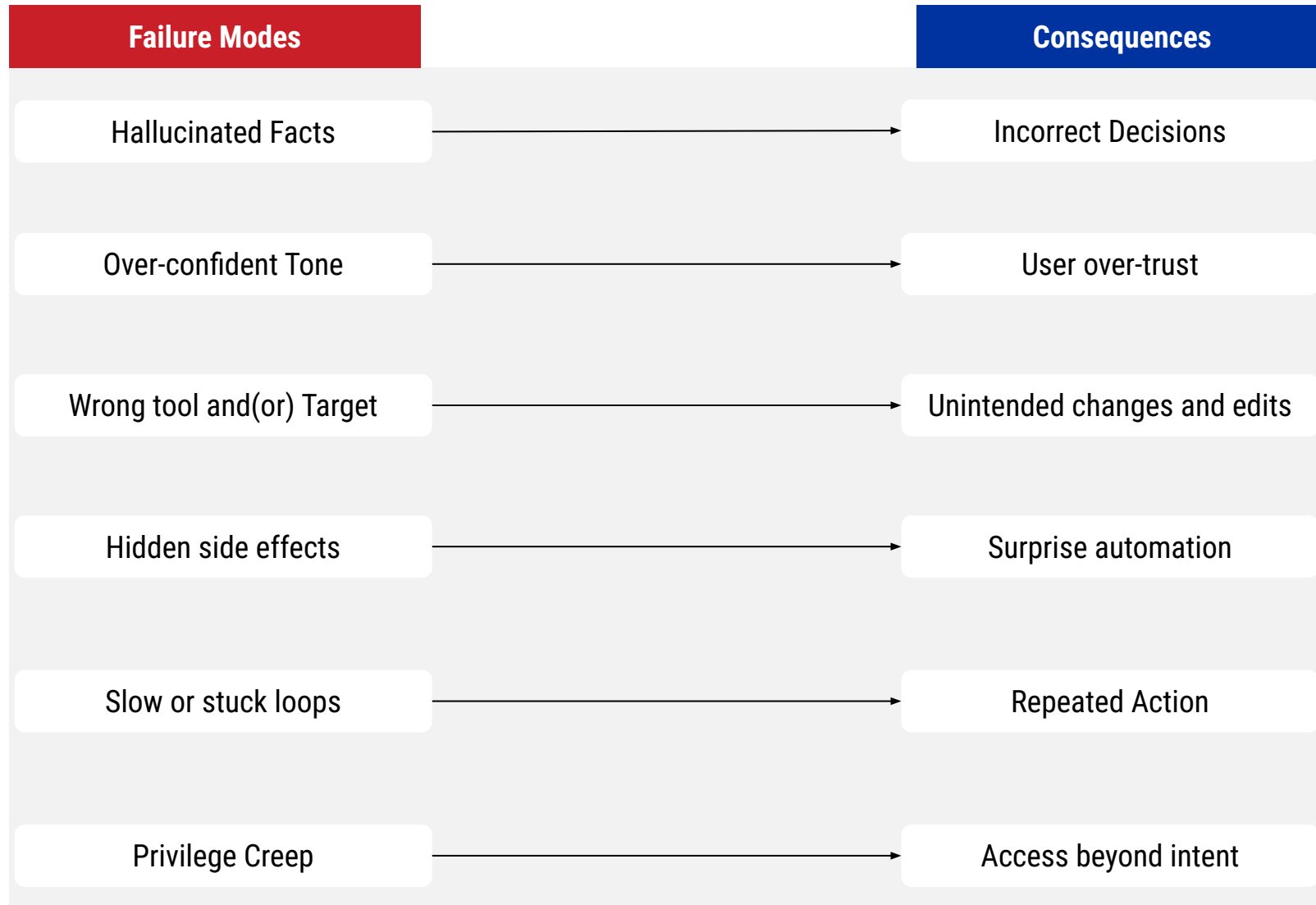
Conversational Agentic Experience layer is the product surface that translates *intent into safe action, by making it explicit, setting its boundaries, asking for confirmation at the right moments* to ensure safe, secure and guided conversation with agents.

We are now building experience layer on top of non-deterministic systems.

Design Goal: Minimize the cost of being wrong without sacrificing the speed and advantage of agent autonomy



Defensive UI | Failure Modes



Defensive UI | Failure Mode Examples

Hallucinated facts – Incorrect Decisions

A replenishment agent tells a merchandising manager “Vendor lead time for Kirkland paper towel is 5 days” and recommends lowering safety stock. Actual lead time is 12-14 days due to port congestion. This results in shelves going empty in multiple warehouses

Over-confident tone – user over-trust

A returns/refunds agent says, “Refund processes and completed”, but it only created a pending case waiting for manager override (high-dollar refund threshold). The customer is told to expect money in 24 hours; it does not arrive and escalations spike.

Wrong tool/target – unintended edits

The agent is asked to “update the item description for the 2-pack AA batteries”. It updates the single-pack listing instead(similar names/SKUs), creating customer confusion and returns(“I ordered 2-pack, got 1”).

Hidden side effects – Surprise Automation

A customer care agent asks, “Draft a response to this complaint”. The system also(without making it obvious) triggers a goodwill \$25 e-gift card

Slow or stuck loops – repeated actions

A “cancel order” action times out due to OMS latency. The agent tries repeatedly, generating multiple cancellation tickets

Privilege Creep – Access beyond intent

A customer asks, “Set Up a 15% off promo for SKU 123 for next weekend”. The agent pulls chain-wide pricing rules, MAP* exceptions, and executive-approved guardrails for other categories, then displays them in the UI. The user needed eligibility for the SKU only; the agent exposed sensitive pricing strategy beyond role and task.



Defensive UI | Design Principles

1

Safe defaults

Configure only safe data sources and tools as “defaults” in the UI

2

Explicit intent

Capture goals, constraints, success criteria, rephrase prompt

3

Scoped Autonomy

Bound what scopes and permissions the agent will have to act on an input

4

Continuous Control

Pause, cancel, edit or take over at any time by the user

5

Explain-before-action

Show plan of action, tools, and expected impact before agent takes action

6

Explicit trust controls

Show source citations, chain of thought traces for a given response



Defensive UI | Pattern Catalog

Pattern	Name	Description
P01	Plan Preview	Capture objective, constraints, and definition of done, get user approval before working on the user query. Suggest refined prompts if needed
P02	Bounding controls	Ability to bound what systems/data/actions the agent may use; block unapproved scope expansion. User explicitly provides access to any tool if required in addition to default ones
P03	Auto-Pause checkpoints	Automatically pauses at milestone or high-risk steps and ask for a go/no-go confirmation
P04	Evidence Chips and Citations	Attach sources/provenance to claims and recommendations; enable quick drill-down to supporting evidence
P05	Live thought trace	Show a live, readable thought trace during execution
P06	Interruptible Generation	Control that lets user interrupt the agent's response generation(or long running reasoning) at any time and immediately receive the best available partial answer. It acts as a human controlled "circuit breaker" for slow responses, suspected loops, or runaway verbosity
P07	Turn Lock (Single In-Flight request)	Prevent users from submitting a new prompt while the agent is generating, reducing race conditions, conflicting instructions, and overlapping tool actions; users should instead interrupt, edit, or cancel the current run before issuing the next request



Defensive UI | Failure Modes Vs Principles Vs Pattern

Failure Modes	Design Principles		Defensive UI Patterns					
Hallucinated facts – Incorrect Decisions	1 Safe Defaults 6 Explicit Trust controls	2 Explicit Intent 5 Explain-before-action	P01 Plan Preview	P02 Bounded Controls	P04 Evidence Chips and Citations			
Over-confident tone – user over-trust	6 Explicit Trust controls		P04 Evidence Chips and Citations					
Wrong tool/target – unintended edits	1 Safe Defaults 5 Explain-before-action	2 Explicit Intent 3 Scoped Autonomy	P01 Plan Preview	P02 Bounded Controls	P05 Live Thought Trace	P03 Auto Pause checkpoints	P07 Turn lock	
Hidden side effects – Surprise Automation	1 Safe Defaults 5 Explain-before-action	2 Explicit Intent 3 Scoped Autonomy	P01 Plan Preview	P02 Bounded Controls	P05 Live Thought Trace	P03 Auto Pause checkpoints	P07 Turn lock	
Slow or stuck loops – repeated actions	4 Continuous Control		P06 Interruptible Generation	P07 Turn lock				
Privilege Creep – Access beyond intent	1 Safe Defaults 3 Scoped Autonomy	2 Explicit Intent	P01 Plan Preview	P02 Bounded Controls				





RAI Controls

[> Return To Catalog](#)

RAI Controls - Dimensions

Accountability

Ensures that there is always a clearly responsible owner for what an agent does, including the ability to prove who approved actions, who operated the system, and who must respond when something goes wrong

Transparency

Makes the agent's behavior understandable and inspectable to the right audience by clearly communicating when AI is involved, what the agent can/can't do, what data and tools influenced an outcome, and providing traceable "why" and "how" explanations

Privacy

Protects personal and sensitive information across the full agent lifecycle by limiting what data is collected and shared, controlling how it's stored(including memory), preventing leakage through prompts/tools/logs

Inclusiveness

Ensures the agent works well for diverse users and contexts, including different languages, literacy levels, abilities, and access needs, so the experience does not unintentionally exclude or disadvantage certain groups

Fairness

Reduces the risk of biased or unequal outcomes across groups by preventing the agent from amplifying historical skews, avoiding proxy discrimination, evaluating performance across subpopulations and ensuring decisions don't systematically harm protected or vulnerable groups

Reliability

Ensures the agent performs consistently and safely under real world conditions-handling edge cases, uncertainty, tool failures, and attacks-while meeting defined quality, accuracy, uptime, and recovery expectations



RAI Controls (Prevent, Detect, respond)

Dimension		RAI Controls		
		Prevent	Detect	Respond
Accountability		- Named control owner per agent/workflow - Human in the loop as approval gates for high-impact actions - Least Privilege tool access	- End to end traceability - Policy compliance checks - Regular access reviews	- Incident runbooks and escalation paths - Evidence pack generation(logs, decisions, versions, approvals)
Transparency		- Mandatory disclosure UX(agent identity, autonomy level, when it takes action) - Model/Agent cards(purpose, limits, tool access, data sources) - Versioning discipline(agent config, prompts, policies, tool schemas)	- Provenance capture(sources used, retrieval snippets, tool outputs, references) - Explanation hooks(Reasoning, thought traces) - Change monitoring(alerts on prompt/policy/tool schema changes)	- User feedback(request explanation, request human review) - Content retract/refactor workflows(update incorrect outputs, notify impacted users) - Transparency audit reviews(periodic sampling of traces)
Privacy		- Data minimization(only required fields; structured inputs over raw text) - PII redaction/tokenization before inferences and before logging - Memory governance(opt-in, TTL/expiry)	- DLP scanning(inputs/outputs for PII/secrets) - Prompt and tool -injection detection - Privacy audit logging(who accessed what personal data and when)	- Secret rotation - Credential revocation for suspected leakage - Privacy incident response(user notification, containment, postmortem)
Inclusiveness		- Multi-lingual support - Inclusive UX patterns(clear error recovery, non-stigmatizing copy)	- Coverage testing across user personas - Disparity monitoring in user success metrics(completion rate, drop-off) - Feedback channels for inclusion harms	- Human hand-off path(when user can't proceed or need accommodation) - Content style remediation(inclusive language updates)
Fairness		- Bias risk assessment per use case - Dataset/RAG governance(source quality, representativeness) - Policy rules: no sensitive trait inference, restricted decisions in high stake areas)	- Output monitoring for protected-class references and harmful stereotypes - Periodic third-party/independent bias audits	- Escalation and human review for contested decisions/outcomes - Remediation documentation
Reliability		- Pre-deploy evaluation gates(task success, hallucination rates, tool correctness) - Action guardrails(allow/deny list, rate limits, sandboxing, dry-run) - Robust fallback design(safe defaults, circuit breaker, manual mode)	- Runtime monitoring (quality, latency, cost, tool failure rate) - Drift detection(response/context drift) - Red team and chaos testing(prompt injection, tool misuse)	- Automated rollback to last known-good - Degraded model-operations(read only, advisory-only, limited tools) - Post incident learning loops

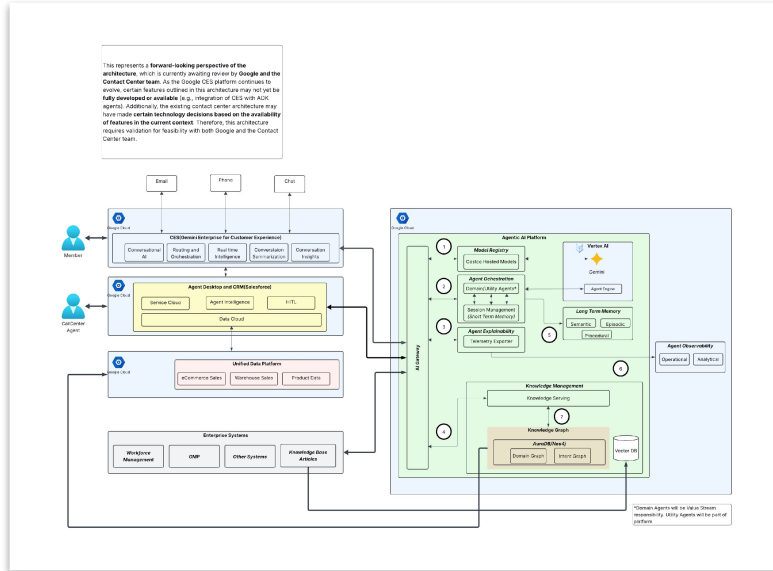




Architecture Recommendation for priority use cases

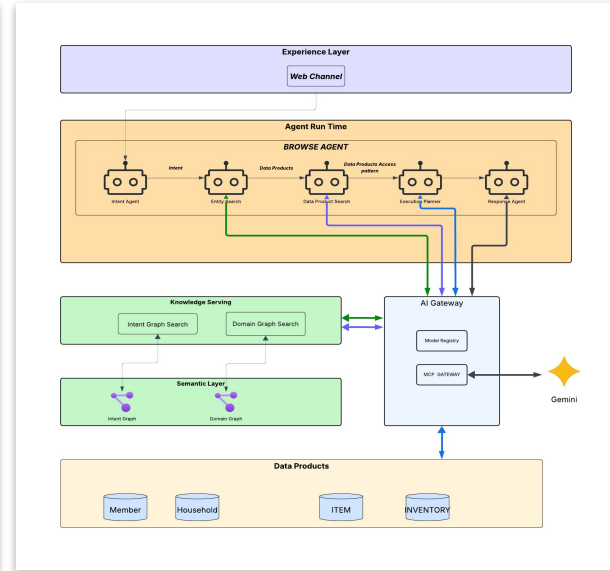
Architecture Recommendation

Contact Center



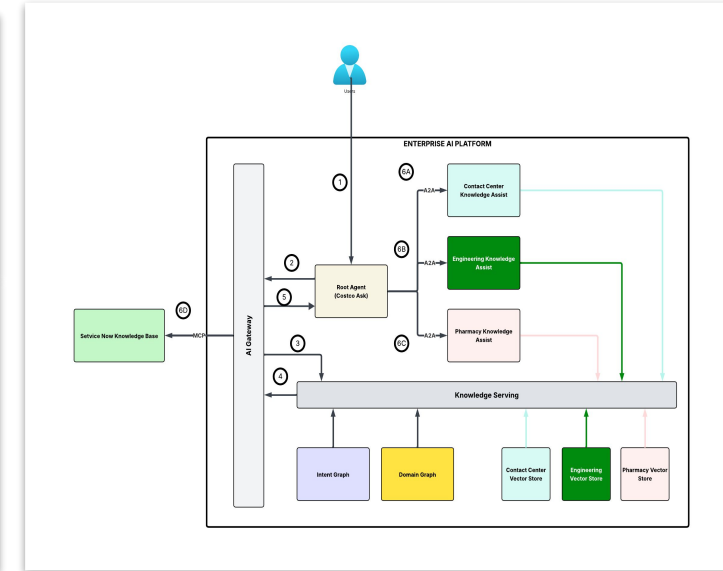
[Go to Lucid Charts](#)

Personalized Search



[Go to Lucid Charts](#)

Knowledge Assist



[Go to Lucid Charts](#)



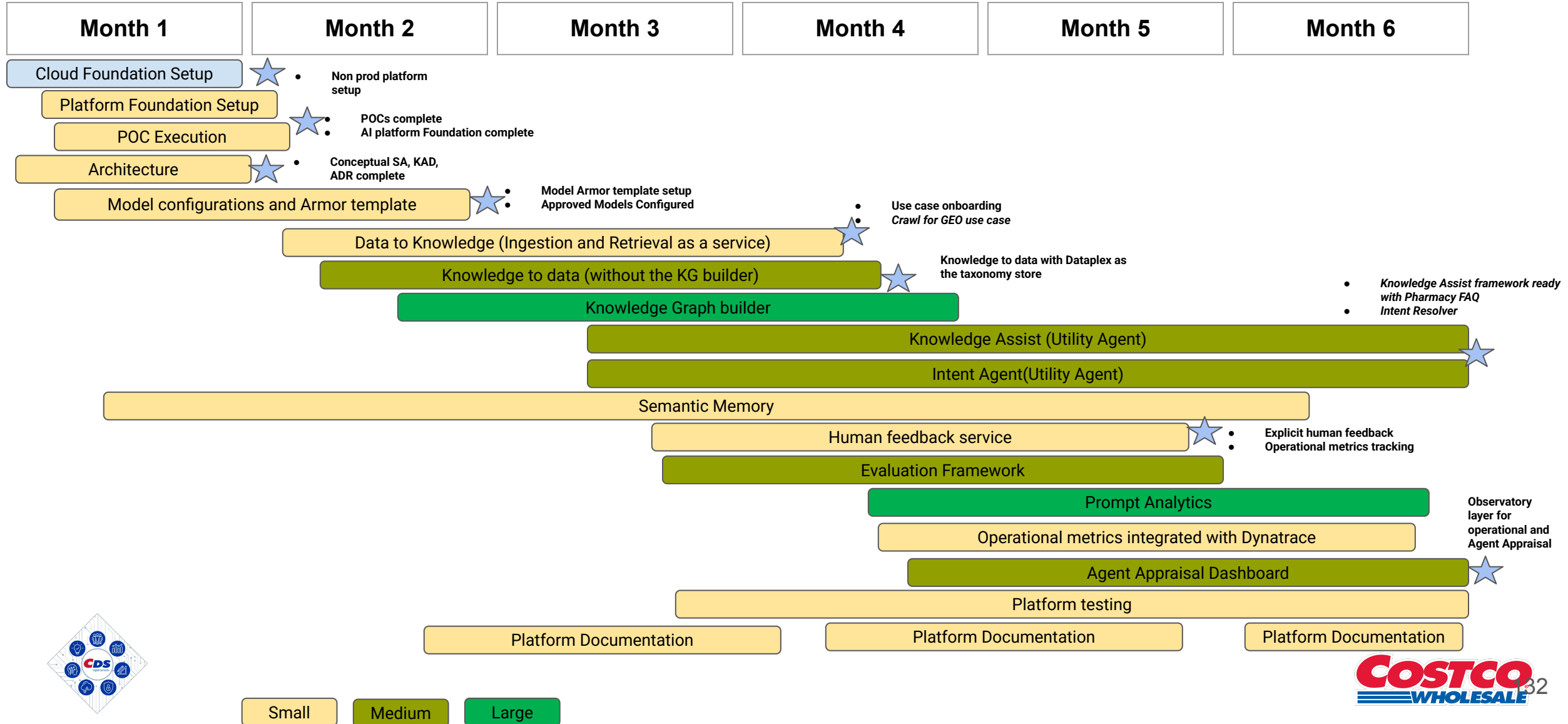


MVP Scoping and implementation plan

AI Platform MVP Definition & Options

Increasing in Scope of MVP as you move to the right			
LOW HIGH			
			
	MVP – Small (No Regret Foundational Capabilities)	MVP-Medium (Utility Agents with Agent Appraisal)	MVP-Large
Scope Definition	• Proof of Concept to validate the capability of APIGEE, Aura DB, Dataplex and Dynatrace(Integration for Operational Metrics) • Cloud and Infrastructure foundation to provision cloud services including GCP onboarding, IAM foundation, IaaS, Containers etc. • Platform foundation services for provisioning services required for Agentic Development e.g., Alloy DB, Agent Engine, Aura DB • Knowledge Layer capabilities to enable data to knowledge patterns for RAG based use cases • Configuration of approved Language Models in AI Gateway(APIGEE) to govern model access by the Agentic Layer • Agent Governance to enable capabilities to collect and process human feedback, collect and process operational metrics and integration with Dynatrace • Provide Semantic Memory As A Service to improve consistency and reduce cost for Agentic operations	MVP-Small + • Enhance knowledge layer to include management of taxonomy in Dataplex and manual build of intent graph(without Knowledge Graph Studio) • Utility Agents(Intent and Knowledge Assist) to provide foundational capabilities that help agentic systems retrieves, synthesizes, and contextualizes trusted enterprise knowledge to support users • Enhance Agent Governance to include design and development of Evaluation framework and build of Agent Appraisal framework and dashboard	MVP Medium + • Enable Prompt Analytics for to track and monitor interaction patterns with agentic systems and leverage the insight for agent performance and security improvement • Enable Knowledge Graph builder Service to enable users to manage and maintain domain graphs leveraging the domain ontology and taxonomy
Deliverables	• Completion of Cloud Foundation Setup • Completion of AI Platform Foundation Setup • Completion of PoC of required Agentic components • Ingestion as a service for RAG enablement • Knowledge Serving Layer (Hybrid Search and Semantic Search) • Human Feedback Service to capture human feedback • Semantic Memory as a service • Operational Metrics Dashboard • AI Platform Testing and documentation	• Intent Graph model created for Knowledge Assist • Knowledge Serving Layer enhancement to serve the intent graph • Knowledge Assist (Utility Agent) • Intent Resolver (Utility Agent) • Agentic AI Evaluation Framework and Agent Appraisal Dashboard	• Prompt Analytics Dashboard • Knowledge Graph Builder Service

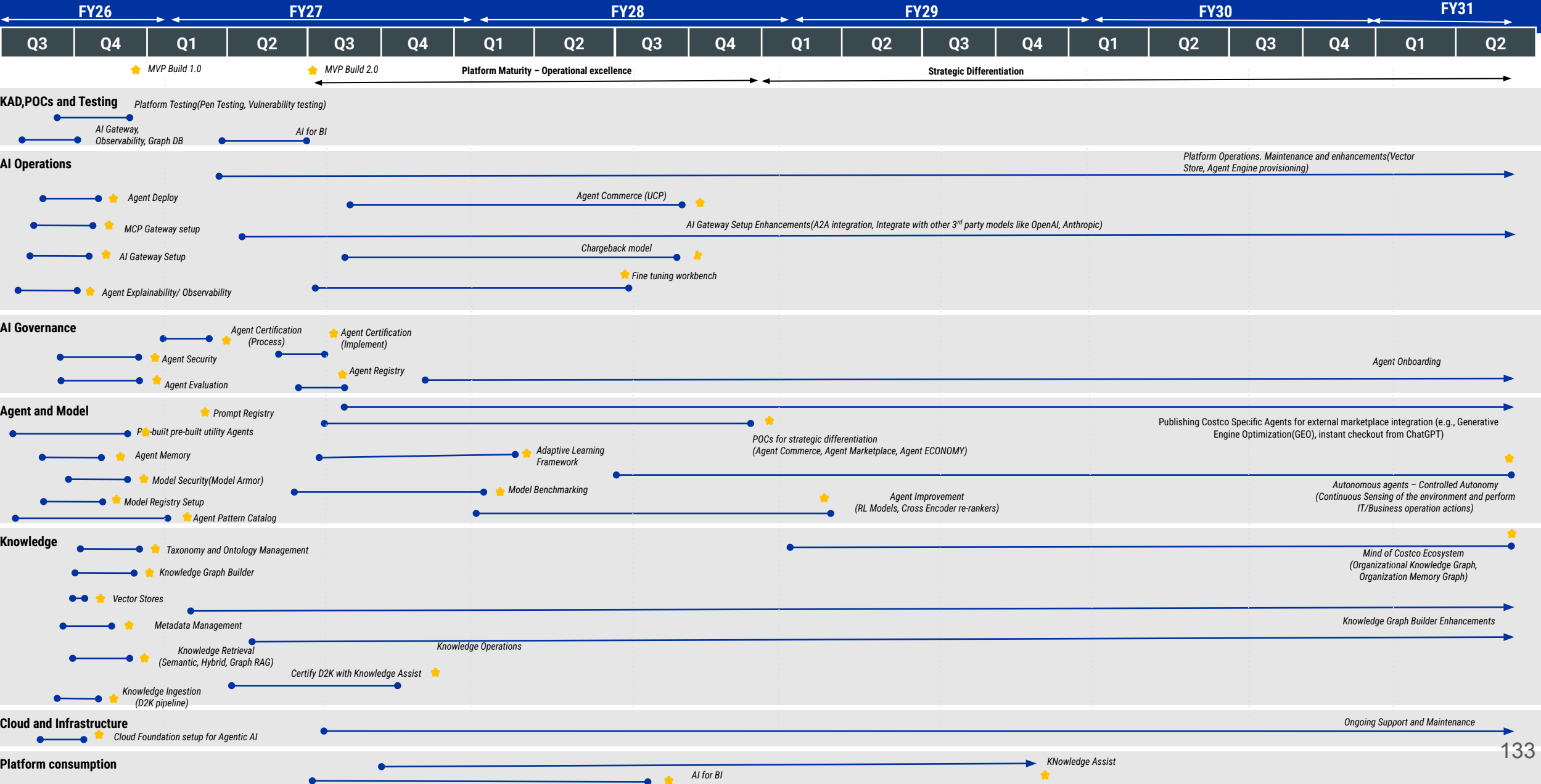
Platform MVP - Road Map (Scope Options)



Platform Capabilities Roadmap

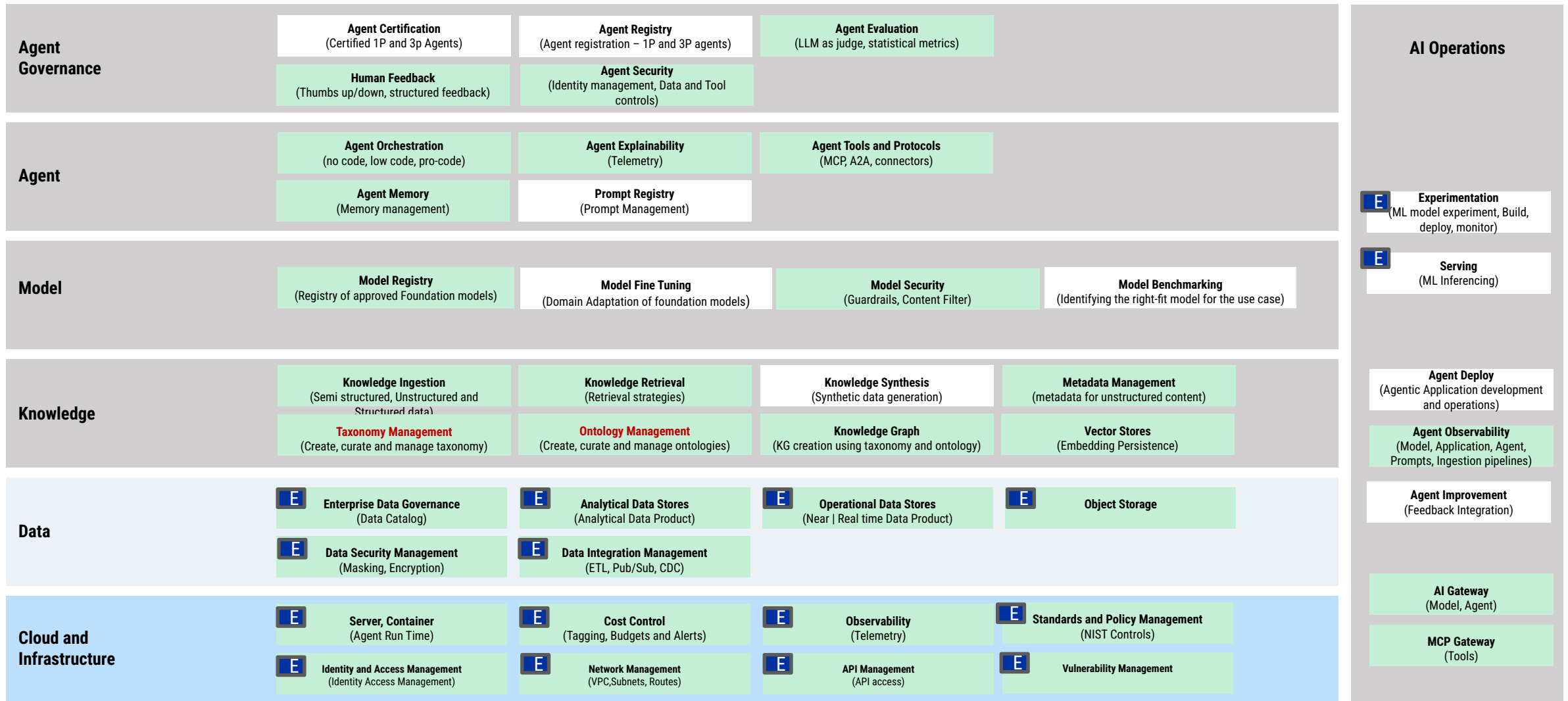
Agentic capabilities with repeatable patterns go into the platform

★ Milestones



Agentic AI Platform Capabilities – Contact Center

*Existing capabilities will need enhancements during use case enablement



Level 2 Capability

Level 3 Capability

EXISTING CAPABILITY

CLOUD & INFRA

PLATFORM

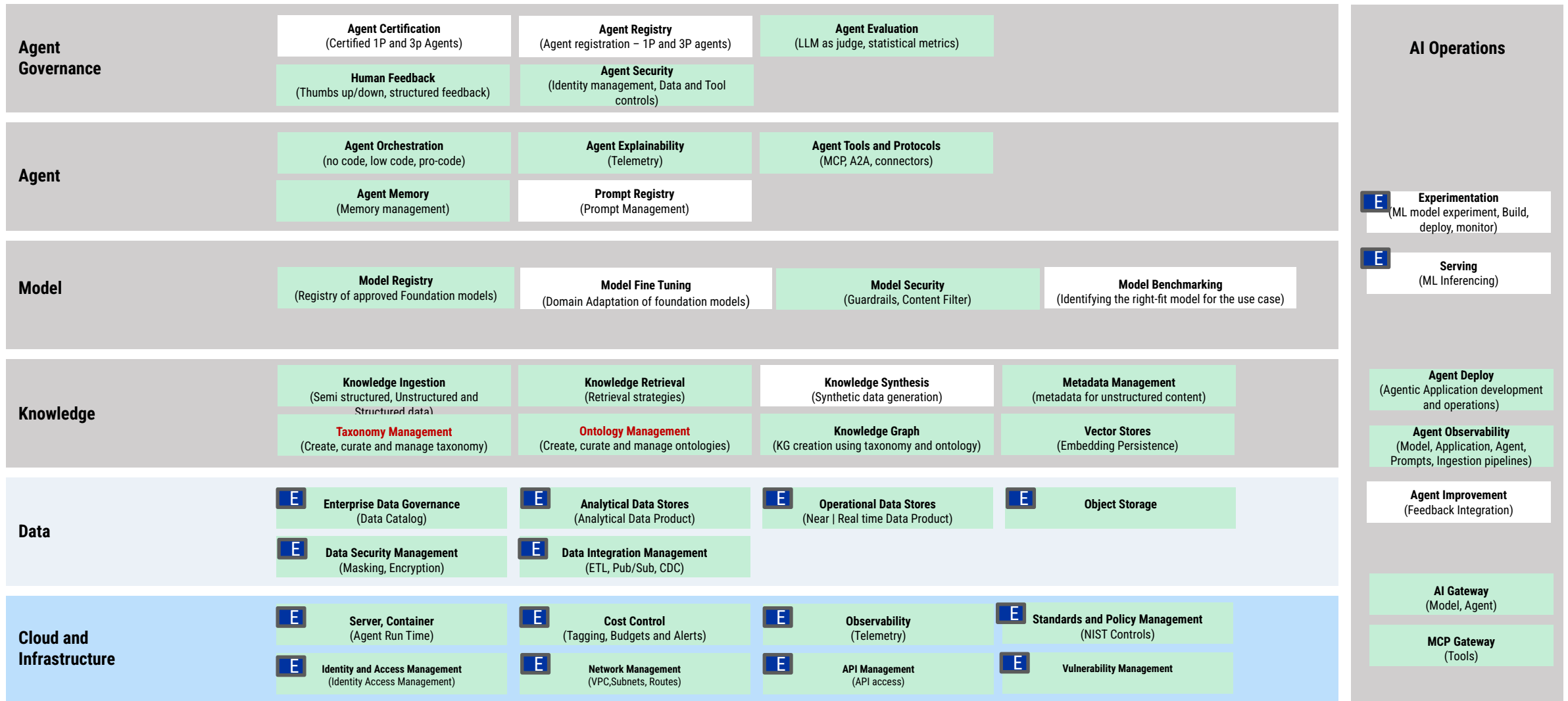
DATA

MVP Level 3 Capabilities



Agentic AI Platform Capabilities – Knowledge Assist

*Existing capabilities will need enhancements during use case enablement



Level 2 Capability

Level 3 Capability

EXISTING CAPABILITY

CLOUD & INFRA

PLATFORM

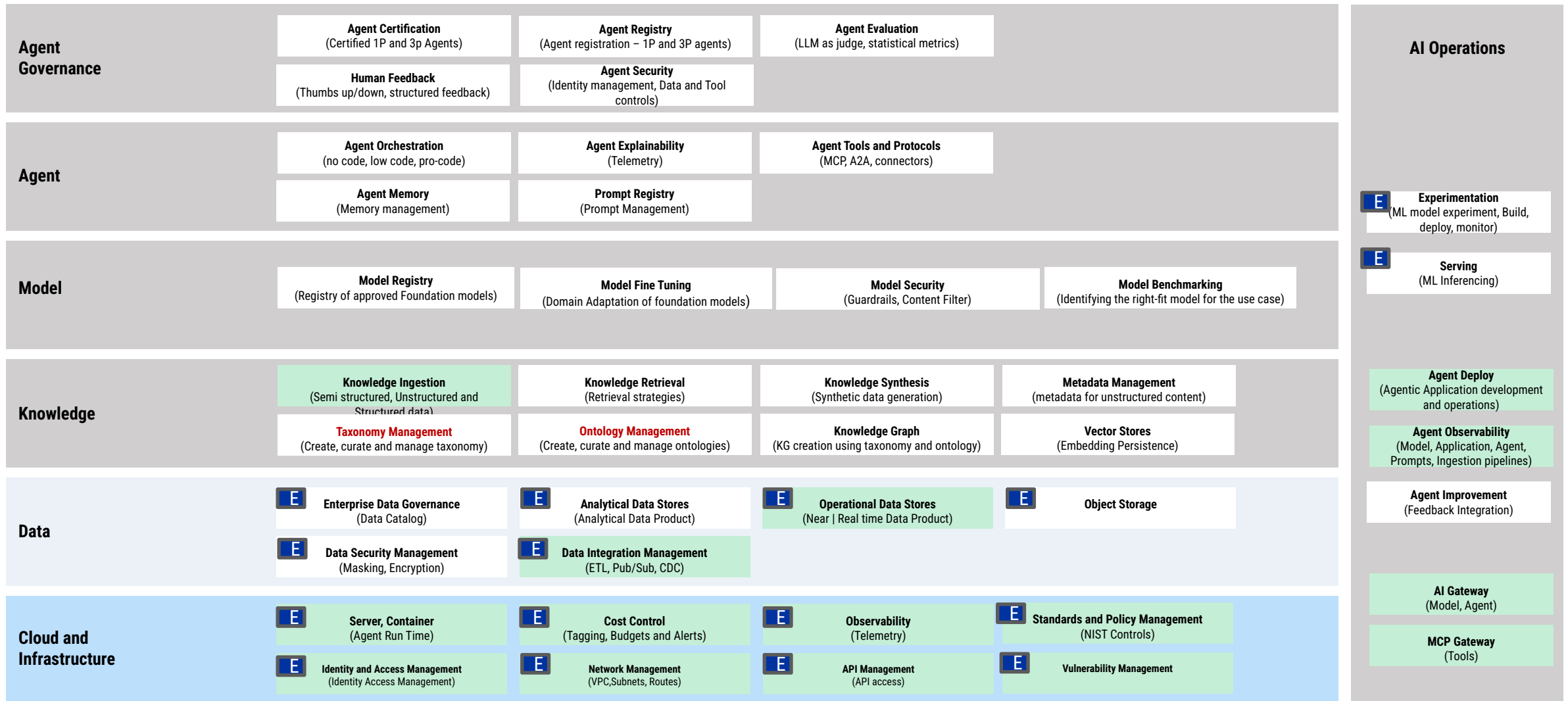
DATA

MVP Level 3 Capabilities



Agentic AI Platform Capabilities – GEO

*Existing capabilities will need enhancements during use case enablement



Level 2 Capability

Level 3 Capability

EXISTING CAPABILITY

CLOUD & INFRA

PLATFORM

DATA

MVP Level 3 Capabilities



Thank You





Appendix



Knowledge Layer Design principles

[Return to design principles](#)

Knowledge | Design Principles

Design Considerations	Design Principles	Why this principles	Anti-patterns/Not recommended (if any)
Controlled Vocabulary	Agentic Systems need to operate on a controlled vocabulary – a standardized, domain-aligned set of terms, definitions and semantic boundaries that describes the concepts, relationships and constraints of the concepts for a particular problem domain	Most of the organizations exhibit terminology drift; teams use different terms for the same concept. AI systems amplify this inconsistency-leading to retrieval errors, misaligned context, reasoning mistakes. A controlled vocabulary mitigates this by providing a standardized and canonical lexicon	Vocabulary must not be technology aligned names but business terminologies which is understood commonly across the organization (e.g Return Orchestration Service vs Return Management)
Context Building – Small World Approach	Agents need to operate on scoped world models – compact, goal-aligned representations of the specific problem space they are designed to solve-rather than relying on a large, all-encompassing world model.	Large monolithic world models will introduce noise and more context overlaps. Those will also be difficult to manage and scale. In order to effectively ground the agents, it is important to enable them with a tightly bound knowledge structure tailored to the domain and the problem space	Knowledge model should not be all-encompassing . We should develop use case aligned knowledge models bounded by the domain's rules, tools and the data sources.
Metadata curation and management	As part of context engineering, metadata must be curated, standardized and governed across knowledge assets with “domain experts(knowledge managers)” in loop	High-quality and high precision context retrieval is possible only when we can reduce the surface area of search through search optimization. Metadata filtering enables efficient filtering of the search space for precise context retrieval	Unstructured knowledge items must not be without metadata. Every unstructured knowledge item must accompany a business verified metadata information which will make the knowledge item AI consumable
Context isolation and retrieval	Once the scoped world model is defined, the retrieval layer must be designed to deliver precise, high-relevance context to the agents – selecting and tuning retrieval mechanisms that best match the domain and the knowledge structure. This layer will also enable a controlled Knowledge caching - enforcing TTLs, provenance, and invalidation rules so cached context remains accurate, auditable, and aligned to the domain's knowledge structure	A well-structured world model is only valuable if the agent can retrieve the right slice of it based on the provided input. Different problem spaces require different retrieval strategies. There is no single “perfect” retrieval mechanism; optimizing requires deliberate experimentation, evaluation and tuning	

Knowledge | Design Principles

Design Considerations	Design Principles	Why this principles	Anti-patterns/Not recommended (if any)
Context Shaping (Prompt Engineering)	The context served to the agents must be shaped-filtered, structured and reformatted, so that the final context window reflects only the most relevant tools, conversation history, memory fragments. The context window should not be overloaded with information that may cause context poisoning	Although the newer language models have large context window real estate, but that does not help much as language models can only attend to precise and structured context. The “lost in the middle” effect ignores middle portion of large context. Structuring of the context is also important because well structured context can leverage the prompt caching feature of language models	see slide
Knowledge Management Lifecycle	Enterprise knowledge(a major portion is unstructured) need to follow a knowledge management lifecycle – transforming human aligned content into AI aligned assets through systematic curation, metadata enrichment, normalization and context-overlap reduction before making it available for agentic systems	The existing unstructured knowledge content in PDFs, PPTs and word documents are authored for human consumption. That knowledge corpus is verbose, redundant, inconsistently structured, and often lack semantic metadata required for precise retrieval. Without a structured knowledge management lifecycle, the raw content flows directly into AI systems causing context overlap, retrieval drift and poor response quality	- Not cleansing the knowledge corpus from distractors - As part of the knowledge ingestion distractor contents(disclaimers, foot notes etc, which are not relevant content should be removed) - Not including the layout tags of the content- While storing the knowledge content in vector DB, store it along with the layout tags (e.g; tags for header, list, tables)
Incremental Knowledge ingestion	While developing Agentic systems, it is critical to design for incremental knowledge. The system must intercept change triggers and re-ingest the modified knowledge into vector stores. It is also important to log and version such changes for auditability	If an incremental ingestion pipeline is not designed, knowledge will soon become stale. Also, without proper logging and versioning, auditability of past responses and interactions will be lost	The knowledge ingestion pipeline should not be designed without a reconciliation mechanism. A reconciliation mechanism ensures that the knowledge is not lost when the knowledge is chunked and embedded in the vector DB. see slide

Knowledge | Design Principles

Design Considerations	Design Principles	Why this principles	Anti-patterns/Not recommended (if any)
Evaluation, Guardrails & Feedback Loops	Agentic systems must include guardrails, evaluation pipelines, and human/automated feedback loops to continuously refine reasoning, detect drift, and enforce safety & compliance.	Without evaluation and guardrails, agents degrade over time, hallucinate more, and produce unsafe behavior.	It is not recommended to use the same LLM which generated the output as the evaluator of the output. When using LLM as a Judge, choose the Judge LLM different from the LLM which created the output

Knowledge | Design Principles | Context shaping (Prompt Engineering)

Anti-Pattern	Description
Stuffing the context with arbitrary few shot examples	The context window of LLM should not be loaded with arbitrary examples . Selecting exemplars that are similar to the prompt is beneficial for performance. This is where we integrate the system with human feedback. As part of human feedback, we collect good set of prompt/response pair. At run time, based on prompt similarity, we bring in the similar examples in the context window
Free flow text within prompt	The content of the context should not be a free flow text. It is recommended to use delimiters to indicate distinct parts of the input. For example <pre>PROMPT = f"""</pre> Answer the question based on the provided rules and context <pre>## Rules ## 1.Rule#1 2.Rule#2 3.... ## Context ## {context} """</pre>
Context overloading with entire conversation history	The context window should not be loaded with all previous conversation summary. This will lead to hallucination and context limit exceptions. There are different strategies that we should follow to load the previous conversation summary. Those strategies are Sliding Window - Keeps a fixed number of recent messages (based on window_size), when the limit is exceeded, the oldest conversation is dropped Compaction - This strategy compresses older messages into summaries while keeping the most recent messages intact Semantic retrieval - This strategy allows to match the new input with older conversation history to bring only the relevant chain of past messages
Mixing dynamic and static part of the prompt	Inappropriate structuring of the prompt will lead to no use of the KV cache. In order to leverage PROMPT(KV) cache, static content of the prompt should be at the beginning of the prompt. Note: Different model providers have different limits on when KV cache can be used. Minimum cacheable prompt length: - Open AI models - 1024 tokens - Claude Opus 4.5 - 4096 tokens - Claude Opus 4.1, 4, Sonnet 4.5, 4 - 1024 tokens - Claude Haiku 4.5 - 4096 tokens - Claude Haiku 3 - 2048 tokens



Model Layer Design principles

[Return to design principles](#)

Model

Model layer provides controlled, governed and secure access to foundation models including embedding, test, image and video generation models. This layer adds the access constraint policies based on team/user role

Model Registry	Model Fine Tuning	Model Security	Model Benchmarking
Model registry refers to the set of approved models from different providers which are approved to be used by the organization. The models in this registry are exposed via the AI gateway. Model registry provides scoped access to the approved models.	Capability to train or adapt foundation models with domain-specific Costco data so the model internalizes the vocabulary, semantics, and constraints of the problem space.	Mechanisms to enforce safety constraints, prohibited topics, refusal behavior, and output filtering at the model level	Suite of services for testing and evaluating base models and custom models against a well- defined set of metrics and create benchmarks for business related functional areas.

Model | Design Principles

Design Considerations	Design Principles	Why this principles	Anti-patterns/Not recommended (if any)
Model selection based on use case (xLM)	Design must consider model routing that routes tasks to the most appropriate model based on domain, task complexity, language, and performance constraints.	Ensures the right model is chosen for the task, domain, modality, and business constraint; improves accuracy and cost-efficiency.	
Model inference acceleration	Uses quantization, batching to reduce latency and compute cost.	Reduces latency and cost through quantization, batching, and compilation—critical for real-time and high-volume use cases.	
Model distillation	Compresses large models into smaller ones without major loss in capability, improving performance and deployability.	Produces smaller, faster models without major performance loss; essential for scale, efficiency, and controlled deployment.	
Quotas and rate limits	A governance layer that enforces execution boundaries such as quotas, rate limits, refusal behavior, and policy controls to keep model usage predictable, compliant, and safe.	Prevents system overload, ensures fair resource usage, and maintains stable, controlled model execution in production.	
Benchmarking recipes	A structured evaluation framework with standardized metrics, datasets, and scorecards to assess model quality, performance, and safety before deployment.	Ensures consistent, repeatable measurement of model performance and protects against regressions during updates.	



Agent Layer Design principles

[Return to design principles](#)

Agent

Agents are a semantic model(e.g a generative language model) at the **core and the periphery makes it agentic**. The peripheral capabilities are planning and reasoning, tool calling and memory usage.

Agent Orchestration	Agent Explainability	Agent tools and protocols	Agent Memory	Prompt Registry
A highly customizable, low-code and pro-code , scalable framework with chain-of-thought reasoning, dynamic task decomposition and management. Agents are designed to collaborate seamlessly through an integrated memory system, and multi-agent collaboration is facilitated via a 3-layer orchestrator/super/utility agent topology.	The continuous stream of spans and traces that captures agent interactions, prompts, tool usage, latency, cost, errors and action outcomes from execution of the agentic flows providing observability into agent execution. This helps in monitoring, tracking of the agents continuously to ensure that they are aligned the goals and objectives.	Agent tools and connectors provides pre-built services that allows the agents to integrate securely to enterprise data and systems ecosystem (CRM, ERP, ITSM tool etc.)	A secure, governed and persistent layer that allows agents to store specific episodes of its interactions which can later be retrieved so that it can learn from past interactions. It stores key facts, preferences, actions an outcomes – across semantic, episodic and entity dimensions	A prompt registry is a centralized, version-controlled catalog where all prompt templates are managed and stored. It ensures that prompts are treated as first class artifacts, which are reviewed, tested, tagged and versioned. It becomes the single source of truth that tracks the evolution of prompts as they mature through review, testing and optimization

Agent | Design Principles

Design Considerations	Design Principles	Why this principles?	Anti-patterns/Not recommended (if any)
Role Based Agents (Macro Vs Micro Agents)	When translating a traditional workflow into an agentic flow, agent boundaries must be defined around roles or sub-roles – not around individual tasks. Each agent should embody a coherent functional responsibility that mirrors how work is naturally partitioned among human or system roles	Agentic AI inherit the distributed-systems characteristics like microservices architectures. Decomposing too broadly creates cross-domain coupling, increase handshakes and constraints autonomy. Decomposing too narrowly introduces excessive coordination overhead or chattiness. Role based decomposition strikes the right balance. It minimizes inter-agent dependencies, maintains contextual integrity, and ensures each agent can act coherently and independently within its defined responsibility domain	Task based decomposition of agents should be avoided. Task based decomposition creates coordination overhead, fragmented context, and governance sprawl, turning the system into a brittle workflow graph than an outcome-driven agent with clear domain boundaries Refer to next slide
Agent Identity	Every agent with an agentic system must possess a verifiable identity-issued, signed, and managed through trusted identity infrastructure-to ensure that all interactions, delegations and executions can be authenticated and trusted end-to-end. Identity must reflect trust boundaries: intra-domain agents are first-party identities; inter-domain agents require explicit trust agreements and no implicit trust across domains	Agentic ecosystems are distributed and involve dynamic, autonomous interactions. Without secure, non-repudiable identity, the system becomes vulnerable to impersonation, unauthorized tool invocation, and integrity breaches. Cryptographic identity(verifiable credentials, signed attestations) establishes a secure foundation of trust, access control, and auditability	We should not provide 3rd party agents with the tool access or model access credentials. All 3rd party agents must go through AI Gateway to access any internal tools like ServiceNow, GitHub etc. If we provide direct tool credentials to 3rd party agents, there will be no control on what those agents can do with the tool access.
Input and output guardrails	Agents must operate within clearly defined input and output guardrails(or the constitution of work) – validating, constraining, and shaping data entering and leaving the agent. This is critical to ensure safe, predictable, and policy-aligned behavior across the agentic system. <i>Guardrails should be applied based on agent risk tier</i>	Agentic systems are inherently dynamic and probabilistic. Without structured input/output constraints, agents may propagate harmful or malformed context or information to downstream receivers	• Avoid instrumenting guardrails within prompts, enforce guardrails outside prompts as a common control point. • Avoid using only LLM based guardrails, have a combination of LLM and rule based guardrails • Avoid having guardrails as the only source of security measure. For example, PII guardrail can be avoided if the data is already sanitized for PII. Every guardrail introduction adds to latency.

Agent | Design Principles

Design Considerations	Design Principles	Why this principles?	Anti-patterns/Not recommended (if any)
Stateless Agent Services with externalized memory management	Agent services should remain stateless delegating all forms of memory, transient state, checkpoints and conversational history externalized in a durable persistence layer. It is therefore critical to design the state model of the agents to ensure only relevant information is persisted	Unlike traditional stateless service calls, agentic systems rely on accumulated or shared context, intermediate reasoning to make accurate, contextual and policy aligned decisions	Anti-patterns for state design • Do not store everything in the state. State should store only essential, relevant data • Do not use complex objects for state persistence. Use simple data types(strings, numbers, booleans) that are serializable • State should not be un-scoped. Use clearly named prefixes to scope the state(e.g user, app, temp) • Avoid deep nesting of the states, when possible
Memory as a service	Agentic systems must externalize long term memory into a dedicated “Memory-as-Service” layer – providing standardized APIs for storing, retrieving and managing agent interactions, user preferences rather than embedding memory logic directly within individual agents	Memory is a foundational capability in an agentic architecture which enables every interaction to improved by leveraging experiences of past interactions. A centralized memory supporting semantic, episodic and procedural memory capabilities ensures consistency in governing, storing and retrieving of memory	Please see this slide
Own your prompts	Prompts must be treated as first-class, governable assets – authored, versioned, evaluated and maintained by the respective project team – rather than being embedded ad hoc within application code or use abstraction layers like DSPY	In an agentic system, prompts are the behavioral contract. They encode policies, role definitions, reasoning expectations, safety boundaries ad task logic. If prompts are scattered, un-versioned, or hidden within abstraction layers, it will be difficult to govern, scale the system.	It is not recommended to abstract the prompts. Owning the prompts enables: Better Control: Provides control to add precise instructions that agents need. Prompt Evaluation: Allows to test the prompts in a way we test code. Quick Iteration: Allows to do prompt iterations faster Better Transparency: Visibility into the decision boundary of the prompt(rules, few shot examples etc.)

Agent | Design Principles

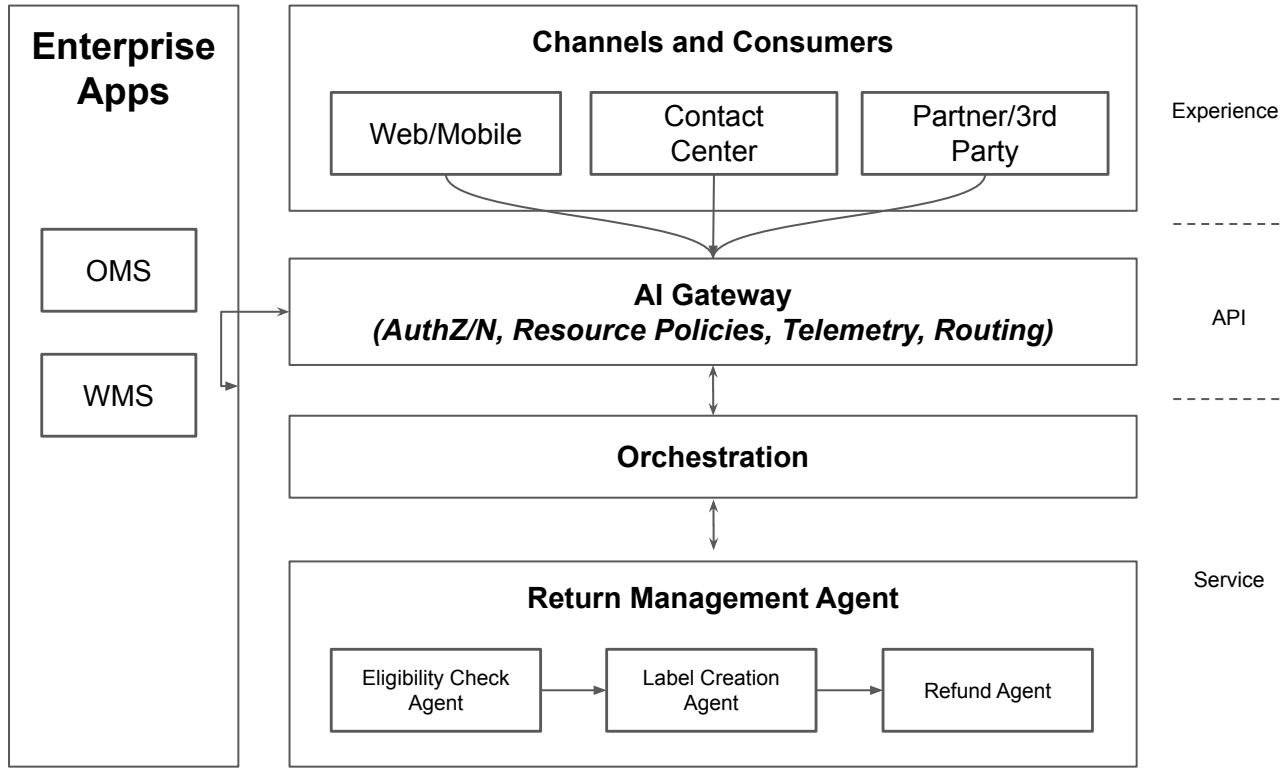
Design Considerations	Design Principles	Why this principles?	Anti-patterns/Not recommended (if any)
Interoperability across multi-agent systems	Agentic applications must be designed to interoperate seamlessly across heterogenous multi-agent systems-supporting shared protocols, common schemas, standardized messaging patterns, and portable capability definitions to ensure agents can collaborate seamlessly across frameworks and platforms	Enterprises are going to take a polyglot approach while leveraging agent development frameworks(e.g. LangGraph, ADK, AG2). Without a standard interoperability protocol, it will be difficult to enable shared communication between the agentic systems	
Integrated Telemetry	Telemetry must be a native, first-class capability of the agent framework – automatically capturing reasoning traces, tool calls, latency, cost and state transitions – so developers are not responsible for manually instrumenting agents through decorators. Manual instrumentation should be an exception when needed	Telemetry is a common concern or service within an agentic application. If left to developers, that may either forget or implement telemetry in an inconsistent way that may be difficult to manage for observability and analysis	Telemetry solutions must not use non-standard telemetry protocol. It should be OTEL compliant
Domain driven MCP development	MCP servers must be designed and deployed using Domain driven Design(DDD) principles – where capabilities are exposed along bounded contexts rather than as monolithic endpoints . This will enable multi-tenancy, capability isolation and seamless cross domain reuse	MCP is the kinetic core of the agents. If an MCP server exposes tools without domain boundaries, capabilities will blur resulting in duplication. It will also be difficult for agentic systems to discover authoritative capabilities within an organization	MCP server must not be one single monolith which hosts all the tools and capabilities in the organization. MCP server deployment strategy should follow 1. Domain driven 2. Trust boundary based 3. and(or) Data classification-based approach see slide

Agent | Design Principles

Design Considerations	Design Principles	Why this principles?	Anti-patterns/Not recommended (if any)
Data to model and Model to Data	In Agentic Architecture, we must accommodate both “data to model” and “model to data” patterns. Data to model is when we ingest the data from source systems and vectorize them in vector stores. Model to Data is when the model retrieves the data from its source.	Data to model pattern requires additional overhead of developing reconciliation mechanism to ensure that information is lost when the data travels to the model	Model to data should be prioritized over data to model pattern
Configurable Agents	Agent behavior must be configurable – allowing prompts, tools , policies and underlying models to be externally defined and dynamically adjusted – so that the agent can evolve without significant code changes as the AI landscape and enterprise requirements continue to evolve	The AI stack is undergoing continuous innovation. Tight coupling of the agent ecosystem makes the system brittle and unable to adapt quickly to leverage newer innovations	
Protocol aware agents	Agent frameworks must natively support standardized interaction protocols such as A2A, so that agents automatically produce protocol-compliant messages, actions, status and tasks without developers manually instrumenting the protocol logic. Where native support is not there, reusable protocol adapters to be provided.	As agent ecosystems expand, standardized protocols become essential for reliable cross-agent communication and orchestration. Making it part of the framework enables consistent implementation of the protocol standards	

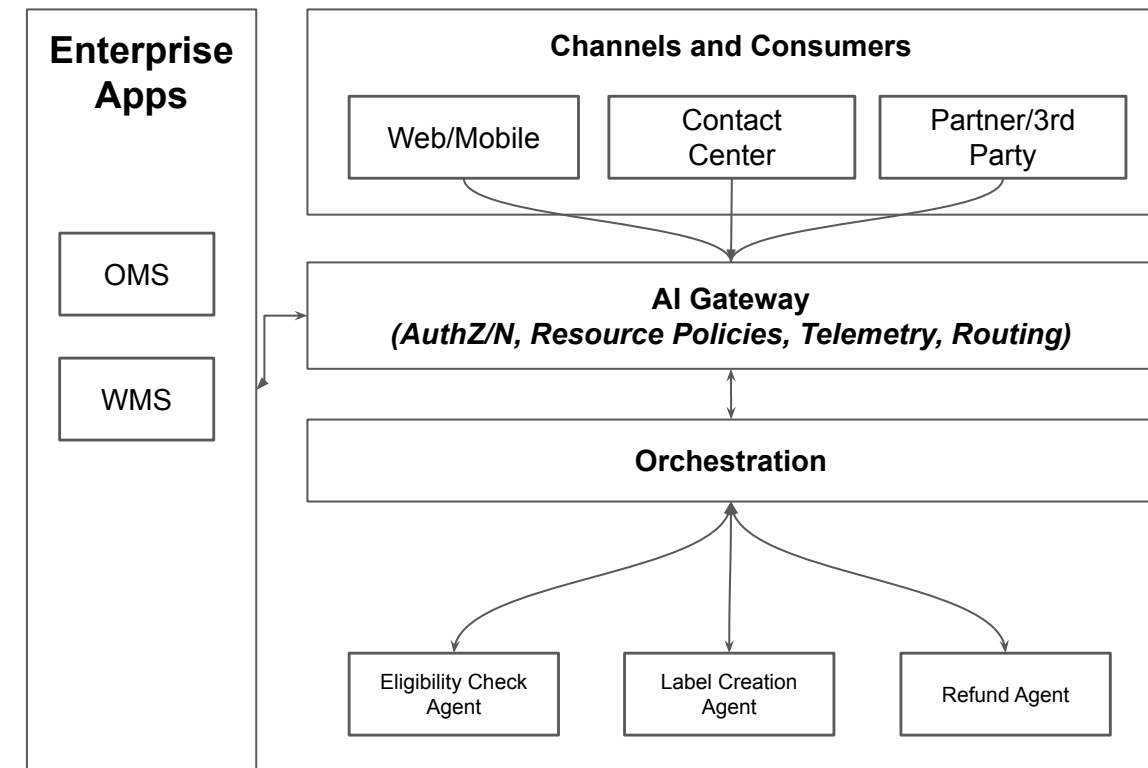
Agent | Design Principles | Macro Vs Micro Agents

Macro Agent Pattern



1. **Stable contract** : Consumers integrate to one outcome-driven API, so we can change internal decomposition without breaking clients
2. **Efficient Governance**: Easier to enforce least privilege, policy guardrails, audit logs and telemetry instead of replicating controls across many endpoints
3. **Better reliability and debugging**: End to end tracing is simpler, reduces hand-offs, improves latency

Micro Agent Pattern



1. **Endpoint Sprawl**: Clients must chain multiple micro calls, increasing coupling and chattiness across the network
2. **Inefficient controls**: Guardrails, auth, rate limits, and logging drift across services, expanding attack surface and weakening compliance policies
3. **Higher latency/more failure points**: Each hop adds latency, small changes cascade across handoffs, making the system brittle.

Agent | Design Principles | Memory Service

Anti-Pattern	Description
Memory as dumping ground	Memory must not be a “dumping ground” . Everything in the session context must not be dumped into memory. Only relevant facts, user preferences and decisions based on use case should be part of the memory
Last write wins	New memory must not overwrite old memory .Memory must consolidate new information with existing memories which allows memories to evolve as new facts and information are extracted
Synchronous write path	Memory write must not be synchronous . It should enable generation of memories asynchronously in the background, so that agent does not have to wait till the memory is generated
Schema less memory extraction	Memory extraction must not be an unstructured text blob without a defined schema . It should be schema driven; every memory item must conform to a defined schema.
Single global memory	Memory must not have one shared namespace for all users/tenants/domains/projects. It must be scoped to a particular namespace schema (e.g /org/domain/app/user)
Tight coupling of memory storage and services	Memory services must not be tight coupled with the underlying memory storage . Memory storage should be decoupled so that right-fit storage can be used based on memory type
Unbounded retrieval	Memory retrieval must not be unbounded . It should be based on intent-aware read policy to only include relevant memory in the conversation window
Versionless memory	Memory must not be without a version. All memory writes must be versioned which is immutable. This enables visibility into how memory has evolved.
Retention-less memory	Memory must not be created without a retention policy . All memory records should have an explicit retention policy(TTL/expiry) and retrieval preference for fresh memory items



Governance Layer Design principles

[Return to design principles](#)

Agent Governance

Agent governance is the set of policies, controls and operating processes that ensure AI agents are safe, compliant, reliable and cost-managed throughout their lifecycle.

Agent Certification	Agent Registry	Agent Evaluation	Human Feedback	Agent Security
Agent certification is the process of assessing agents against a capability maturity and agent readiness dimension. The capability maturity define the autonomy/agency level of the agent and the readiness dimension is measured by security, effectiveness and interoperability aspects. <i>(add lifecycle in design principle)</i>	Agent registry is SOR for all certified agents-capturing their identity, owner, purpose, versions, allowed tools/data, and policy constraints. Each agent is documented through a A2A compliant Agent Card, a standardized profile that captures the agent skills, capabilities, its hosting location etc.	Measurement systems that evaluate how well an agent reasons, retrieves, and acts. Evaluation frameworks (RAGAS, TruLens, custom scoring pipelines) assess: ❖ Factual accuracy ❖ Relevance & grounding of retrieved context ❖ Coherence of reasoning steps ❖ Correctness of tool use ❖ Policy alignment ❖ Overall task success Evaluation ensures continuous reliability and tracks drift over time	A structured human-in-the-loop (HITL) mechanism that guides agent behavior toward safe, aligned outcomes. Human input—approvals, feedback, corrections, and reinforcement—helps: ❖ Adjust agent decisions ❖ Update or correct memory ❖ Improve reasoning pathways ❖ Strengthen compliance and business alignment This creates a closed feedback loop for continuous refinement.	Agent security ensures every AI Agent treated like non-human entity with strong control over what it can access and what it can do. It starts with agent identity - each agent has a unique, verifiable identity used for authentication, authorization, and full audit logging of actions and tool calls. Access is governed by least privilege, granting only the minimum data, tools and permission required for the agents approved purpose, ideally with scoped and time bound credentials

Agent Governance | Design Principles

Design Considerations	Design Principles	Why this principles	Anti-patterns/Not recommended (if any)
Certification as pipeline	Agent governance should be a pipeline (dev->stage->prod) with clear “promotion” criteria. Certification is the gate that confirms an agent met required security and evaluation thresholds for its risk tier	Ensures the right model is chosen for the task, domain, modality, and business constraint; improves accuracy and cost-efficiency.	
Risk-tiered controls(policy profiles)	Not all agents need the same rigor. Define tiers and scale requirements for evaluation coverage, approval depth and security restrictions accordingly.	Treating all agents the same either slows delivery or increases risk; tiers focus rigor where impact and compliance exposures are highest.	
Standardized artifacts via Agent Card	Every agent must ship with an Agent Card that document's purpose, intended users, tool/data access, guardrails, evaluation results, known	Agent cards make discoverability and agent interoperability efficient	
Registry as the single source of truth with immutable versioning	The Agent registry should hold the canonical, versioned definition of each agent. Treat production agents as immutable releases-changes require a new version + recertification	Prevents agent sprawl , versioning enables auditability	
Identity-first security with least privilege by default	Each agent has a unique agent identity used for authN/authZ, policy enforcement, and audit logs. Grant least privilege(scoped tools, minimal data, time bound credentials) and prefer just-in-time access for sensitive actions	Strong identity and least privilege minimizes blast radius, prevents data over reach and improves compliance posture.	

Agent Governance | Design Principles

Design Considerations	Design Principles	Why this principles	Anti-patterns/Not recommended (if any)
Full traceability and auditability end-to-end	Every run should produce a trace: inputs, decisions, tool calls, outputs and policy decisions, linked to the agent version in the registry.	Traces provide explainability , support audits, and enable root-cause analysis and governance reporting	
Operational guardrails + kill switches	Governance should include runtime controls, rate limits, spend/usage budgets, escalation rules and the ability to immediately disable an agent or revoke credentials if something goes wrong.	Because agentic workflows are iterative and autonomous, errors can compound across steps which can trigger run away costs or unintended side effects. A kill switch enables immediate containment to prevent runaway behavior.	

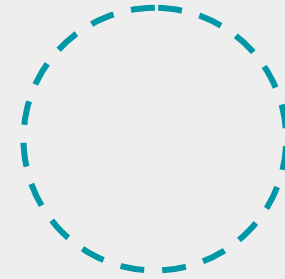
Additional Security Controls

Agent Governance | Design Principles | Agent Security

Category	Description
Input/Output Level Security Implement validation and sanitization processes to prevent malicious inputs and outputs from exploiting vulnerabilities in the multi-agent system.	Prompt Injection Prevention Methodology enabling input validation and injection detection based on user input queries. Input/Output Content Filtering Methodology defining malicious input/output categories and policies, applying rule-based and LLM-based filters to sanitize user inputs and model outputs Rate Limiting & Throttling Methodology defining rate limits and quotas, and monitoring and alerting on potential abuse. Input Context Length Limiting Methodology enabling input context length limiting, ensuring compliance with specified limits and managing oversized inputs.
Model Level Security Secure the LLM models against adversarial attacks and unauthorized manipulations to maintain system integrity and performance	Hard/Soft Constraints Content constraint rules in an application or API middleware to ensure safe and compliant content generation. (define the constraint rules and categories, such as what content should never be generated, model output filtering) Secure Model and Monitoring Deploy the models via HTTPS-only endpoints and restricting model access to authorized users. Model Documentation All approved models should have a model card which should clearly explain the limitations and boundaries of the model
Identity and Access Management Utilize robust authentication and authorization mechanisms to manage the access of critical components and data	Fine-grained RBAC/ABAC Comprehensive access control system using role-based and attribute-based access control (RBAC/ABAC) principles. Least Privilege Assign least privilege roles based on identities and continuously monitor & alert for privilege escalation. Access Revocation & Expiry Controls Practices enabling the creation, rotation and revocation of user credentials.

Agent Governance | Design Principles | Agent Security

Category	Description
Data Protection Encrypt data at rest and detect and filter sensitive data with retention policies.	Sensitive Data Protection Detect and manage sensitive data based on usage requirements to ensure data privacy and compliance Data Formatting and Filtering Based on the use case, cleanse the data of any sensitive information before being used as input to agentic systems Incident Response Structured process for identifying, analyzing, and addressing security and privacy vulnerabilities discovered
Log monitoring and governance Continuously monitor and analyze logs for anomalies, ensuring compliance with security policies and regulatory requirements	Comprehensive Logging Architecture Dedicated logging system to enable runtime visibility and monitoring resource consumption of AI applications. User Activity Monitoring Log requests and responses per use and associate logs with session IDs or API tokens for auditing purposes.
Security testing and assurance Conduct regular security assessments, including penetration testing, to identify vulnerabilities and enhance overall system security posture	Security Testing in SDLC Regular security testing as part of SDLC, and findings must be documented and remediated. (testing strategy to include SAST, DAST, and OSS security assessments) Change Management in SDLC System changes and updates of critical components are documented, tested, reviewed and approved. System Integrity with Multiple Environment Controls and separated environment to protect the integrity of AI development and production environments.



Operations Layer Design principles

[Return to design principles](#)

AI Operations

AI Operations is the operational layer that keeps agentic systems reliable, governed and continuously improving in production. It combines an AI Gateway to standardize access across models/agents/tools, Agent Deploy to manage prompts and agent development with CI/CD discipline, agent observability to capture end-to-end telemetry and finally enable agent improvement through closed loop learning using human feedback

AI Gateway	MCP Gateway	Agent Deploy	Agent Observability	Agent Improvement
AI Gateway is a centralized control plane that sits between agent applications, model providers and MCP servers. It enforces governance and operations at runtime - authentication, rate limits, policy checks, logging/tracing and spend/budget controls. It standardizes access to model and context which makes cost, security and reliability manageable at scale. In addition, it also enables semantic caching and usage analytics	A control plane or a proxy layer that manages how agents can securely access tools, data, and other resources through the MCP servers. It acts as the policy enforcing middle layer – validating requests, brokering capabilities, and ensuring that every tool invocation follows enterprise rules around safety, observability and authorization	Agent Deploy enhances traditional DevOps practices by adding or enhancing checks and balances that are unique to developing Agentic systems. This includes adding prompt scanning, MCP tool scanning to the pipeline	Ability to collect, analyze and observe how agents behave in production. It captures end-to-end telemetry across agent runs, model calls, and tool interactions-including latency, errors, quality signals, and cost	Continuous cycle of making agents more accurate , safe, and cost efficient based on real production signals. It uses evaluations, user/human feedback to facilitate reinforcement learning for continuous agent improvement

AI Operations | Design Principles

Design Considerations	Design Principles	Why this principles?	Anti-patterns/Not recommended (if any)
Prompt as code	Prompt templates must live in source control with change history	Prompt as code means treating prompts as first class software artifacts as they control agent behaviour and agentic system outcomes. Like code, Prompts can introduce bugs, regressions and security issues, so they must be versioned, tested and released through CI/CD principles	It is not recommended to hard code prompts within the agent code. Prompt and Code management should be decoupled
Model routing	Route by use-case and risk tier: cheaper models for low-risk tasks , stronger models for complex/high impact tasks with automatic fallback	Different tasks within an agentic system has different needs, some require deep reasoning, others need faster response, structured output etc. routing helps pick the right model for the right job to optimize cost, latency and quality	
Cost controls	Budgets, rate limits, caching and unit cost tracking are built into platform observability so that spend scales predictably	Agents can generate unbounded work through multi step reasoning, tool loops, infinite retries, long context and silently multiply token spend and API calls. Resource and budget restraints helps to prevent runaway cost and brings cost predictability	
Feedback driven iteration loops	Capture human feedback and production traces, convert them into test cases, and continuously improve prompts and agent output efficiency	It turns production usage into measurable improvements capturing user feedback, tool outcomes, and evaluation signals that identifies failure modes enabling shipping targeted fixes with confidence	
Centralized policy enforcement	All model traffic flows through the gateway so auth, content/PII policies, tool/MCP allowlists, and tenant controls are enforced consistently	A centralized policy layer gives a single control point to enforce least privilege, data boundaries, tool permissions and action constraints across agentic systems, so that governance is consistent even as new agents are added	